

Để tránh trường hợp tài khoản khóa học bị kick, ban hoặc xóa mà không biết nguyên nhân tại sao, các bạn vui lòng đọc kỹ điều khoản sử dụng dịch vụ cung cấp bởi tác giả Hỏi Dân IT.

QUY ĐỊNH CHUNG

Chính Sách Bảo Vệ Bản Quyền và Sử Dụng Dịch Vụ Hỏi Dân IT

Chào mừng bạn đến với Hỏi Dân IT! Khi bạn quyết định đầu tư vào sự phát triển cá nhân thông qua các khóa học của chúng tôi, bạn không chỉ nhận được quyền lợi tốt nhất mà còn có trách nhiệm bảo vệ tài nguyên và nội dung mà chúng tôi cung cấp.

1. Bảo Vệ Nội Dung Khóa Học

- Mọi hành động phát tán, truyền bá hoặc cổ vũ cho việc can thiệp, phá hoại hoặc xâm nhập trái phép vào dữ liệu khóa học của Hỏi Dân IT là nghiêm cấm.
- Việc phát tán nội dung các bài học ra bên ngoài mà không có sự đồng ý của Hỏi Dân IT sẽ bị coi là vi phạm nghiêm trọng, dẫn đến hủy bỏ tài khoản và có thể bị xử lý theo pháp luật liên quan đến vi phạm bản quyền.

2. Sử Dụng Tài Khoản

- Mỗi tài khoản chỉ dành cho một người sử dụng. Việc hai người hoặc nhiều hơn cùng sử dụng một tài khoản khi bị phát hiện sẽ dẫn đến việc tài khoản đó bị xóa khỏi hệ thống mà không cần thông báo trước.
- Nếu bạn và bạn bè có nhu cầu sử dụng chung khóa học nhằm mục đích tiết kiệm chi phí, vui lòng liên hệ trực tiếp với chúng tôi để thảo luận về các gói đăng ký nhóm phù hợp, điều này sẽ đảm bảo bạn vẫn giữ được quyền lợi hợp pháp và hợp lý.

3. Chia Sẻ Khóa Học

- Chia sẻ khóa học cho người thứ ba mà không có sự cho phép của Hỏi Dân IT là vi phạm các điều khoản sử dụng.

Việc này không chỉ vi phạm quyền lợi bản thân mà còn vi phạm pháp luật bản quyền.

Hành vi này sẽ dẫn đến hủy bỏ tài khoản và các biện pháp pháp lý nghiêm ngặt khác.

4. Trách Nhiệm Của Khách Hàng Đối Với Thông Tin Cá Nhân

- Khách hàng cần cung cấp thông tin chính xác và trung thực khi đăng ký và sử dụng dịch vụ tại Hỏi Dân IT. Việc cung cấp thông tin không chính xác có thể ảnh hưởng đến quyền lợi và khả năng truy cập dịch vụ của bạn.
- Khách hàng phải có trách nhiệm tự bảo quản thông tin cá nhân và tài khoản, bao gồm nhưng không giới hạn ở địa chỉ email và mật khẩu đăng nhập. Trong trường hợp mất tài khoản email liên kết với khóa học do lỗi cá nhân, Hỏi Dân IT không chịu trách nhiệm giải quyết hoặc hỗ trợ khôi phục tài khoản.
- Chúng tôi khuyến khích khách hàng thực hiện các biện pháp bảo mật cần thiết như sử dụng mật khẩu mạnh và cập nhật thường xuyên, cũng như hạn chế chia sẻ thông tin đăng nhập với người khác để tránh rủi ro mất quyền truy cập và vi phạm bản quyền.

Chính sách này nhằm bảo vệ quyền lợi của khách hàng cũng như duy trì một môi trường học tập công bằng và chất lượng.

Cảm ơn bạn đã hợp tác và tuân thủ những quy định này để cùng nhau xây dựng cộng đồng Hỏi Dân IT ngày càng phát triển.

TÀI LIỆU KHÓA HỌC “Nest.JS với TypeScript/MongoDB Siêu Dễ”

Tác giả: Hỏi Dân IT & Eric

Version: 7.0

Note cập nhật:

- Update source code theo từng video (v7.0)
- Update chapter 0 (v.6.0)
- Bổ sung #116: giới thiệu fullstack Nest.js/Nextjs (v6.0)
- Thêm hướng dẫn sử dụng source code (final) của khóa học (video #0.5) (v5.0)

Chính Sách Bảo Vệ Bản Quyền và Sử Dụng Dịch Vụ Hỏi Dân IT	0
Chapter 0: Bắt buộc xem - Không bỏ qua chương học này	7
#0.1 Hướng Dẫn Sử Dụng Khóa Học Hiệu Quả	7
#0.2 Hướng Dẫn Quyền Truy Cập Tài Liệu Khóa Học	9
#0.3 Hướng Dẫn Quyền Truy Cập Source Code	10
#0.4 Hướng Dẫn Tải Source Code Theo Từng Video	12
#0.5 Demo Kết Quả Đạt Được Khi Kết Thúc Khóa Học	13
#0.6 Yêu Cầu Của Khóa Học	14
#0.7 Cách Dùng Udemy - Hỗ Trợ Hỏi Đáp Q&A	15
#0.8 Về Tác Giả	16
Chapter 1: Tổng quan về Nest.JS	17
#1. Nest.JS là gì ?	17
#2. Create A New Project	19
#3. Hello world	20
#4. Kiến trúc của dự án Nest.JS	22
Chapter 2: NestJS và Typescript	23
#5. Typescript Decorators	23
#6. Sử dụng Decorator với Nest.JS	25
#7. Mô hình Router với Nest.JS	27
#8. Controllers	28
Chapter 3: Inversion Of Control	31
#9. Inversion Of Control (IoC)	31
#10.1 Khái Niệm Modules	33
#10.2 Dependencies Injection (DI)	35
#11. Injecting a dependency	37
Chapter 4: NestJS và MVC	38
#12. Ví dụ về controller/service	38
#13. Template View Engine	38
#14. Mô hình MVC	39
#15. Vấn Đề tồn đọng với SSR ?	39
Chapter 5: Connect Database	41
#16. Lựa chọn Database	41
#17. Cài đặt MongoDB	42
#18. Sử dụng MongoDB với NestJS	43
#19. ENV Variables	44
Chapter 6: Restful API	46
#20. Generate resources	46
#21. Create Schema (Model)	47
#22.1 Create A User	48
#22.2 Hash User's Password	49
#22.3 DTO - Data Transfer Object	49
#22.4 Pipe	50
#22.5 Validation	50

#23. Get User by Id	51
#24. Update a User	51
#25. Delete a User	52
Chapter 7: Stateful và Stateless	53
#26. Setup Backend Test	53
#27. Debug NestJS Applications	54
#28. Stateful Application	56
#29. Cơ chế hoạt động của stateful app	57
#30. Ưu, nhược điểm của Stateful ?	58
#31. Mô hình stateless	59
#32. Ưu, nhược điểm của Stateless	60
Chapter 8: JWT - Json Web Token	61
#33. JWT là gì ?	61
#34. Phân loại token sử dụng	62
#35. Giới thiệu về Passport.JS	64
#36. Local Strategies với NestJS	65
#37. Nestjs Guard	67
#38. LocalGuard với Passport	69
#39. Sử dụng JWT	69
#40. Implementing Passport JWT	70
#41. Enable authentication globally	70
#42. Disable Guard	71
#43. Fix bugs và tổng kết	72
Chapter 9: Tư duy phân tích database	74
#44. Giới thiệu đề bài	74
#45. Phân tích model & relationship	75
#46. Thiết kế model	76
Chapter 10: Mongoose Plugins	79
#47. Timestamps plugin	79
#48. Soft-delete plugin	80
#49. Query Builder	81
#50. Setup dự án frontend	82
#51. CORS là gì ?	83
#52.1 Làm sao để bypass CORS ?	85
#52.2 Setup Cors NestJS	86
Chapter 11: Modules Company	87
#53. Generate Modules Company	87
#54. Bài Tập Create Company	87
#55. Update User Type (JWT)	88
#56. Passing Req.user	90
#57. Bài Tập Update Company	91
#58. Bài Tập Delete Company	91
#59. Query with Pagination	92
#60. Giới Thiệu Interceptor	94

#61.1 Transform Response	95
#61.2 Customize Message (Decorator)	96
#62. Version APIs	96
Chapter 12: Modules User	97
#63. Update User Schema	97
#64.1 Bài Tập CRUD Users	98
#64.2 Hướng Dẫn Bài Tập CRUD Users	104
#65. Demo Chức năng login	104
#66. API login	105
#67. Set Cookies	106
#68. Bài Tập API Get Account (F5 Refresh)	106
#69.1 API Refresh Token (Part 1)	108
#69.2. API Refresh Token (Part 2)	109
#70. Bài Tập API Logout	110
#71.1 Test giao diện frontend (Part 1)	110
#71.2 Test giao diện frontend (Part 2)	111
Chapter 13: Modules Job/Resume	112
#72. Bài Tập Tạo Module Jobs	112
#73. Bài tập CRUD Jobs	113
#74. Hướng Dẫn Bài tập CRUD Jobs	118
#75. Giới Thiệu Upload files	119
#76. Validate Upload Files	120
#77. Destination Upload File	121
#78. Bài Tập Update Company with Image	123
#79. Test Giao Diện Frontend Upload File	124
#80. Bài tập Tạo Modules Resumes	126
#81. Bài tập CRUD Resumes	127
#82. Hướng Dẫn Bài tập CRUD Resumes	134
#83.1 Fetch Data with Ref	134
#83.2 Test Giao Diện Frontend	134
Chapter 14: Modules Permission & Roles	135
#84. Bài tập Tạo Modules Permission & Roles	135
#85. Bài tập CRUD Permissions	137
#86. Hướng Dẫn Bài tập CRUD Permissions	142
#87. Bài tập CRUD Roles	142
#88. Hướng Dẫn Bài tập CRUD Roles	146
#89. Fix bugs	146
#90. Nest Lifecycle Event	148
#91. Bài tập Tạo Sample Data	150
#92. Hướng Dẫn Bài Tập Tạo Fake Data	150
#93. Test Giao Diện Frontend	151
#94. Roles Guards	154
Chapter 15: Module Subscribers	155
#95. Bài tập CRUD Subscribers	155

#96. Giới thiệu về gửi email với Node.js	156
#97. Template Email	157
#98. Setup Tài khoản/Server gửi Email	159
#99. Nestjs - Mailer	161
#100. Gửi Email với template	165
#101. Fetch Subscribers by skills	167
#102. Test giao diện frontend	168
#103. Cron job	169
#104. Tự động gửi email với cron job	170
#105. Nhận Xét về Dự Án Thực Hành Frontend	170
Chapter 16. Giới Thiệu Kiến Thức Nâng Cao	171
#106. Authorization	171
#107. Helmet	173
#108. Rate Limiting	173
#109. Swagger	174
#110. Healthchecks	178
#111. Docs Whole App Backend	179
#112. Fix Bugs Còn Tồn Động	180
#113. Upgrade NestJS Version	181
#114. Build Docker	184
#115. Nhận Xét Về Khóa Học	185
#116. Tự Học Fullstack Nest.js/Next.js (Extra)	185
Đánh Giá (Review/Rating) Khóa Học	186
Lời Kết	188

Chapter 0: Bắt buộc xem - Không bỏ qua chương học này

Giới thiệu về khóa học và tác giả, cũng như demo kết quả đạt được sau khi kết thúc khóa học này.

#0.1 Hướng Dẫn Sử Dụng Khóa Học Hiệu Quả

Bạn vui lòng "xem video lần lượt" theo trình tự. Vì khóa học như 1 dòng chảy, video sau sẽ kế thừa lại kết quả của video trước đó.

1. Dành cho học viên "có ít thời gian"

Nếu bạn vội, cần học nhanh, hoặc "bạn đã biết rồi", thì "vẫn xem video, cơ mà không cần code theo".

Lưu ý: vẫn xem qua tài liệu khóa học để biết "video hướng dẫn gì".

Đã "Không xem video", thì cần "đọc giáo án".

Có như vậy mới biết khóa học nó làm cái gì.

2. Dành cho học viên "thông thường"

Nguyên tắc:

- Xem video lần lượt
- Xem video kết hợp với giáo án. Bạn không cần take note, vì những điều quan trọng đã có trong giáo án

- Bạn vui lòng code theo video.

Nếu bạn "code theo ý bạn", vui lòng "không hỏi khi có bugs".

Câu chuyện này giống như việc bạn đi khám bệnh, nhưng không tin lời bác sĩ

=> Nếu bạn giỏi, bạn làm luôn bác sĩ, còn đi khám bệnh làm gì.

- Bạn có thể "code theo ý bạn muốn", sau khi "đã kết thúc khóa học"

- Nếu bạn có thắc mắc (hoặc có ý tưởng/nhận thấy bugs), take note lại, bạn hỏi, rồi mình giải đáp.

Chứ không phải là "tự ý làm theo điều các bạn muốn".

Vì đa phần, các bugs trong khóa học mình đã fix hết rồi.

Nên là yên tâm để học theo bạn nhé.

3. Về cách code.

Bạn vui lòng code theo video, từ cách đặt tên biến, hàm. Vì mình đã tuân theo "convention tối thiểu" khi bạn đi làm đấy.

4. Về bài tập thực hành

Đối với bài tập thực hành, bạn cứ code theo cách bạn hiểu, và kết hợp với "search Google, stackoverflow..."

KHÔNG DÙNG CHATGPT. Đây giống kiểu chưa học "phép tính", mà đã "dùng máy tính".

Nên nhớ 1 điều, trước 2023, không có chat gpt, thì mình học như thế nào ?

Khi bạn đã đi làm, bạn có quyền dùng cái gì bạn thích, còn với beginner, hãy biết say NO với CHAT GPT.

tương tự bạn dạy con bạn:

học lớp cấp 1: không chịu học tính nhẩm => đưa luôn máy tính cho nó. Rồi máy tính ra, là không biết làm phép tính

còn với học sinh cấp 2, 3 : dùng máy tính tùy thích

#0.2 Hướng Dẫn Quyền Truy Cập Tài Liệu Khóa Học

Lưu ý: học viên cũ đăng ký khóa học trước ngày 01/11/2024, hoặc học viên tham gia dưới hình thức Udemý Business, vui lòng inbox [Fanpage Hỏi Dân IT](#) để được hỗ trợ.

1. Truy Cập Tài liệu khóa học

Bạn inbox qua [Fanpage Hỏi Dân IT](#) theo cú pháp sau:
(thay **tên khóa học**, và tên **email học viên** của bạn vào nhé)

Cấp quyền truy cập tài liệu khóa học "TÊN-KHÓA-HỌC"
Email học viên: yourEmail@gmail.com

Ví dụ:

bạn muốn truy cập tài liệu khóa học "Nest.js Basic",
và email học viên của bạn là : myEmail@gmail.com

Thì cú pháp tin nhắn của bạn là:

Cấp quyền truy cập tài liệu khóa học "Nest.js Basic"
Email học viên: myEmail@gmail.com

2. Cách sử dụng hiệu quả

Trong quá trình sử dụng tài liệu, nếu bạn thấy chỗ nào chưa đúng, hoặc link tham khảo trong tài liệu không sử dụng được (dead), các bạn chủ động inbox qua Fanpage Hỏi Dân IT (hoặc gửi email tới địa chỉ ads.hoidanit@gmail.com) để mình update tài liệu các bạn nhé

Link fanpage Hỏi Dân IT: <https://www.facebook.com/askITwithERIC/>

Các tiêu đề bài học có chữ "Extra", có nghĩa là chủ đề mở rộng (thông thường sẽ là các topic nâng cao mình làm thêm để đáp ứng nhu cầu của học viên)

#0.3 Hướng Dẫn Quyền Truy Cập Source Code

Lưu ý: học viên cũ đăng ký khóa học trước ngày 01/11/2024, hoặc học viên tham gia dưới hình thức Udemy Business, vui lòng inbox [Fanpage Hỏi Dân IT](#) để được hỗ trợ.

1. Source Code Full Cả Khóa

Yêu cầu để thực hiện video này: bạn cần có hiểu biết cơ bản về Git.

Nếu bạn chưa biết gì về Git, vui lòng xem video này cho biết, rồi thực hành sau.

Nếu chưa biết gì về Git, vui lòng xem nhanh [tại đây](#)

Bước 1: Tài khoản Github

Trường hợp 1: Bạn đã có tài khoản Github

Case 1.1: Email đăng nhập vào Github, chính là email đăng ký học viên.

=> chúc mừng bạn, bạn không cần làm gì thêm, chuyển qua bước 2.

Case 1.2: Email đăng nhập vào Github khác với email đăng ký học viên

=> bạn vui lòng tạo mới một tài khoản Github, sử dụng email đăng ký học viên để đăng nhập

Trường hợp 2: Bạn chưa có tài khoản Github

Tạo mới tài khoản Github, sử dụng "chính xác" email đăng ký học viên làm tên đăng nhập vào Github

Kết thúc bước này, đảm bảo rằng tài khoản đăng nhập vào Github, chính là email đăng ký học viên

Bước 2: Đăng Ký Thông Tin Source Code

Bạn inbox qua [Fanpage Hỏi Dân IT](#) theo cú pháp sau:
(thay tên **khóa học**, và tên **email học viên** của bạn vào nhé)

Cấp quyền truy cập source code khóa học "TÊN-KHÓA-HỌC"
Email học viên: yourEmail@gmail.com

Ví dụ:

bạn muốn truy cập source code khóa học "Nest.js Basic",
và email học viên của bạn là : myEmail@gmail.com

Thì cú pháp tin nhắn của bạn là:

Cấp quyền truy cập source code khóa học "Nest.js Basic"
Email học viên: myEmail@gmail.com

Bước 3: Check Email nhận thông báo

- Kiểm tra hộp thư đến (Inbox) lẫn Spam cho chắc chắn.
- Lời mời vào source code khóa học chỉ có tác dụng trong 7 ngày.
- Hướng dẫn cách accept vào source code

#0.4 Hướng Dẫn Tải Source Code Theo Từng Video

Mục đích video này ra đời, để phục vụ trường hợp: bạn muốn có source code của cả khóa học (mà không cần code), hoặc muốn có source code ứng với video bạn đang xem.

Lưu ý 1: Bạn cần xem video [#0.3](#) để được cấp quyền truy cập source code.

Lưu ý 2: Bạn xem video này cho biết, chỉ sử dụng khi nào cần, còn source code thực hành khóa học sẽ được mình cung cấp tại các video tiếp theo (cứ xem video lần lượt) bạn nhé.

Link source code final **dự án Nest.js Basic**, xem [tại đây](#)

Lưu ý: Bạn vẫn cần xem video để biết cấu hình các tham số môi trường (nếu có)

1.Cách xem source code theo từng video thực hành

Lưu ý: lựa chọn đúng branch của code (trong khóa học này là master)

Download source code cuối mỗi bài giảng

Tại mỗi video bạn xem, mình có để link để download nhanh source code.

//todo (mình họa với video [#40](#))

#0.5 Demo Kết Quả Đạt Được Khi Kết Thúc Khóa Học

Video demo: <https://youtu.be/z0B7I-P7tUs>

Một vài highlight của khóa học:

- Hiểu rõ, thực hành và áp dụng được ngôn ngữ Typescript cho dự án NestJS
- Xây dựng tư duy Backend với tính “mở rộng” cao, bằng cách học & sử dụng framework NestJS
- Thực hành dự án viết backend cho website clone itviec.com (website đăng tin/ứng tuyển việc làm)

#0.6 Yêu Cầu Của Khóa Học

Các yêu cầu cần biết trước khi thực hành khóa học:

- Cần có sự hiểu biết cơ bản về cú pháp Typescript. Nếu bạn chưa biết, có thể học nhanh [tại đây](#)
- Cần có kiến thức về MongoDB (sử dụng với Mongoose). Nếu chưa biết, bạn có thể tham khảo khóa học học backend Restful API (Express) [tại đây](#)

Một vài yêu cầu kiên quyết:

- **Cài đặt môi trường Node.JS** : trong khóa học này, máy tính của mình dùng **version Node.JS 16.20.0**
<https://nodejs.org/download/release/v16.20.0/>
Để hạn chế lỗi, các bạn nên dùng version trên.(v16.20.0)
Có thể dùng NVM để sử dụng nhiều version của Node.JS trên cùng máy tính
Về NPM, xem [tại đây \(windows\)](#)
- **Cài đặt Git**: các thao tác với Git và sử dụng Github, mình không hướng dẫn.
Bạn nào chưa biết dùng Git, xem nhanh [tại đây](#)
- **Cài đặt Visual Studio Code**: đây là công cụ dùng để code

#0.7 Cách Dùng Udemy - Hỗ Trợ Hỏi Đáp Q&A

Lưu ý: không bỏ qua video này. Xem để biết cách sử dụng Udemy, cũng như các đặt Q/A khi cần hỗ trợ (support)

1. Sử dụng trên máy tính

Xem hướng dẫn tài liệu chi tiết [tại đây](#)

- [Cách bắt đầu sử dụng khóa học](#) (bắt buộc xem)
- [Cách đặt câu hỏi cho khóa học](#) (bắt buộc xem)
 - Hướng dẫn cách sử dụng Q&A
 - Hướng dẫn cách liên hệ Instructor qua Message
- [Cách sử dụng phím tắt](#)
- [Take note trực tiếp trên video đang xem](#)

2. Sử dụng trên điện thoại

Udemy có hỗ trợ ứng dụng trên điện thoại Android/IOS

Xem hướng dẫn tài liệu chi tiết [tại đây](#)

#0.8 Về Tác Giả

Mọi thông tin về Tác giả Hỏi Dân IT, các bạn có thể tìm kiếm tại đây:

Website chính thức: <https://hoidanit.vn/>

Youtube “Hỏi Dân IT” : <https://www.youtube.com/@hoidanit>

Tiktok “Hỏi Dân IT” : <https://www.tiktok.com/@hoidanit>

Fanpage “Hỏi Dân IT” : <https://www.facebook.com/askITwithERIC/>

Udemy Hỏi Dân IT: <https://www.udemy.com/user/eric-7039/>

Nếu bạn muốn nói chuyện với mình (giao lưu trao đổi võ công :v), có thể xem mình livestream trực tiếp tối thứ 2 & thứ 5 hàng tuần trên [Youtube Hỏi Dân IT](#)

Chapter 1: Tổng quan về Nest.JS

Giới thiệu tổng quan về Framework Nest.js và trả lời câu hỏi, tại sao chúng ta lại cần Nest.JS, từ đây viết chương trình Hello World với Framework này.

#1. Nest.JS là gì ?

Tài liệu:

Trang chủ: <https://nestjs.com/>

Tài liệu về Nest.js: <https://docs.nestjs.com/>

Github: <https://github.com/nestjs/nest>

1. What:

- Nest (NestJS) là 1 framework dành cho Server-side applications, chạy với Node.js. (tức nó là backend đấy :v)
- Mặc định sử dụng Typescript, tuy nhiên vẫn có thể code javascript với nestjs (not recommended)

Nestjs khác Nextjs các bạn ha :v

Nest (cái tổ chym), là **framework chuyên làm backend** (sử dụng typescript)

Next(kế tiếp), là **framework tích hợp React phía server, tức là nó chuyên làm frontend.**

Nó có thể làm cả backend nữa, cơ mà, có nhiều nhược điểm đấy :v

2. Why: tại sao dùng Nest.js, trong khi đã có các framework như Express, Fastify...

- Nest được xây dựng để đảm bảo tính "hiệu quả" (efficient) và "khả năng mở rộng" (scalable) của app backend
- Nest.js mặc định sử dụng **Express**, tuy nhiên, có thể lựa chọn **Fastify**, tùy thuộc vào nhu cầu của bạn
- **Bằng cách sử dụng Typescript, Nest tuân theo:**
 - + **OOP** (Object Oriented Programming - lập trình hướng đối tượng)
 - + **FP** (Functional Programming), tương tự như function component (khi dùng với React.js)
 - + **FRP** (Functional Reactive Programming) (tương tự rxjs của angular)

=> sử dụng Nest.js khi muốn xây dựng ứng dụng backend có tính mở rộng và đảm bảo hiệu năng (ngoài ra còn có khả năng 'maintain' code).

=> Nest.js được lấy cảm hứng từ Angular (highly testable, scalable, loosely coupled, and easily maintainable applications)

Nếu bạn lựa chọn Angular thay vì React, thì tương tự, lựa chọn Nest.js thay vì code Express thuần.

Nest.js sẽ cung cấp bộ khung framework (architecture - kiến trúc), việc của chúng ta là sử dụng bộ khung đấy để build sản phẩm mình muốn.

#2. Create A New Project

Tài liệu: <https://docs.nestjs.com/#installation>

Về tài liệu cũ hơn (ứng với version cũ hơn), thay đổi đường link url:

<https://docs.nestjs.com/v8/>

Trong khóa học này, sử dụng: <https://docs.nestjs.com/v9/>

Lưu ý: môi trường cài đặt Node.js 16.20.0

Bạn vui lòng sử dụng version nodejs 16.20.0 xuyên suốt khóa học này (để hạn chế lỗi không cần thiết)

Link download: <https://nodejs.org/download/release/v16.20.0/>

Cài đặt Nest CLI : **npm i -g @nestjs/cli@9.4.2**

CLI (command line interface), là công cụ cung cấp câu lệnh. Bạn gõ câu lệnh, thay vì code trực tiếp :v

1. Clone từ git (BẮT BUỘC)

Version Nest.js sử dụng là v9.x

Clone source code [tại đây](#)

2. Tạo mới từ đầu

Chỉ tạo project mới từ đầu, khi bạn đã có khả năng tự đọc tài liệu và thực hành Nest.js

Các bạn lưu ý, không tự tạo project khi thực hành khóa học này. Chỉ tạo project, khi và chỉ khi bạn đã học xong khóa học.

Những trường hợp “tự tạo project theo cách của bạn” sẽ không được mình support trong quá trình học.

Sử dụng câu lệnh sau để tạo mới project:

nest new project-name

#3. Hello world

1. Chạy dự án

```
"start": "nest start",  
"dev": "nest start --watch"
```

câu lệnh start dùng để chạy ứng dụng Nest.js (sau khi đã build)

--watch (tương tự như nodemon), dùng để theo dõi sự thay đổi của files
-> auto restart lại server

=> Để chạy dự án (môi trường dev), sử dụng câu lệnh: npm run dev

Mặc định, ứng dụng Nest.js chạy trên port 3000
có thể thay đổi tham số port này tại file main.ts

2. Cấu trúc dự án NestJS đã xây dựng

- Thư mục dist: thư mục build của Nest.js. Cần build, vì code Nest.js dùng typescript
-> dịch ra javascript thì Node.js mới thực thi được (server)/hoặc browser thực thi.

- Thư mục test: viết code test

Trong phạm vi của dự án này, bỏ qua thư mục test, và các file có tiền tố spec.ts

- Thư mục src: nơi chứa source code dự án

+ với Nest.js, src được chia theo module (phân tích cụ thể ở video tiếp theo).

+ hiện tại, dự án có 1 module duy nhất là "module app", bao gồm "app controller, app module và app service"

+ **file main.ts là file chính.** ứng dụng Nest.js sẽ chạy file này đầu tiên (tương tự file server.js trong khóa học restful api)

- Các file phụ:

- + .eslintrc.js : config eslint, giúp check code khi viết theo cú pháp typescript
- + .gitignore : khai báo các file không muốn git quản lý
- + .prettierrc : config prettier, giúp viết code "đẹp hơn" => hiểu đơn giản là format code
- + nest-cli.json : config Nest.js khi chạy CLI/build
- + package.json/package-lock.json : quản lý các thư viện cài đặt cho dự án
- + tsconfig.json : khai báo cách dịch code typescript thành javascript (môi trường development)
- + tsconfig.build.json: khai báo cách dịch code typescript thành javascript (khi build với môi trường production)

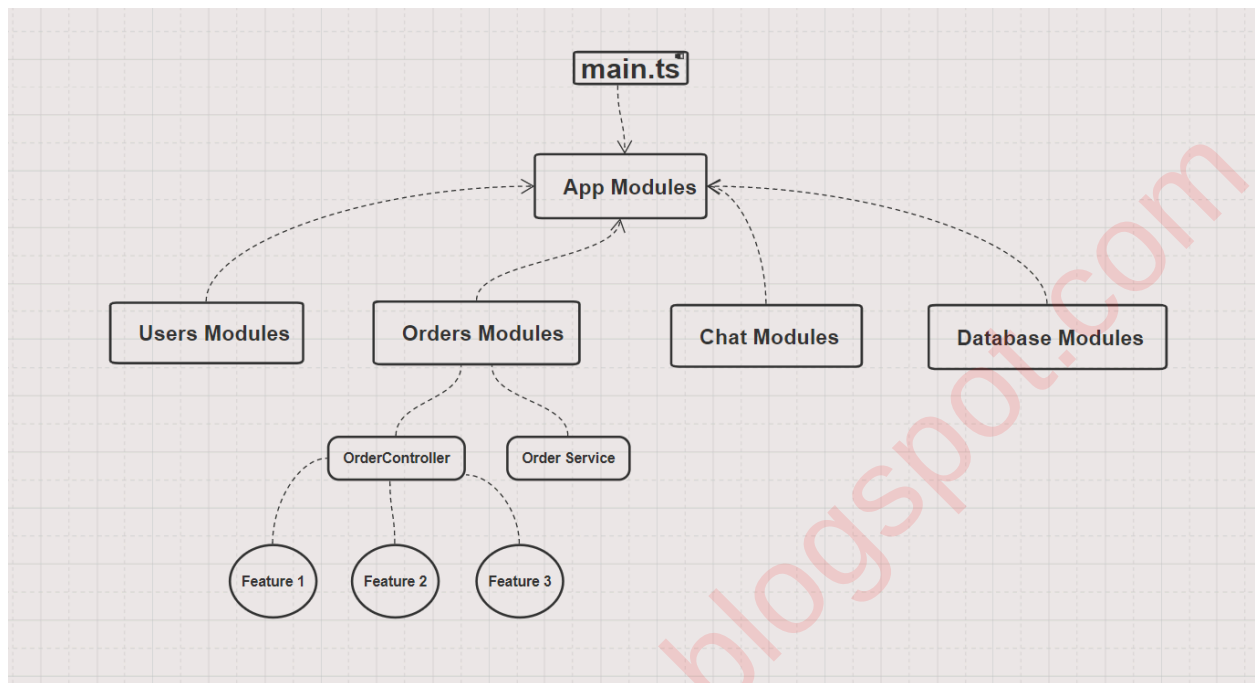
Về source code video này:

Các file thay đổi tại video, xem [tại đây](#)

Source code ứng với video này, xem [tại đây](#)

Video hướng dẫn cách download source sau khi kết thúc video này, xem [tại đây](#)

#4. Kiến trúc của dự án Nest.JS



Dự án Nest.JS được chia thành các module, phân theo chức năng.

- Cần chia theo module, để có thể phát triển độc lập, đồng thời, tính năng này sẽ ít ảnh hưởng tới tính năng kia (on/off theo ý muốn)
- File main.ts sẽ chạy app modules, và app modules sẽ bao gồm tất cả các modules con.
- Modules con có thể bao gồm (hoặc không) các modules khác

Ví dụ: để kết nối tới database, Users Modules có thể bao gồm Database Modules

Chapter 2: NestJS và Typescript

Tìm hiểu và học cách sử dụng Decorator của Typescript vào Nest.JS, từ đây nắm vững cách hoạt động Router và Controller khi sử dụng các Decorator có sẵn do Nest.JS cung cấp.

#5. Typescript Decorators

Tài liệu:

<https://www.typescriptlang.org/docs/handbook/decorators.html>

<https://www.digitalocean.com/community/tutorials/how-to-use-decorators-in-typescript>

Lưu ý: video này là cách chúng ta **"hiểu và sử dụng" decorator, không phải 'học cách viết decorator'**.

Tương tự như 'generic', bạn chỉ học, khi và chỉ khi bạn muốn reuse - tái sử dụng code, viết code ở 1 level cao hơn

1. Ví dụ về Decorators khi sử dụng với Nest.JS

@Controller()

@Injectable()

- Nếu bạn đã học java spring boot, Decorators của Nest.js, tương tự như "annotation", sẽ có dấu @ đặc trưng

Ví dụ: khi định nghĩa 1 class với @Controller => Nest sẽ "sử dụng" class này như là controller

2. Decorator là gì (what)

- Decorator là những ký hiệu (chú thích) giúp cho class/property/functions/params có thêm "sức mạnh" mới.

- **Decorator bắt đầu bằng dấu @, sau đấy là "expression".**

(express là function sẽ được thực thi khi được sử dụng với "class sử dụng nó")

bao gồm 5 level:

- + Class declaration itself
- + Properties
- + Accessors (getter/setter)
- + Methods
- + Parameters

ví dụ:

@classDecorator

```
class Person {
```

@propertyDecorator

```
public name: string;
```

@accessorDecorator

```
get fullName() {
```

```
  // ...
```

```
}
```

@methodDecorator

```
printName(@parameterDecorator prefix: string) {
```

```
  // ...
```

```
}
```

```
}
```

3. Tại sao cần Decorator ?

- Decorator giúp bạn viết code ngắn hơn, có khả năng tái sử dụng ở nhiều nơi, đồng thời bổ sung 'sức mạnh' cho các thành phần được 'trang trí' (decorated).

Ví dụ:

```
class User {
```

```
  @Min(0)
```

```
  @Max(10)
```

```
  @IsEmail
```

```
  email: string;
```

```
  @Max(20)
```

```
  password: string;
```

```
}
```

#6. Sử dụng Decorator với Nest.JS

Tài liệu:

<https://www.typescriptlang.org/docs/handbook/decorators.html#introduction>

1. Setup

- Mặc định, Nest đã cấu hình để sử dụng Decorator bằng setup trong file tsconfig.json

```
{
  "compilerOptions": {
    "target": "ES5", // tối thiểu là ES5 trở lên
    "experimentalDecorators": true
  }
}
```

Lưu ý: cho dù tính năng decorator của Typescript đang là “thử nghiệm”, có thể thay đổi trong tương lai, tuy nhiên chúng ta không cần lo lắng. Vì nếu có thay đổi, framework Nestjs sẽ lo chuyện này cho chúng ta :v

2. Phân loại Decorator

- Nest đã viết sẵn (amazing)
- Customize (tự định nghĩa) - sẽ học tại các chương sắp tới

3. Sử dụng Decorator

<https://stackoverflow.com/questions/70449766/typescript-decorators-why-do-some-decorators-require-brackets-and-others-dont>

```
class ExampleClass {  
  @decorate1( )  
  @decorate2  
  method( ) {  
    return "something";  
  }  
}
```

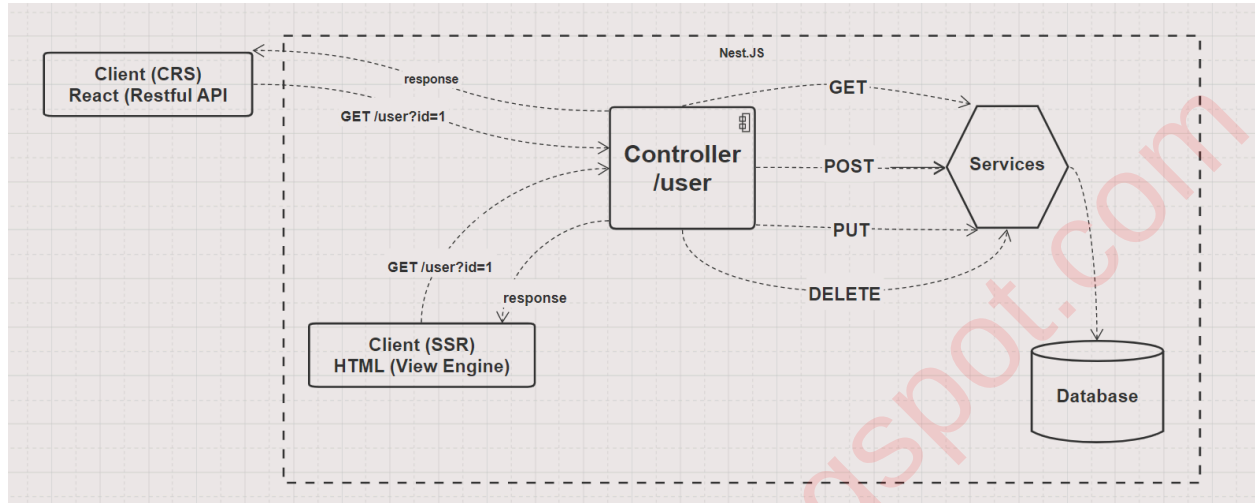
tại sao lại sử dụng: @decorate1() => cần dấu () và @decorate2 => không cần ()
=> phụ thuộc vào cách viết decorator

=> với Nestjs, sử dụng các Decorator viết sẵn, đa phần sẽ cần dấu ()
=> thực thi decorator

Ví dụ: @Controller(), @Injectable()...

#7. Mô hình Router với Nest.JS

Mô hình router của NestJS:



#8. Controllers

Tài liệu: <https://docs.nestjs.com/controllers>

- Remove file .controller.spect.ts (nếu có)

Một Class được gọi là controller, khi nó sử dụng @Controller()

1. Ví dụ về controller

- Tạo thêm userController : lưu ý, không cần hiểu cách khai báo controller vào app modules
- update hello world của appController, không dùng service (service sẽ học ở chương sau)

Ví dụ 1:

```
@Controller('user')
export class UserController {
  @Get()
  findAll(): string {
    return 'This action returns all users';
  }

  @Delete("/by-id")
  findById(): string {
    return 'This action will delete a user by id';
  }
}
```

Ví dụ 2:

```
@Controller()
export class AppController {

  @Get()
  getHello(): string {
    return 'This action hello world';
  }
}
```

2. Cách xử lý của Nest.JS

Bản chất của website, là xoay quanh URL (đường link user truy cập).
ở phía backend, URL chính là routes.

với ví dụ 1:

```
@Controller('user')
export class UserController {
  @Get()
  findAll(): string {
    return 'This action returns all users';
  }

  @Delete("/by-id")
  findById(): string {
    return 'This action will delete a user by id';
  }
}
```

khi khai báo @Controller('user') => Nest sẽ hiểu là "/user"
@Get() => không có tham số đính kèm => ứng với route "/"
=> cộng gộp sẽ ra GET "/user"

@Delete("/by-id") => có tham số đính kèm => ứng với route "/by-id"
=> cộng gộp sẽ ra DELETE "/user/by-id"

=> việc truyền tham số cho @controller("tham-số") sẽ giúp ko phải lặp lại code không cần thiết.

ví dụ, thay vì khai báo

```
@Controller() //không có tham số
@Get('/order') // GET /order
@Put('/order/by-name') // PUT /order/by-name
@Delete('/order/by-id') // DELETE /order/by-id
```

=> khai báo theo nhóm:

```
@Controller('order')
@Get() // GET /order
```

@Put('/by-name') // PUT /order/by-name
@Delete('/by-id') // DELETE /order/by-id

3. Tổng kết

@Controller() : khi sử dụng decorator này, nếu bạn không truyền tham số => sinh ra route "/"

@Controller("/prefix"): truyền tham số => sinh ra route "/prefix"

với method GET, POST, PUT, DELETE...

@Get() nếu bạn không truyền tham số => sinh ra GET "/"

@Get("/path") nếu bạn truyền tham số => sinh ra GET "/path"

=> NestJS sẽ tự động "cộng gộp" Controller và method. có nghĩa là

@Controller()

@Get('/user') => sinh ra route "/user"

@Controller("user1")

@Get('/filter-by-name') => sinh ra route "/user1/filter-by-name"

Về source code video này:

Các file thay đổi tại video, xem [tại đây](#)

Source code ứng với video này, xem [tại đây](#)

Video hướng dẫn cách download source sau khi kết thúc video này, xem [tại đây](#)

Chapter 3: Inversion Of Control

Khi Học về OOP (lập trình hướng đối tượng), một trong các cách dùng để tối ưu hóa ứng dụng là áp dụng mô hình **IoC(Inversion of control)** và **DI (Dependency injection)**. Chương này sẽ giúp chúng ta học cách sử dụng các "design pattern" trên vào framework Nest.JS

Lưu ý:

1. nội dung mang tính chất khái quát (giới thiệu), các bạn có thể tìm hiểu thêm từ tài liệu tại các video
2. Khái niệm Inversion of Control, hay Dependencies Injection được sử dụng trong lập trình hướng đối tượng, không phân biệt ngôn ngữ lập trình

#9. Inversion Of Control (IoC)

Tài liệu: <https://www.tutorialsteacher.com/ioc/introduction>

Where ? OOP (object oriented programing)

What ?

- Inversion of Control (IoC), là cách "đảo ngược" sự kiểm soát (tương tự tủ lạnh/điều hòa inverter, có nóng và lạnh :v)
- IoC là khái niệm (principle), ko phải là "cách làm" (làm như thế nào)

Why/When ?

Vấn đề gặp phải:

```
public class A
{
    B b;

    public A()
    {
        b = new B();
    }

    public void Task1() {
        // do something here..
        b.SomeMethod();
        // do something here..
    }
}
```



```
public class B {
```

```
    public void SomeMethod() {
```

```
        //doing something..
```

```
    }
```

```
}
```

=====

class A gọi b.SomeMethod() để hoàn tất task1 (của nó)

class A không thể hoàn thành "task1" nếu nó không có class B => Class A "phụ thuộc" vào class B

class A "kiểm soát" (control) vòng đời của class B (tạo và sử dụng class B)

=> Nhờ có IoC, chúng ta OOP, chúng ta có thể "giảm thiểu" sự phụ thuộc giữa các class với nhau.

Mục đích: có thể tách rời và "test" độc lập, không phụ thuộc vào nhau

How ?

- Sử dụng các "pattern", như Factory, Dependencies Injections...

#10.1 Khái Niệm Modules

Nhắc lại:

IoC (inversion of control) - đảo ngược sự kiểm soát, là nguyên tắc "giảm thiểu" sự phụ thuộc giữa các class với nhau.

thay vì class A control class B, chúng ta sẽ đảo ngược, cho phép A và B có thể hoạt động độc lập

IoC là nguyên tắc (cách thiết kế/bản vẽ), không phải cách làm như thế nào :v

Tài liệu:

<https://angular.io/guide/dependency-injection#understanding-dependency-injection>

1. Modules (Why)

- Khi bạn phát triển ứng dụng nhỏ, chúng ta có thể code "all-in-one" (tất cả code trong 1 file/1 thư mục/1 project).

Với dự án lớn, chúng ta không thể làm vậy, vì

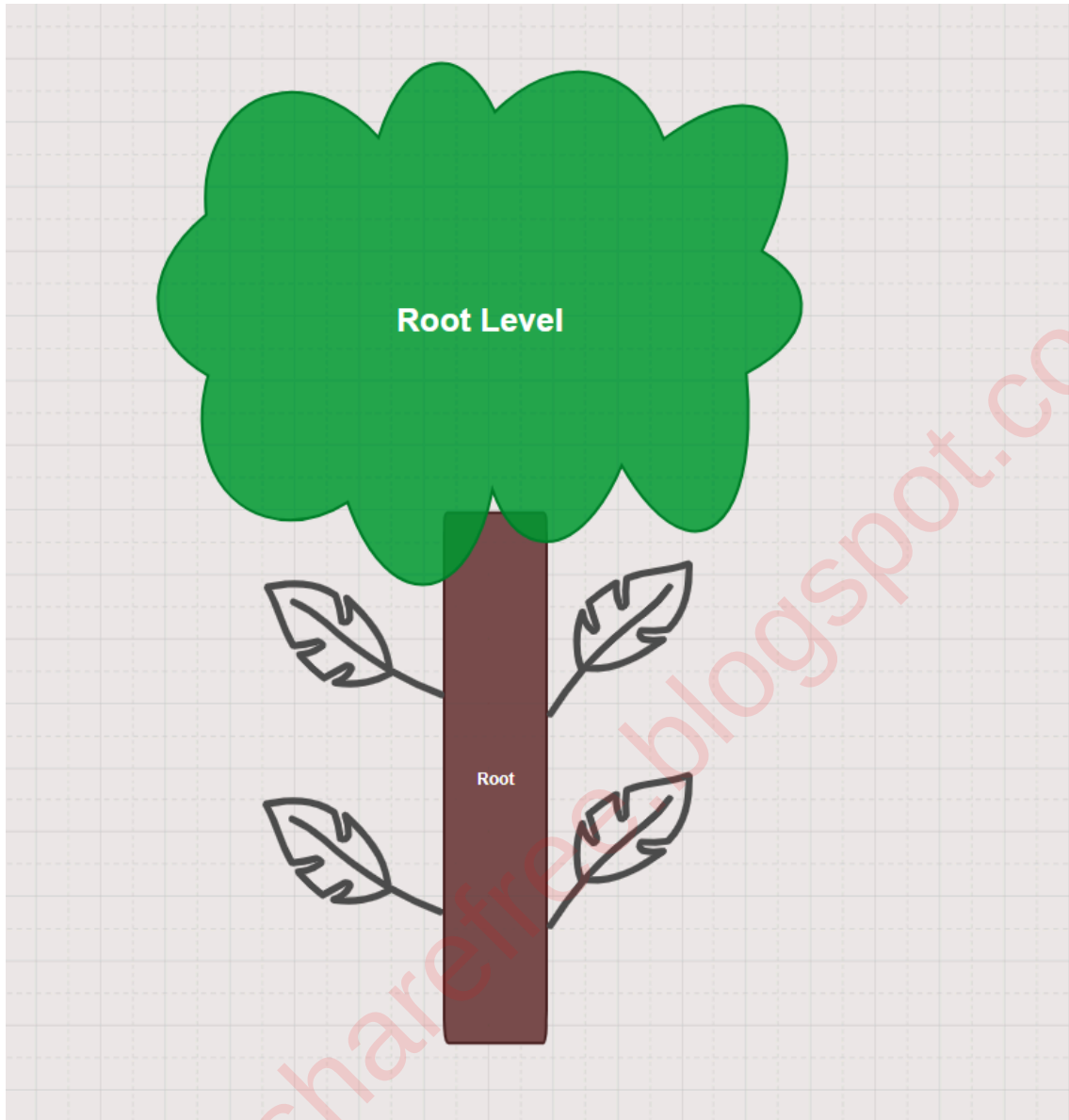
- + không thể đọc code (code quá nhiều)
- + không thể test/maintain

- Giải pháp đặt ra là chia dự án thành các modules. 1 module có thể là 1 tính năng hoặc 1 nhóm tính năng có liên quan tới nhau.

Ưu điểm lớn nhất của cách làm này:

- + sự phát triển các modules có thể làm độc lập
- + on/off modules này không làm ảnh hưởng tới modules kia (nếu các modules không phụ thuộc vào nhau)

Mình họa = hình cái cây (tree). Các modules chính là các lá của 1 cái cây. Lá này rụng sẽ không làm ảnh hưởng tới cây, hay các lá còn lại.



Ứng dụng trong thực tế:

Ví dụ Facebook có lỗi không gửi được tin nhắn (modules gửi tin nhắn), nhưng website/app facebook vẫn dùng các tính năng khác bình thường.

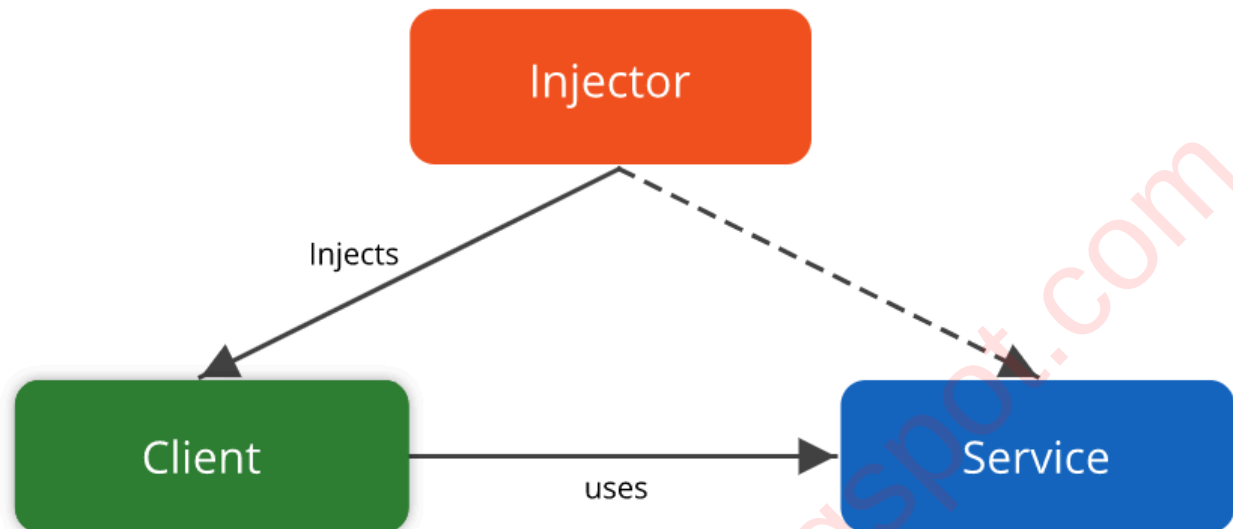
Nếu code all-in-one => lỗi 1 tính năng sẽ gây ra lỗi cả hệ thống không hoạt động

=> DI giúp chúng ta có thể chia code thành các modules độc lập :v

#10.2 Dependencies Injection (DI)

1. Các thành phần của DI (What)

gồm 3 thành phần chính: consumer, provider và injector



ví dụ: class A sử dụng service B

```
class A {
  serviceB.doSomething()
}
```

=> class A gọi là **consumer** (người tiêu thụ/sử dụng)

service B gọi là provider (nhà cung cấp)

còn 1 thành phần, gọi là injector (giúp tạo ra DI, và làm cho class A và B không phụ thuộc vào nhau :v)

2. Ví dụ về phạm vi (scope) và cách sử dụng DI

Ví dụ của angular: <https://angular.io/guide/dependency-injection#providing-dependency>

Sử dụng decorator: @Injectable()

Về scope:

- Nếu inject vào 1 module cụ thể => chỉ có module đó sử dụng.
- Nếu inject vào root module => tất cả có thể dùng :v

#11. Injecting a dependency

Tài liệu: <https://www.tutorialsteacher.com/ioc/dependency-injection>

Có 3 level của DI:

- Constructor Injection
- Property Injection
- Method Injection

Với NestJS (sử dụng javascript), tập trung sử dụng Constructor Injection :

Tài liệu:

<https://stackoverflow.com/questions/72446772/how-does-the-private-keyword-let-us-b-oth-declare-and-initialize-a-class-instance/72447725>

<https://www.typescriptlang.org/docs/handbook/2/classes.html#parameter-properties>

1. Constructor Injection

Viết đầy đủ theo OOP:

```
@Controller('cats')
export class CatsController {
  private catsService: CatsService;

  constructor(service: CatsService) {
    this.catsService = service;
  }
  /* TODO */
}
```

Viết ngắn gọn, sử dụng keyword "private":

```
@Controller('cats')
export class CatsController {
  constructor(private catsService: CatsService) {}
  /* TODO */
}
```

Chapter 4: NestJS và MVC

NestJS cũng có đủ khả năng và sức mạnh để làm Server theo mô hình MVC truyền thống. Tại đây, chúng ta sẽ tìm hiểu về View Engine, cách viết code theo mô hình Model-View-Controller và các nhược điểm đang còn tồn đọng của mô hình này.

#12. Ví dụ về controller/service

Tài liệu: <https://docs.nestjs.com/recipes/crud-generator>

Mục đích: hiểu các decorator và code Nest.JS "đã làm sẵn".

- Tạo controller
- Tạo Service
- Ôn tập lại DI (dependency injection)

#13. Template View Engine

Tài liệu: <https://docs.nestjs.com/techniques/mvc>
<https://expressjs.com/en/guide/using-template-engines.html>

Cài đặt thư viện:

npm i --save-exact ejs@3.1.9

yarn add ejs@3.1.9

Về source code video này:

Các file thay đổi tại video, xem [tại đây](#)

Source code ứng với video này, xem [tại đây](#)

Video hướng dẫn cách download source sau khi kết thúc video này, xem [tại đây](#)

#14. Mô hình MVC

- Không có Model ?

Về source code video này:

Các file thay đổi tại video, xem [tại đây](#)

Source code ứng với video này, xem [tại đây](#)

Video hướng dẫn cách download source sau khi kết thúc video này, xem [tại đây](#)

#15. Vấn Đề tồn đọng với SSR ?

1. Server Side Rendering ?

Thực tế, chúng ta có 2 cách để render ra dữ liệu: CSR và SSR.

CSR: Client Side Rendering, có nghĩa client sẽ render ra dữ liệu HTML (ví dụ sử dụng React truyền thống)

SSR: Server Side Rendering, server sẽ render dữ liệu HTML

2. Khi nào sử dụng SSR

Khi bạn muốn client nhận được phản hồi nhanh nhất (vì client có phải làm gì đâu, ngoài việc gửi request. Server sẽ render dữ liệu và gửi response về cho client)

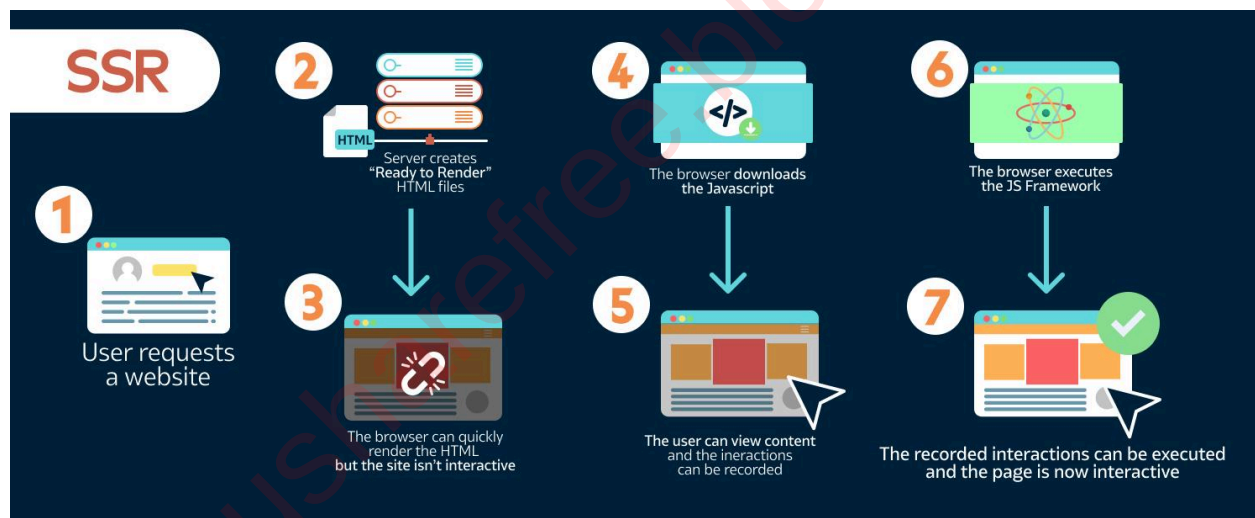
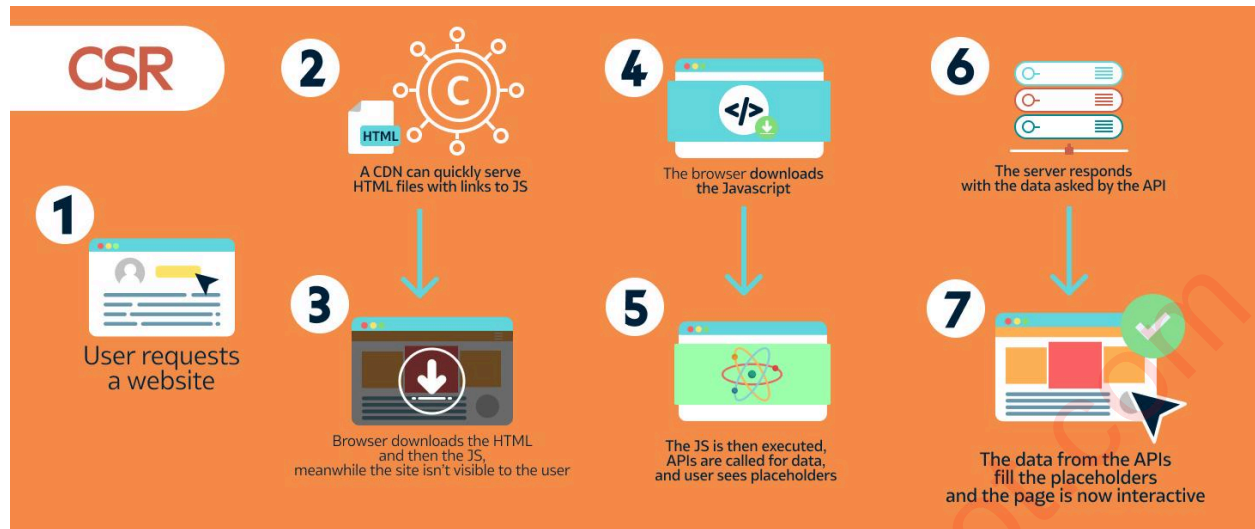
Ở đây, server phải mạnh (nhiều RAM, CPU...) thì mới nhanh. Server yếu thì cũng hơi lâu đấy :v

Ví dụ:

Client truy cập vào hoidanit.com.vn

Server của hoidanit.com.vn sẽ cần render ra trang chủ, ứng với route '/', sau đấy gửi kết quả html (chính là giao diện mà người dùng nhìn thấy)

Nhược điểm: chi phí thông thường sẽ cao :v



Chapter 5: Connect Database

Trong chương này, chúng ta sẽ cùng nhau kết nối backend tới Database bằng cách sử dụng MongoDB. Ngoài ra, sẽ tối ưu hóa cách sử dụng "hàng số", dựa vào cách sử dụng tham số môi trường (.env) với Nest.JS Config.

#16. Lựa chọn Database

Tài liệu: <https://docs.nestjs.com/techniques/database>

Chúng ta code backend thông qua ORM/ODM để chọn nhanh, không tính việc viết 'raw query' truyền thống

1. Các loại database hỗ trợ

- Nest hỗ trợ với bất kỳ loại SQL và NoSQL database nào, giống hệt như việc làm với Express (or Fastify)

Với SQL, hỗ trợ các thư viện nổi tiếng sau: (mysql, sql server, postgres, oracle...)

+ MikroORM: <https://mikro-orm.io/>

+ Sequelize: <https://sequelize.org/>

+ Knex.js: <https://knexjs.org/>

+ TypeORM: <https://typeorm.io/>

+ Prisma: <https://www.prisma.io/>

Với NoSQL, hỗ trợ MongoDB: <https://www.mongodb.com/>

Đặc biệt, NestJS hỗ trợ TypeORM, Sequelize và Mongoose (mongodb) với các package @nestjs/typeorm , @nestjs/sequelize, @nestjs/mongoose

=> giúp việc coding càng trở nên dễ dàng hơn

2. Lựa chọn loại database cho dự án

Với SQL, NestJS "gợi ý" sử dụng TypeORM. Đi làm, việc dùng công nghệ gì/thư viện gì là do cty + người leader quyết định.

Nếu bạn "đang tự học" NestJS, mình cũng gợi ý dùng TypeORM nhé

Yên tâm 1 điều, là khi bạn học được 1 thư viện, các thư viện khác cú pháp nó khác, nhưng mà cách tư duy nó "sam same" (tương tự nhau)

với TypeORM, hỗ trợ: Postgres, Oracle, Microsoft SQL Server, SQLite, MySQL (ko nên dùng MongoDB với typeORM :v)

Với Nest.js và khóa học này, mình sử dụng MongoDB (Mongoose) vì:

- Nodejs (javascript) vốn dĩ mềm dẻo, rất phù hợp với MongoDB
- SQL sẽ được dùng trong khóa học với Java, tham khảo [tại đây](#)

#17. Cài đặt MongoDB

Lưu ý: xem hết video này để hình dung cách làm. Cách dễ nhất để có được database MongoDB là sử dụng online với MongoDB Atlas

Phần mềm cần có : MongoDB Compass

Link download: <https://www.mongodb.com/try/download/compass>

Tạo tài khoản MongoDB Atlas : <https://www.mongodb.com/cloud/atlas/register>

Yêu cầu cần đạt được khi kết thúc video này, là có 1 đường link url kết nối vào MongoDB.

Setup account Mongo Atlas => get URL => check connection với Mongo Compass

Có 2 cách nữa để cài đặt và sử dụng mongodb (mục đích học tập)

1. Thông qua docker

Bạn nào đã học [khóa restful API của mình với Express](#), thì đã không lạ gì với MongoDB và docker nhỉ :v

Sau khi dựng database MongoDB bằng docker image, chúng ta dùng phần mềm MongoDB Compass để có view dữ liệu.

2. Thông qua MongoDB Server

Cài đặt MongoDB Server hoặc tạo tài khoản và sử dụng database Atlas online

#18. Sử dụng MongoDB với NestJS

Lưu ý: bạn cần xem video [#17](#) để tự tạo URL database MongoDB của bạn, bạn không thể sử dụng URL giống hệt video này (vì đây là database của hoidanit)

Để sử dụng ORM, chúng ta sử dụng **mongoose**

Tài liệu: <https://docs.nestjs.com/techniques/mongodb>

Cài đặt thư viện:

npm install --save-exact @nestjs/mongoose@9.2.2 mongoose@7.1.0

Về source code video này:

Các file thay đổi tại video, xem [tại đây](#)

Source code ứng với video này, xem [tại đây](#)

Video hướng dẫn cách download source sau khi kết thúc video này, xem [tại đây](#)

#19. ENV Variables

1. Sử dụng .dotenv

Cài đặt thư viện:

```
npm i --save-exact dotenv@16.0.3
```

Tạo file .env ở level root, và để truy cập 1 biến, chúng ta sử dụng cú pháp:

```
process.env.VARIABLE_NAME
```

Với NestJS, chúng ta không sử dụng cách làm này, mặc dù nó vẫn “chạy code” bình thường.

Trong 1 dự án lớn, đôi khi chúng ta cần:

- Nhiều hơn 1 file .env, ví dụ mỗi môi trường sử dụng là 1 file .env
Môi trường test: .env.test
Môi trường development: .env.development
Môi trường uat: .env.uat
Môi trường production: .env.production
- Chúng ta muốn validate dữ liệu của biến trong .env . Cách làm truyền thống, sử dụng trực tiếp .env là không làm được

Rất may mắn, NestJS đã cung cấp giải pháp để giải quyết 2 khóa khăn trên.

2. Config Service

Tài liệu:

<https://docs.nestjs.com/techniques/configuration>

<https://github.com/nestjs/nestjs-config/issues/19>

Cài đặt thư viện:

```
npm i --save-exact @nestjs/config@2.3.1
```

```
yarn add @nestjs/config@2.3.1
```

Về bản chất, Config Service cũng sử dụng thư viện .env, tuy nhiên, nó đã “code thêm” để giải quyết 2 vấn đề chúng ta nêu ở trên (sử dụng nhiều file .env và validate file .env)

3. Sử dụng Config Service

Tài liệu: <https://docs.nestjs.com/techniques/configuration#using-the-configservice>

- Khai báo config service ở constructor (dùng dependency injection)
constructor(private configService: ConfigService) { }
- Lấy giá trị của .env, theo cú pháp:

```
// get an environment variable  
const myVar = this.configService.get<string>('VARIABLE_NAME');
```

4. Sử dụng với file main.ts

Tài liệu: <https://docs.nestjs.com/techniques/configuration#using-in-the-maints>

```
const configService = app.get(ConfigService);
```

```
const port = configService.get('PORT');
```

5. Sử dụng khi khai báo Module

Tài liệu với MongoDB:

<https://docs.nestjs.com/techniques/mongodb#async-configuration>

Về source code video này:

Các file thay đổi tại video, xem [tại đây](#)

Source code ứng với video này, xem [tại đây](#)

Video hướng dẫn cách download source sau khi kết thúc video này, xem [tại đây](#)

Chapter 6: Restful API

Ngoài khả năng có thể tạo server theo mô hình MVC, NestJS cũng có thể làm server Restful APIs. Chúng ta sẽ cùng nhau viết nhanh module Users theo đúng chuẩn Restful, sử dụng các công cụ có sẵn của NestJS, bao gồm generate resources, schema, controller và services.

#20. Generate resources

1. Câu lệnh generate resources

Tài liệu: <https://docs.nestjs.com/recipes/crud-generator>

To avoid generating test files, you can pass the --no-spec flag, as follows:

```
nest g resource users --no-spec
```

2. Cài đặt và sử dụng PostMan

<https://www.postman.com/downloads/>

File package.json:

```
"@nestjs/mapped-types": "1.2.2"
```

Về source code video này:

Các file thay đổi tại video, xem [tại đây](#)

Source code ứng với video này, xem [tại đây](#)

Video hướng dẫn cách download source sau khi kết thúc video này, xem [tại đây](#)

#21. Create Schema (Model)

Tài liệu: <https://docs.nestjs.com/techniques/mongodb>

Lưu ý: update tên của database

Tạo User Model với các thuộc tính:

- Email
- Password
- Name
- Address
- Phone
- Age

Về source code video này:

Các file thay đổi tại video, xem [tại đây](#)

Source code ứng với video này, xem [tại đây](#)

Video hướng dẫn cách download source sau khi kết thúc video này, xem [tại đây](#)

#22.1 Create A User

Tài liệu: <https://docs.nestjs.com/controllers#request-payloads>
<https://docs.nestjs.com/techniques/mongodb#model-injection>

Full example: <https://github.com/nestjs/nest/tree/master/sample/06-mongoose>

Mục tiêu:

Tạo API với endpoint:

POST <http://localhost:8000/users>

Body : {
 Email
 Password
 Name
 Address
 Phone
 Age
}

Về source code video này:

Các file thay đổi tại video, xem [tại đây](#)

Source code ứng với video này, xem [tại đây](#)

Video hướng dẫn cách download source sau khi kết thúc video này, xem [tại đây](#)

#22.2 Hash User's Password

Tài liệu:

<https://www.npmjs.com/package/bcryptjs>

Cài đặt thư viện:

npm i --save-exact bcryptjs@2.4.3

npm i --save-dev @types/bcryptjs@2.4.2

yarn add bcryptjs@2.4.3 @types/bcryptjs@2.4.2

Về source code video này:

Các file thay đổi tại video, xem [tại đây](#)

Source code ứng với video này, xem [tại đây](#)

Video hướng dẫn cách download source sau khi kết thúc video này, xem [tại đây](#)

#22.3 DTO - Data Transfer Object

<https://docs.nestjs.com/controllers#request-payloads>

Về request object: <https://docs.nestjs.com/controllers#request-object>

DTO là 1 object định nghĩa hình dạng dữ liệu được “transfer” (frontend và backend)

Lưu ý sử dụng class, thay vì type hay interface

Về source code video này:

Các file thay đổi tại video, xem [tại đây](#)

Source code ứng với video này, xem [tại đây](#)

Video hướng dẫn cách download source sau khi kết thúc video này, xem [tại đây](#)

#22.4 Pipe

<https://docs.nestjs.com/pipes>

Pipe có 2 tác dụng:

- Transform data : convert string => number ,array ...
- Validate data

#22.5 Validation

<https://docs.nestjs.com/techniques/validation>

<https://docs.nestjs.com/pipes#class-validator>

Cài đặt thư viện:

npm i --save-exact class-validator@0.14.0 class-transformer@0.5.1

yarn add class-validator@0.14.0 class-transformer@0.5.1

Về class-validator: <https://github.com/typestack/class-validator>

Về customize message ?

Về source code video này:

Các file thay đổi tại video, xem [tại đây](#)

Source code ứng với video này, xem [tại đây](#)

Video hướng dẫn cách download source sau khi kết thúc video này, xem [tại đây](#)

#23. Get User by Id

Mục tiêu:

Tạo API với endpoint:

GET <http://localhost:8000/users/:id>

Body : Không cần truyền, mà truyền id ở url

Lưu ý về method get => dễ bị trùng url

Về source code video này:

Các file thay đổi tại video, xem [tại đây](#)

Source code ứng với video này, xem [tại đây](#)

Video hướng dẫn cách download source sau khi kết thúc video này, xem [tại đây](#)

#24. Update a User

Tài liệu: <https://docs.nestjs.com/techniques/validation#mapped-types>

Lưu ý: omit password

Mục tiêu:

Tạo API với endpoint:

PUT <http://localhost:8000/users/:id>

Body : Không cần truyền, mà truyền id ở url

Về source code video này:

Các file thay đổi tại video, xem [tại đây](#)

Source code ứng với video này, xem [tại đây](#)

Video hướng dẫn cách download source sau khi kết thúc video này, xem [tại đây](#)

#25. Delete a User

Mục tiêu:

Tạo API với endpoint:

DELETE <http://localhost:8000/users/:id>

Body : Không cần truyền, mà truyền id ở url

Về source code video này:

Các file thay đổi tại video, xem [tại đây](#)

Source code ứng với video này, xem [tại đây](#)

Video hướng dẫn cách download source sau khi kết thúc video này, xem [tại đây](#)

Chapter 7: Stateful và Stateless

Mọi website đều hoạt động dựa trên mô hình Stateful và Stateless. Ở đây, chúng ta cần nắm vững, hiểu rõ và có khả năng phân biệt điểm khác nhau của 2 mô hình này, bởi vì, việc chúng ta lựa chọn mô hình nào, nó sẽ quyết định cách thức tổ chức code cho dự án của chúng ta.

#26. Setup Backend Test

Clone source code dự án [tại đây](#)

1. Các việc cần làm

Sau khi kéo code về, chạy các câu lệnh sau: (version mình sử dụng là node 16.x)

`npm i` => cài đặt thư viện

`setup url MongoDB, file .env`

`npm run dev` => chạy ở chế độ development

2. Test ứng dụng

Lưu ý:

- **với stateful:** sử dụng route `/login` `/user` và route `'/'`

- **với stateless:** sử dụng endpoint

+ login: `/stateless/login` => truyền body gồm username và password

+ get users `/stateless/user` => cần truyền bearer token ở header request

#27. Debug NestJS Applications

Tài liệu: https://code.visualstudio.com/docs/editor/debugging#_launch-configurations

Mục tiêu: biết cách đặt breakpoint xem code chạy

=> check giá trị của biến tại các thời điểm khác nhau

Lưu ý cực quan trọng: bạn vui lòng sử dụng version nodejs 16.20.0 để tính năng debug chạy ổn định

1. Chạy nestjs với chế độ debug

Mặc định, nestjs hỗ trợ câu lệnh debugs để đặt break point: `start:debug: "nest start --debug --watch",`

`npm run start:debug`

2. Cấu hình vscode

Tài liệu: <https://www.youtube.com/watch?v=QL3KXE1hOgA>

<https://stackoverflow.com/a/63325135>

- **Cần tạo file launch.json** : lưu ý chọn môi trường NodeJS

=> ngay lập tức sẽ có thư mục .vscode (chứa file launch.json - đây là file cấu hình debug)

Nội dung file debug (launch.json), copy/paste [tại đây](#) cho nhanh các bạn nhé.

```
"configurations": [  
  {  
    "type": "node",  
    "request": "launch",  
    "name": "Nest Debug",  
    "runtimeExecutable": "npm",  
    "runtimeArgs": [  
      "run",  
      "start:debug",  
      "--",  
      "--inspect-brk"  
    ],  
    "console": "integratedTerminal",  
    "restart": true,  
    "protocol": "auto",  
    "port": 9229,  
    "autoAttachChildProcesses": true  
  }  
]
```

3. Sử dụng debugs

Run app với debugs mode => node need npm run start:debug

Về các actions: https://code.visualstudio.com/docs/editor/debugging#_debug-actions

Về khóa học debug website (cả frontend/backend) với React/Node.js, các bạn tham khảo [tại đây](#)

Về source code video này:

Các file thay đổi tại video, xem [tại đây](#)

Source code ứng với video này, xem [tại đây](#)

Video hướng dẫn cách download source sau khi kết thúc video này, xem [tại đây](#)

#28. Stateful Application

Who you are ? user/guest

What can you do ? (view/create/update/delete)

Mỗi lời gọi request từ client => gửi lên server, cần biết, ai là người thực hiện hành động ấy, và người đấy được phép làm những gì ?

Giữa các lời gọi khác nhau, thông tin user không được lưu lại (**http request**). tức là:

GET /user

GET /profile

với 2 lời gọi này, client không truyền lên dữ liệu user là ai ?

Vậy làm sao để server biết ai là người đang đăng nhập sử dụng hệ thống ?

1. Session

Session là "bộ nhớ" của server, dùng để lưu trữ thông tin của người dùng (phiên đăng nhập)

Do client không truyền lên dữ liệu user => client cần lưu giữ "id" của user, và gửi lên giữa các lời gọi request
(để bảo mật, id thường lưu ở cookies)

Server dựa vào id này, truy vấn vào "session" để biết được người dùng đang đăng nhập là ai => quyết định xử lý request với thông tin đã có

2. Các phương pháp lưu trữ thông tin của client và server

Client (browser): local storage, session, cookies

Server: session (memory/RAM, disk storage/file, database)

#29. Cơ chế hoạt động của stateful app

Giải thích cơ chế hoạt động của stateful với session:

1. cần config session/cookies/passport cho express (file main.ts). như vậy khi app khởi động lên, nó đã biết được sự tồn tại của session.

ở đây, làm product ko lưu session vào Memory.
do dùng với mongodb => lưu session vào mongodb

2. Tạo route login POST /login
với route này, sử dụng 'guard' của nestJS (truyền vào local strategy của passport) => passport xử lý phần còn lại

3. Local Strategy được 'nhúng' khi module khởi động lên, input từ HTML (bao gồm username/password) sẽ tự động chạy vào hàm "validate"

- Nếu username/password không hợp lệ => hiển thị thông báo lỗi

- Nếu username/password hợp lệ => lưu thông tin vào session (request).
session này cũng được lưu "1 bản sao" vào database.
đồng thời, session cookies được lưu tại client (html)

4. Mỗi lần f5, cookies sẽ được gửi lên server.

server dựa vào cookies (lấy ra sessionId) => kết hợp với passport (session serializer),
query xuống database bảng session (setup tại bước 10). lấy ra session tương ứng

=> như vậy sẽ maintain được "session của user" mỗi lần refresh page

#30. Ưu, nhược điểm của Stateful ?

Ưu điểm:

1. client không lưu giữ thông tin, ngoại trừ session_id

=> tính bảo mật cao, ít bị lộ thông tin người dùng

2. Server có thể "terminated"/chấm dứt/destroy/delete session của user bất cứ khi nào khi cần thiết.

Khi session bị deleted, **user === logout**

Nhược điểm:

- Cần phải có cơ chế save/query session đủ nhanh (save Redis)
(nếu số lượng người dùng truy cập lớn).

- Không thể share sessions giữa các hệ thống khác nhau.

Ví dụ: khi bạn thực hiện chuyển khoản liên ngân hàng. từ vcb => mb , sử dụng ứng dụng của vcb. như vậy, khi bạn chuyển khoản tới mb, mb ko biết bạn là ai, có hợp lệ hay không

...

#31. Mô hình stateless

- Không dùng session

- Client sử dụng: access token/refresh token để định danh thay cho session

- Với mỗi lời gọi request, client gửi kèm token ở header: access token (đã mã hóa/encoded).

Token này chứa thông tin giúp định danh user là ai, và chỉ server mới có thể giải mã (decoded)

- Server sẽ decoded token gửi lên để biết ai là người thực hiện request
=> xử lý request như bình thường

- Để cho an toàn (trường hợp lộ access token, người khác có thể mạo danh bạn), access token thường có thời gian sử dụng ngắn (3 phút, 5 phút, 30 phút...)

Khi request gửi lên server với access_token đã hết hạn
=> thông báo lỗi, và cần sử dụng refresh_token

Sử dụng refresh token để đổi lấy access_token/refresh_token với thời hạn sử dụng mới.

#32. Ưu, nhược điểm của Stateless

Ưu điểm:

- Không dùng session
- Dựa hoàn toàn vào cơ chế tạo token (access_token, refresh_token)
- Backend xử lý đơn giản hơn khi chỉ quan tâm encoded/decoded token
- Có thể “xác thực” giữa các hệ thống khác nhau

Nhược điểm:

- Tìm ẩn rủi ro nếu người dùng để lộ/bị hack token
 - 1 token khi đã issued (đã được cấp cho user), không có cách nào để có thể delete token đấy.
- Có nghĩa là, nếu token đang hợp lệ, bạn không thể delete token đấy

Chapter 8: JWT - Json Web Token

Sử dụng mô hình Stateless, server sẽ không có "session" để kiểm tra user có hợp lệ hay không, nên JWT (Json web token) là một giải pháp rất phổ biến được dùng cho mô hình này. Ở đây, chúng ta sẽ tìm hiểu về JWT, cũng như áp dụng nó để xác thực người dùng trong dự án NestJS

#33. JWT là gì ?

Tài liệu: <https://jwt.io/>

1. What ?

JWT - Json Web Token là 1 chuỗi ký tự đã được mã hóa (tương tự như việc hash password)

Mục đích mã hóa token là để trao đổi giữa các hệ thống với nhau và không làm lộ thông tin nhạy cảm (ví dụ frontend và backend)

Ví dụ về JWT:

eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9

.eyJzdWIiOiIxMjM0NTY3ODkwIiwibmFtZSI6IkpvaG4gRG9lIiwiaWF0IjoxNTE2MjM5MDIyfQ

.SflKxwRJSMeKKF2QT4fwpMeJf36POk6yJV_adQssw5c

JWT gồm 3 thành phần:

- + Header (algorithm & token type) : chứa thuật toán mã hóa, và loại token
- + payload (data): data được truyền đi giữa các bên sử dụng . được sử dụng dưới dạng json (object)
- + verify signature (chữ ký): client ký vào token (đánh dấu). chỉ có server (nắm giữ secret) là có thể giải mã token này.

2. Why ?

- Sử dụng JWT như là 1 cách an toàn để trao đổi thông tin giữa các bên liên quan. (client/server)

- Hỗ trợ thuật toán mạnh mẽ (encoded), và chỉ có secret của server mới có thể giải mã (decoded)

3. When ?

- Có các hệ thống khác nhau, cần có hình thức để xác thực user

#34. Phân loại token sử dụng

1. Access Token

- Được backend issued (backend là người tạo ra token này & lưu dữ secret để decoded token/giải mã token)

- Thông thường được mã hóa dưới dạng JWT.

- Trường hợp hay gặp nhất, là user login . Nếu login thành công => server sẽ trả về cho client bộ access_token/refresh_token

Ứng với mỗi lời gọi request (API). client sẽ cần truyền thêm vào header access_token

Backend sẽ lấy thông tin của user trong token (ví dụ như username, email, id : những thông tin để định danh user là ai)

Lưu ý: không truyền thông tin nhạy cảm trong token (ví dụ như password, otp...)

2. Refresh Token

Để đảm bảo an toàn, token thường có thời hạn sống (expired date)

Vì nếu bị lộ token, người khác có thể mạo danh bạn.

access_token (chứa thông tin user) => có thời hạn sống ngắn

refresh_token (chứa thông tin để tạo ra bộ access_token/refresh_token mới), có thời hạn sống dài: 1 tháng, 3 tháng, 6 tháng, 1 năm...

3. Hình thức lưu trữ token

Frontend:

- **localStorage: access_token** -> thuận tiện việc truy cập/lưu trữ token.

lấy data từ localStorage: localStorage.getItem('item').

Đồng thời, do **access_token có thời hạn sống ngắn**,

nên khi bị lộ (hack localStorage), xác suất có rủi ro cũng giảm thiểu rất nhiều

- sessionStorage: không dùng, vì khi close browser sẽ mất dữ liệu

- cookies: refresh_token.

Cookies có nhiều chế độ để đảm bảo độ an toàn khi truy cập. ví dụ, chỉ cho phép server sử dụng cookies, http= true

client (javascript): không thể lấy cookies = hàm **document.cookie**

Cookies sẽ tự hết hạn theo 1 thời gian nhất định (nếu set expired date) => ko dùng nó cũng tự mất :v

Backend:

Backend có thể lưu trữ token ở memory (RAM), disk (file), hoặc database (hay dùng nhất).

tailieusharefree.blogspot.com

#35. Giới thiệu về Passport.JS

Tài liệu: <https://www.passportjs.org/>

1. What ?

- Passport (hay passportjs) là thư viện giúp việc authentication (xác thực/login) với Node.js trở nên dễ dàng hơn bao giờ hết.

Ờ. dùng thư viện bao giờ không dễ. vậy tại sao cần passport, trong khi chúng ta có thể "tự code" được.

- Thực chất passport là 1 middleware, can thiệp vào req, và res. như vậy sẽ xác nhận được (authenticated) là user đã đăng nhập hay chưa.

- Lưu ý: **authentication vs authorization** (passport là xác thực người dùng đã đăng nhập chưa, ko liên quan gì tới phân quyền người dùng).

2. Why ?

Với việc login local (bạn tự quản lý database cùng với username/password), ok thì tự code được.

Vậy login với bên thứ 3 (third-party) thì sao, ví dụ như Facebook, Google, Apple, Amazon...

chẳng lẽ lại đọc tài liệu, rồi đi ghép api ?

Passport sinh ra để giải quyết vấn đề trên:

- đơn giản hóa việc xác thực người dùng (authentication)
- support hơn 500+ strategies (các loại login khác nhau)
- an toàn hơn so với việc bạn tự code, vì ít nhất thư viện đã được sử dụng rộng rãi + được testing về security

3. Strategies

- Do passport hỗ trợ rất nhiều "kiểu login", nên có rất nhiều strategies được làm ra (phục vụ các mục đích khác nhau)

Trong khóa học này, chúng ta sử dụng các strategies sau:

passport-local: xác thực người dùng thông qua username/password

passport-jwt: xác thực người dùng với jwt (json web token)

#36. Local Strategies với NestJS

Tài liệu: <https://docs.nestjs.com/recipes/passport>

Yêu cầu đặt ra: (mô hình stateless)

- Client login với username/password
- Nếu login thành công, server cần tạo ra access_token (dưới dạng jwt - json web token) gửi về cho client
- Kể từ lúc này, tất cả request gửi lên server, đều cần bearer token ở header (theo chuẩn oauth)
- Nếu request gửi lên ko có token => báo lỗi (protected routes)

1. Xác thực người dùng

Đây là quá trình client gửi lên server username/password. Nhiệm vụ của server là check xem thông tin đăng nhập có hợp lệ hay không ?

Cài đặt thư viện:

```
npm install --save-exact @nestjs/passport@9.0.3 passport@0.6.0  
passport-local@1.0.0
```

cài đặt package check type khi code typescript:

```
npm install --save-dev @types/passport-local@1.0.35
```

Cài 3 thư viện (không phải là cài 1 thư viện duy nhất vì):

- **passport** là thư viện gốc => giúp ra tạo ra middleware (can thiệp vào req và res), và **lưu trữ thông tin người dùng đăng nhập (req.user)**

- **@nestjs/passport** là thư viện viết theo phong cách của nestjs, giúp việc can thiệp vào passport dễ dàng hơn

- **passport-local**: đây là strategy hỗ trợ việc đăng nhập sử dụng username/password

Sau này, khi cần tạo ra jwt, chúng ta sẽ cài thêm strategy: passport-jwt

Về cú pháp Imports, exports sử dụng trong video, các bạn tham khảo ở đây:

<https://docs.nestjs.com/modules>

Với @Modules thì:

Providers: the providers that will be instantiated by the Nest injector and that may be shared at least across this module

Providers thông thường là các services, chúng ta dùng để injects vào constructor thông qua DI (dependency injection)

Imports: the list of imported modules that export the providers which are required in this module

Chúng ta sẽ cần import vào modules sử dụng

Exports: the subset of providers that are provided by this module and should be available in other modules which import this module. You can use either the provider itself or just its token (provide value)

Chúng ta cần xuất ra (xuất services/providers) thì cái import ở trên mới có tác dụng

Về source code video này:

Các file thay đổi tại video, xem [tại đây](#)

Source code ứng với video này, xem [tại đây](#)

Video hướng dẫn cách download source sau khi kết thúc video này, xem [tại đây](#)

#37. Nestjs Guard

1. Middleware

Tài liệu: <https://expressjs.com/en/guide/using-middleware.html>

Với Node.js, chúng ta có 1 khái niệm là middleware

myfunction (req, res, next){ ...}

request => middleware => response

req => routes => middleware => controller => service => res

Có 2 trường hợp xảy ra:

- req -> route -> middleware. dữ liệu ko hợp lệ, không thực thi tiếp -> res (thông báo lỗi)
- req -> route -> **middleware. dữ liệu hợp lệ => next()** -> controller -> service -> res

Hiểu đơn giản, middleware, như tên gọi của nó. giúp bạn can thiệp vào giữa req và res :
req -> middleware -> res

2. Guards

Tài liệu: <https://docs.nestjs.com/guards>

Guards (chú bảo vệ :v)

Nó làm nhiệm vụ giống middleware, can thiệp vào req và res . **req -> guards -> res**

But middleware, by its nature, is dumb.

Nếu bạn dùng middleware, bạn chỉ có thể can thiệp vào req và res (req, res, next).

Ví dụ: router('/test', myMiddleware, myController).

với middleware, bạn không thể biết "handler" phía sau là gì, vì lúc nào, bạn cũng làm việc với req và res. phần còn lại là hàm next () đã lo :v

đấy là lý do nó "dumb" => hơi ngầu ngầu ấy :v

Guards thì hoàn toàn ngược lại, nó mạnh mẽ hơn nhờ middleware.

Ngoài khả năng truy cập req, res, nó còn được sử dụng "**ExecutionContext**".

Hiểu 1 cách đơn giản, là không gian thực thi code.

p/s: google để hiểu tổng quan về execution context của javascript.

Guard có nhiệm vụ check true/false

- Nếu true: cho đi tiếp
- Nếu false: trả về phản hồi

3. Ví dụ về viết và sử dụng Guard đơn giản

#38. LocalGuard với Passport

Tài liệu: <https://docs.nestjs.com/recipes/passport#built-in-passport-guards>

Về source code video này:

Các file thay đổi tại video, xem [tại đây](#)

Source code ứng với video này, xem [tại đây](#)

Video hướng dẫn cách download source sau khi kết thúc video này, xem [tại đây](#)

#39. Sử dụng JWT

Tài liệu: <https://docs.nestjs.com/recipes/passport#jwt-functionality>

Cài đặt thư viện:

```
npm install --save-exact @nestjs/jwt@10.0.3 passport-jwt@4.0.1
```

```
npm install --save-dev @types/passport-jwt@3.0.8
```

Việc đã làm:

- Xác thực được user đã đăng nhập thành công hay chưa với **passport-local** strategy
- hàm validateUser được sử dụng cho việc trên
- khi login thành công, đã lưu được **thông tin user vào req.user**

Việc chưa làm:

- login thành công chưa trả về jwt (json web token) => sẽ làm trong video này
- chưa bảo vệ route với jwt (sẽ làm ở #40)

Về cú pháp Import/Exports... của modules, tham khảo tại:

<https://docs.nestjs.com/modules>

Về source code video này:

Các file thay đổi tại video, xem [tại đây](#)

Source code ứng với video này, xem [tại đây](#)

Video hướng dẫn cách download source sau khi kết thúc video này, xem [tại đây](#)

#40. Implementing Passport JWT

Tài liệu: <https://docs.nestjs.com/recipes/passport#implementing-passport-jwt>

todo: bảo vệ route với jwt

Về source code video này:

Các file thay đổi tại video, xem [tại đây](#)

Source code ứng với video này, xem [tại đây](#)

Video hướng dẫn cách download source sau khi kết thúc video này, xem [tại đây](#)

#41. Enable authentication globally

check jwt cho tất cả api

Tài liệu: <https://docs.nestjs.com/recipes/passport#enable-authentication-globally>

Về source code video này:

Các file thay đổi tại video, xem [tại đây](#)

Source code ứng với video này, xem [tại đây](#)

Video hướng dẫn cách download source sau khi kết thúc video này, xem [tại đây](#)

#42. Disable Guard

Sử dụng @Public()

Về metadata: <https://stackoverflow.com/a/73171823>

Tham khảo:

<https://stackoverflow.com/questions/49429241/nest-js-global-authguard-but-with-exceptions>

Về set global guard in main.ts:

<https://github.com/nestjs/nest/issues/964#issuecomment-480834786>

Về source code video này:

Các file thay đổi tại video, xem [tại đây](#)

Source code ứng với video này, xem [tại đây](#)

Video hướng dẫn cách download source sau khi kết thúc video này, xem [tại đây](#)

#43. Fix bugs và tổng kết

Tài liệu: <https://www.npmjs.com/package/ms>

1.Convert String to milliseconds

Cài đặt thư viện:

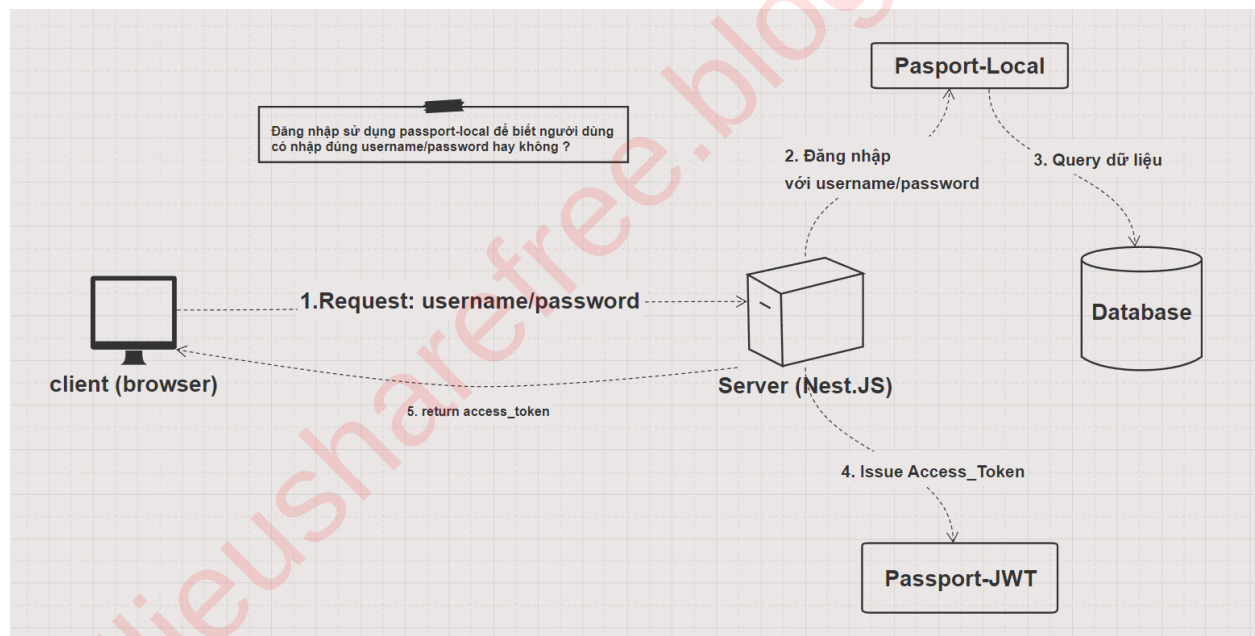
npm i --save-exact ms@2.1.3

npm i --save-dev @types/ms@0.7.31

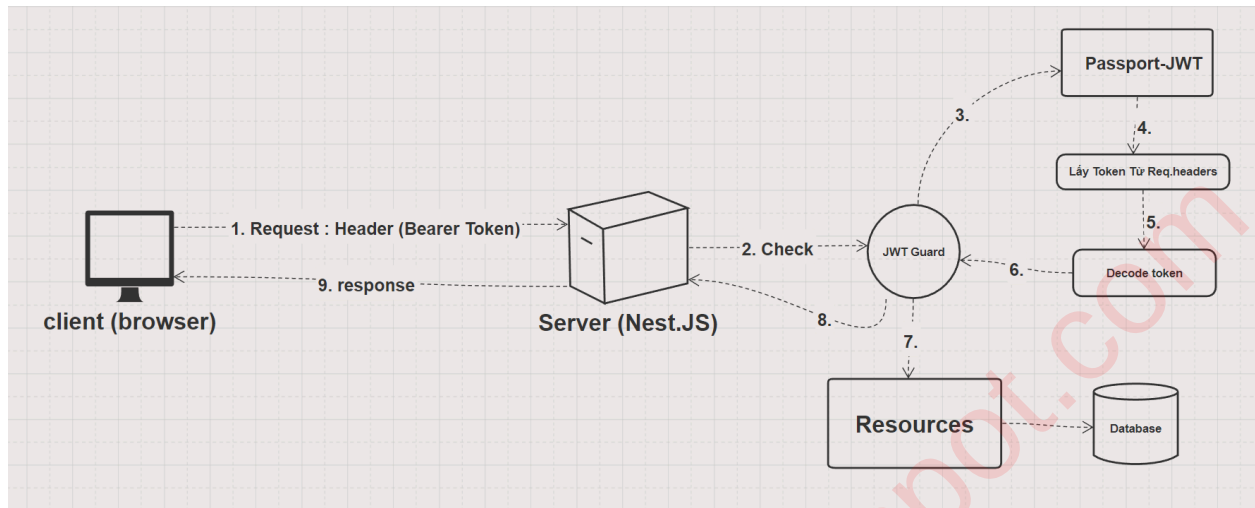
and adding "esModuleInterop": true to compilerOptions of tsconfig.json works for me.

2. Mô hình hoạt động của passport với Local/JWT strategy

Luồng login để lấy access_token:



Luồng protected routes với JWT (cần truyền lên JWT ở header request)



Về source code video này:

Các file thay đổi tại video, xem [tại đây](#)

Source code ứng với video này, xem [tại đây](#)

Video hướng dẫn cách download source sau khi kết thúc video này, xem [tại đây](#)

Chapter 9: Tư duy phân tích database

Giới thiệu đề tài dự án thực hành cuối khóa, cũng như tư duy phân tích database, cách thiết kế table và mối quan hệ giữa các đối tượng trong dự án thực hành.

#44. Giới thiệu đề bài

Ý tưởng:

<https://itviec.com/>

<https://topdev.vn/>

Đăng tin tuyển dụng, việc làm => kết hợp việc phân quyền

- Ứng viên có thể tìm việc làm theo skills
- Nhà tuyển dụng có thể đăng việc làm
- Đặt lịch (schedule) gửi mail cho subscribers

Phân tích chi tiết chức năng

- Đăng ký, đăng nhập (basic: local)
- Tạo skills để search
- nhà tuyển dụng : thông tin giới thiệu về cty
- ứng viên: người có thể xem bài đăng tuyển dụng và gửi CV
- admin: duyệt CV rồi mới gửi tới nhà tuyển dụng

Nếu 1 ứng viên subscribe 1 skills => gửi mail hàng tuần về những jobs này

#45. Phân tích model & relationship

Warning: với NoSQL, không nên dựa vào relationship, đây không phải là cách làm tối ưu nhất.

Link download file excel [tại đây](#)

1. Đối tượng

Actors: (người sử dụng hệ thống)

- **Ứng viên** (employee)
- **Nhà tuyển dụng**: company
- **admin**

đối tượng khác:

- **CV** của ứng viên
- **jobs** đăng tuyển
- **Skills** để search

- **role**: vai trò của user trong hệ thống (admin, hr...)

- **permission**: quyền hạn sử dụng hệ thống - ám chỉ api của backend

2. Relationship

1 ứng viên có thể gửi đi nhiều CV 1 lúc

1 CV chỉ thuộc 1 ứng viên

1 nhà tuyển dụng có thể đăng nhiều jobs

1 job chỉ thuộc 1 nhà tuyển dụng

1 skills có thể thuộc nhiều jobs

1 jobs có thể có nhiều skills

Về phân quyền:

1 permission (quyền hạn), ám chỉ 1 apis của backend

1 role (vai trò), bao gồm nhiều quyền hạn

1 người dùng sẽ có 1 role duy nhất. nếu muốn merge role => tạo role mới

1 users -> có 1 role

1 role có thể có nhiều user cùng có role này

1 role -> có nhiều permission (apis)

1 permission -> có thể thuộc nhiều role khác nhau

#46. Thiết kế model

1. Users: (admin, ứng viên, nhân viên cty -> phân biệt nhờ role)

- name: string
- email: string <unique>
- password: string
- age: number
- gender: string
- address: string
- company: object {_id, name}
- role: string
- refreshToken: string;
-
- createdAt: Date
- updatedAt: Date
- isDeleted: boolean
- createdBy: object {_id, email}
- updatedBy: object {_id, email}
- deleteBy: object {_id, email}

2. Companies

- name: string
- address: string
- description: string <markdown>
-
- createdAt: Date
- updatedAt: Date
- isDeleted: boolean
- createdBy: object {_id, email}
- updatedBy: object {_id, email}
- deleteBy: object {_id, email}

3. Resume

- email: string
- userId: objectId
- url: string
- status: string //PENDING-REVIEWING-APPROVED-REJECTED

history: array object [{ status: string, updatedAt: Date, updatedBy: {_id, email}}]

- createdAt: Date
- updatedAt: Date
- isDeleted: boolean
- createdBy: object {_id, email}
- updatedBy: object {_id, email}
- deleteBy: object {_id, email}

4. Jobs

- name: string
- skill: string
- company: string
- location: string
- salary: number
- quantity: string (số lượng tuyển)
- level: string //INTERN/FRESHER/JUNIOR/SENIOR
- description: string <markdown>
- startDate: Date
- endDate: Date
- isActive: boolean

- createdAt: Date
- updatedAt: Date
- isDeleted: boolean
- createdBy: object {_id, email}
- updatedBy: object {_id, email}
- deleteBy: object {_id, email}

5. Subscribers

- email: string
- skill: string

6. Roles

- name: string <unique>
- description: string
- isActive: boolean
- permissions: array object

- createdAt: Date
- updatedAt: Date
- isDeleted: boolean
- createdBy: object {_id, email}
- updatedBy: object {_id, email}
- deleteBy: object {_id, email}

7. Permissions

- name: string
- path: string
- method: string
- description: string

- createdAt: Date
- updatedAt: Date
- isDeleted: boolean
- createdBy: object {_id, email}
- updatedBy: object {_id, email}
- deleteBy: object {_id, email}

Chapter 10: Mongoose Plugins

Để có thể sử dụng Mongoose (MongoDB) hiệu quả, chúng ta sẽ cùng nhau cài đặt các plugin hỗ trợ việc Query/Filter kết quả, thực hiện "soft-delete", tự động tạo "timestamp" và đặc biệt, khắc phục lỗi CORS với Nest.JS

#47. Timestamps plugin

1. Timestamps

<https://mongoosejs.com/docs/guide.html#timestamps>

Mục đích:

- khi create => tự động update trường createdAt/updatedAt
- khi update => tự động update trường updatedAt

cần set option: cho schema

<https://stackoverflow.com/a/71743707>

@Schema({timestamps: true }) và khai báo **@Prop createdAt/updatedAt**

Về source code video này:

Các file thay đổi tại video, xem [tại đây](#)

Source code ứng với video này, xem [tại đây](#)

Video hướng dẫn cách download source sau khi kết thúc video này, xem [tại đây](#)

#48. Soft-delete plugin

- Lưu ý:

Với dự án đã có sẵn data, cần update "data cũ" với 2 fields là deletedAt và isDeleted => duplicate data (or error nếu unique)

#soft-delete

<https://www.npmjs.com/package/soft-delete-plugin-mongoose>

Cài đặt:

npm i --save-exact soft-delete-plugin-mongoose@1.0.15

1. import plugin với nestjs:

<https://docs.nestjs.com/techniques/mongodb#plugins>

```
import { softDeletePlugin } from 'soft-delete-plugin-mongoose';
```

```
MongooseModule.forRootAsync({  
  imports: [ConfigModule],  
  useFactory: async (config: ConfigService) => ({  
    uri: config.get<string>('MONGO_DB_URL'),  
    connectionFactory: (connection) => {  
      connection.plugin(softDeletePlugin);  
      return connection;  
    }  
  }  
}),  
  inject: [ConfigService],  
}),
```

2. Merge type cho Model

<https://github.com/nour-karoui/mongoose-soft-delete>

//2nd way (nestjs way)

```
constructor(@InjectModel('Test') private readonly testModel: SoftDeleteModel<Test>) {}
```

lưu ý: update model thêm 2 fields sau:

deletedAt: null,

isDeleted: false

Về source code video này:

Các file thay đổi tại video, xem [tại đây](#)

Source code ứng với video này, xem [tại đây](#)

Video hướng dẫn cách download source sau khi kết thúc video này, xem [tại đây](#)

#49. Query Builder

Tài liệu: <https://www.npmjs.com/package/api-query-params>

Cài đặt:

npm i --save-exact api-query-params@5.4.0

Về source code video này:

Các file thay đổi tại video, xem [tại đây](#)

Source code ứng với video này, xem [tại đây](#)

Video hướng dẫn cách download source sau khi kết thúc video này, xem [tại đây](#)

#50. Setup dự án frontend

Bạn vui lòng sử dụng version nodejs 16.20.0 xuyên suốt khóa học này (để hạn chế lỗi không cần thiết)

Link download: <https://nodejs.org/download/release/v16.20.0/>

Với dự án frontend, bạn nên dùng version nodejs 16.20.0

Với các version khác, có thể gây ra lỗi không cần thiết (do thư viện không hỗ trợ)

Link source code github: <https://github.com/harypham/react-for-nest>

Source code final (cả khóa học) sẽ là nhánh master

Lưu ý: checkout code theo từng modules để thực hành song song với backend

#51. CORS là gì ?

Tài liệu: <https://developer.mozilla.org/en-US/docs/Web/HTTP/CORS>

**Demo CORS: checkout source code frontend sang nhánh cors
git checkout cors**

1.What?

Cross-Origin Resource Sharing (CORS): là cơ chế "server" cho phép "origin (domain/port)" từ phía "browser" được truy cập nguồn tài nguyên (APIs)

CORS sẽ xảy ra, khi browser gửi request lên server, và server "chưa cấu hình CORS)

2.Why?

ví dụ nếu không có CORS:
facebook.com

POST facebook.com/change-password

bạn dùng máy tính (browser), nhận được tin nhắn từ người lạ, là đường link : **hacker.com**
Vì tò mò, bạn click vào link trên, và 5 phút sau account FB bị đổi mật khẩu, Why ?

===

Cơ chế của browser:

- Khi bạn login thành công, sẽ lưu "**cookies**". Cookies này sẽ lưu tại máy tính bạn. gồm 2 thành phần:

- + **domain**: tên website sử dụng cookies này
- + **value**: giá trị của cookies

Nếu bạn login google thành công, máy tính bạn sẽ có cookies thứ nhất:

+ **domain: google.com**

+ value: ...

Nếu bạn login facebook thành công, máy tính bạn sẽ có cookies thứ hai:

+ **domain: facebook.com**

+ value: ...

Bạn tắt máy tính & đi ngủ. ngày hôm sau vào google, facebook không cần đăng nhập nữa, Why ?

===

Hiện tại, máy bạn đang có 2 cookies ở trên, khi bạn vào facebook.com, trình duyệt "sẽ tự động gửi" những cookies nào dành cho domain trên.

Như vậy, nhờ có cookies mà bạn "không cần đăng nhập nhiều lần".

Tương tự, khi vào google.com, browser sẽ tự động gửi kèm cookies ứng với domain này.

Check = cách F12, check request header

**=> như vậy, việc gửi kèm cookies này là do browser tự làm.
và cookies "đại diện/tượng trưng" cho user (mà không cần password)**

=====

Khi không có CORS:

- máy tính bạn "đã lưu" cookies của facebook
- bạn vào hacker.com, và trang này gọi request tới facebook.com => browser sẽ tự động gửi kèm cookies của bạn tới facebook.com

Nếu hacker.com thực hiện POST /change-password => change password dựa vào cookies, mà không cần người dùng nhập mật khẩu :v

Thực tế: do facebook.com và hacker.com là 2 domain khác nhau, nên sẽ bị chặn CORS default :v

#52.1 Làm sao để bypass CORS ?

1. update browser

Chỉ sử dụng cách này nếu bạn **“chỉ code Frontend”** và **“không kiểm soát backend”**

settings, extensions

2. Sử dụng Backend

- **Backend do bạn kiểm soát**, lỗi xảy ra khi frontend gọi lên backend => **fix ở backend**
(cách làm này sẽ được hướng dẫn ở video tiếp theo)

- **Backend do bên thứ 3 làm.**

chúng ta chỉ có "mỗi APIs để sử dụng", không có khả năng sửa đổi, cập nhật backend đấy

=> cần viết backend để gọi tới backend

Mô hình: **frontend = > backend (của bạn) => backend (của đối tác)**

Bạn có biết rằng , thư viện axios, dùng được cả frontend lẫn backend :v

#52.2 Setup Cors NestJS

Tài liệu:

<https://docs.nestjs.com/security/cors>

<https://github.com/expressjs/cors#configuration-options>

```
{  
  "origin": "*",  
  "methods": "GET,HEAD,PUT,PATCH,POST,DELETE",  
  "preflightContinue": false,  
  "optionsSuccessStatus": 204  
}
```

Về source code video này:

Các file thay đổi tại video, xem [tại đây](#)

Source code ứng với video này, xem [tại đây](#)

Video hướng dẫn cách download source sau khi kết thúc video này, xem [tại đây](#)

Chapter 11: Modules Company

Bài tập thực hành: tạo module "Company". Ngoài việc thực hiện CRUD, chúng ta sẽ tìm hiểu cách phân trang (paginate data), sử dụng Interceptor để xử lý data trước khi trả về client, customize message response cũng như "đánh version" cho API

#53. Generate Modules Company

- generate: **nest g resource companies --no-spec**

- tạo schema (lưu ý về timestamp và soft-delete)

- tạo DTO + validation

Về source code video này:

Các file thay đổi tại video, xem [tại đây](#)

Source code ứng với video này, xem [tại đây](#)

Video hướng dẫn cách download source sau khi kết thúc video này, xem [tại đây](#)

#54. Bài Tập Create Company

Yêu cầu:

Tạo mới endpoint:

(cần truyền token ở header)

POST /companies

Body { name, address, description }

Có validate dữ liệu khi tạo mới

Về source code video này:

Các file thay đổi tại video, xem [tại đây](#)

Source code ứng với video này, xem [tại đây](#)

Video hướng dẫn cách download source sau khi kết thúc video này, xem [tại đây](#)

#55. Update User Type (JWT)

1. Create user interface

// users.interface.ts

```
export interface IUser {  
  _id: string;  
  name: string;  
  email: string;  
  role: string;  
}
```

2. Update JWT token (issue token when login)

iss (issuer),
exp (expiration time),
sub (subject),
aud (audience),
iat : timestamp of creation of the JWT

//auth.service.ts

```
async login(user: IUser) {  
  const { _id, name, email, role } = user;  
  const payload = {  
    sub: "token login",  
    iss: "from server",  
    _id,  
    name,  
    email,  
    role  
  };  
  return {  
    access_token: this.jwtService.sign(payload),  
    _id,  
    name,  
    email,  
    role  
  };  
}
```

3. Update decode token

//jwt.strategy.ts

```
async validate(payload: IUser) {  
  const { _id, name, email, role } = payload;  
  return {  
    _id,  
    name,  
    email,  
    role  
  };  
}
```

4. Update message error

//local.strategy.ts

```
async validate(username: string, password: string): Promise<any> {  
  const user = await this.authService.validateUser(username, password);  
  if (!user) {  
    throw new UnauthorizedException("Invalid Username/Password!");  
  }  
  return user; //req.user  
}
```

//jwt-auth.guard.ts

```
handleRequest(err, user, info) {  
  // You can throw an exception based on either "info" or "err" arguments  
  if (err || !user) {  
    throw err || new UnauthorizedException("Token không hợp lệ");  
  }  
  return user;  
}
```

5. Move /login => auth controller

Về source code video này:

Các file thay đổi tại video, xem [tại đây](#)

Source code ứng với video này, xem [tại đây](#)

Video hướng dẫn cách download source sau khi kết thúc video này, xem [tại đây](#)

#56. Passing Req.user

Tài liệu:

<https://docs.nestjs.com/custom-decorators#param-decorators>

Mục đích: passing request to service => lấy thông tin users đang đăng nhập

//decorator/customize.ts

```
export const User = createParamDecorator(  
  (data: unknown, ctx: ExecutionContext) => {  
    const request = ctx.switchToHttp().getRequest();  
    return request.user;  
  },  
);
```

//update controller => passing user

```
@Post()  
create(@Body() createCompanyDto: CreateCompanyDto, @User() user: IUser) {  
  return this.companiesService.create(createCompanyDto, user);  
}
```

//update service

```
create(createCompanyDto: CreateCompanyDto, user: IUser) {  
  return this.companyModel.create({  
    ...createCompanyDto,  
    createdBy: {  
      _id: user._id,  
      email: user.email  
    }  
  })  
}
```

Về source code video này:

Các file thay đổi tại video, xem [tại đây](#)

Source code ứng với video này, xem [tại đây](#)

Video hướng dẫn cách download source sau khi kết thúc video này, xem [tại đây](#)

#57. Bài Tập Update Company

Yêu cầu:

Tạo endpoint:

Patch /companies/id

(lưu ý truyền token JWT ở header)

Body {

name, address, description

}

Có validate dữ liệu, và đồng thời cập nhật trường updatedBy.

Về source code video này:

Các file thay đổi tại video, xem [tại đây](#)

Source code ứng với video này, xem [tại đây](#)

Video hướng dẫn cách download source sau khi kết thúc video này, xem [tại đây](#)

#58. Bài Tập Delete Company

Đọc về limitation of plugin [tại đây](#)

Yêu cầu:

Tạo endpoint:

Delete /companies/id

Không cần truyền lên body (lưu ý truyền token JWT ở header)

đồng thời cập nhật trường deletedBy. (soft-delete)

Về source code video này:

Các file thay đổi tại video, xem [tại đây](#)

Source code ứng với video này, xem [tại đây](#)

Video hướng dẫn cách download source sau khi kết thúc video này, xem [tại đây](#)

#59. Query with Pagination

Tài liệu : <https://www.npmjs.com/package/api-query-params>

Input: Frontend truyền lên:

- page: number (trang hiện tại)
- limit: number (số lượng bản ghi muốn lấy)
- query string: điều kiện query (ví dụ tìm theo tên, tìm theo email...)

Output:

```
return {  
  meta: {  
    current: page, //trang hiện tại  
    pageSize: limit, //số lượng bản ghi đã lấy  
    pages: totalPages, //tổng số trang với điều kiện query  
    total: totalItems // tổng số phần tử (số bản ghi)  
  },  
  result //kết quả query  
}
```

Về source code video 59.1:

Các file thay đổi tại video, xem [tại đây](#)

Source code ứng với video này, xem [tại đây](#)

Video hướng dẫn cách download source sau khi kết thúc video này, xem [tại đây](#)

Về source code video 59.2:

Các file thay đổi tại video, xem [tại đây](#)

Source code ứng với video này, xem [tại đây](#)

Video hướng dẫn cách download source sau khi kết thúc video này, xem [tại đây](#)

Logic xử lý:

- Để phân trang dữ liệu, cần **tìm hiểu về offset và limit**
- Cần lấy ra @Query page/limit
- Count tổng bản ghi với điều kiện filter
- tính offset

@Query /skip/limit

```
const { filter, projection, population } = aqp(rq);  
let { sort } = aqp(rq);  
let offset = (+page - 1) * (+limit);  
let defaultLimit = +limit ? +limit : 10;
```

```
const totalItems = (await this.model.find(filter)).length;  
const totalPages = Math.ceil(totalItems / defaultLimit);
```

```
if (isEmpty(sort)) {  
  // @ts-ignore: Unreachable code error  
  sort = "-updatedAt"  
}
```

```
const result = await this.model.find(filter)  
  .skip(offset)  
  .limit(defaultLimit)  
  // @ts-ignore: Unreachable code error  
  .sort(sort)  
  .populate(population)  
  .exec();
```

```
let { sort } = <{sort: any}>aqp(rq);  
let { sort }: {sort: any} = aqp(rq);  
.sort(sort as any)
```

#60. Giới Thiệu Interceptor

Tài liệu:

<https://docs.nestjs.com/interceptors>

Interceptor là cách can thiệp vào "req và res", tương tự guard - thông minh hơn middleware

Reactive programming:

<https://rxjs.dev/>

Observable > < Promise

#61.1 Transform Response

Tài liệu:

<https://docs.nestjs.com/interceptors#response-mapping>

<https://stackoverflow.com/questions/60189849/how-to-format-response-before-sending-in-nest-js>

<https://docs.nestjs.com/interceptors#binding-interceptors>

Yêu cầu: thống nhất data trả ra của backend (trường hợp thành công):

```
{
  message: "",
  statusCode: "", //200 404
  data: ""
}
```

trường hợp lỗi :

<https://docs.nestjs.com/exception-filters>

default:

```
{
  statusCode: "",
  message: ""
}
```

// nếu dùng custom message:

```
{
  "statusCode": "",
  "message": "",
  "error": ""
}
```

//file: transform.interceptor.ts tham khảo [tại đây](#)

Về source code video này:

Các file thay đổi tại video, xem [tại đây](#)

Source code ứng với video này, xem [tại đây](#)

Video hướng dẫn cách download source sau khi kết thúc video này, xem [tại đây](#)

#61.2 Customize Message (Decorator)

Về source code video này:

Các file thay đổi tại video, xem [tại đây](#)

Source code ứng với video này, xem [tại đây](#)

Video hướng dẫn cách download source sau khi kết thúc video này, xem [tại đây](#)

#62. Version APIs

Tài liệu: <https://docs.nestjs.com/techniques/versioning#uri-versioning-type>

```
express: app.use("/api/v1", router)
```

Thực tế:

- Backend là server APIs
- Cần phân chia theo version của api

Ví dụ: <https://developers.facebook.com/docs/graph-api/get-started>

Do các hệ thống (lớn) phát triển theo thời gian, đôi khi có "breaking change" giữa các version => backend thường dùng "version"

- Nếu bạn code 1 mình, không cần version

- Nếu bạn sử dụng dịch vụ của bên thứ 3 (Facebook, Google, Momo), hoặc bạn viết "api cho đối tác dùng" => dùng version cho apis

```
app.enableVersioning({  
  type: VersioningType.URI,  
  // defaultVersion: ['1', '2'],  
  // prefix: 'api/v'  
});
```

sử dụng global-prefix cho backend:

//<https://docs.nestjs.com/faq/global-prefix>

Về source code video này:

Các file thay đổi tại video, xem [tại đây](#)

Source code ứng với video này, xem [tại đây](#)

Video hướng dẫn cách download source sau khi kết thúc video này, xem [tại đây](#)

Chapter 12: Modules User

Bài tập thực hành: tạo module User. Ngoài CRUD, chúng ta sẽ tìm hiểu về cookies, cơ chế tạo access token, refresh token cho client.

#63. Update User Schema

1. Update model

Users: (admin, ứng viên, nhân viên cty -> phân biệt nhờ role)

_id

- name: string
- email: string <unique>
- password: string
- age: number
- gender: string
- address: string
- company: object {_id, name}
- role: string
- refreshToken: string;
- createdAt: Date
- updatedAt: Date
- isDeleted: boolean
- createdBy: object {_id, email}
- updatedBy: object {_id, email}
- deleteBy: object {_id, email}

2. Update dto/validation

Validate nested object => company

<https://stackoverflow.com/questions/53786383/validate-nested-objects-using-class-validator-and-nestjs>

Về source code video này:

Các file thay đổi tại video, xem [tại đây](#)

Source code ứng với video này, xem [tại đây](#)

Video hướng dẫn cách download source sau khi kết thúc video này, xem [tại đây](#)

#64.1 Bài Tập CRUD Users

Lưu ý:

- đang hardcode ROLE & chưa áp dụng phân quyền ở đây
- về frontend: admin có view create user riêng. client có view register.
- Video này là bài tập thực hành, sẽ được chữa bài tại video tiếp theo.

Các endpoint cần tạo (6 apis)

1. POST /api/v1/auth/register

- **Không cần truyền lên JWT** (vì chức năng này dành cho client -> đăng ký tài khoản)
- Body : name , email , password , age , gender , address

Ở phía backend, hardcode 'role' === USER & cần hash password trước khi lưu.
Không cần cập nhật createdBy (vì không sử dụng jwt token)

The screenshot shows a REST client interface with the following details:

- Method:** POST
- URL:** http://localhost:8000/api/v1/auth/register
- Body Type:** x-www-form-urlencoded
- Body Data:**

Key	Value
name	hỏi dân it
email	eric@gmail.com
password	123456
age	15
gender	male
address	vietnam

Response:

```
{
  "statusCode": 201,
  "message": "Register a new user",
  "data": {
    "_id": ".....", //id của user được tạo
    "createdAt": "....." //thời gian tạo user
  }
}
```

2. POST /api/v1/users

- Cần truyền lên JWT
- Body:
 - name, email, password
 - age, gender, address, role
 - company**: object {_id, name}

Ở backend, tự động cập nhật **createdBy**: object {id, email}

=> Data postman sử dụng dạng RAW (json)



Lưu ý về việc import trùng tên User ở service (model và decorator)

Response:

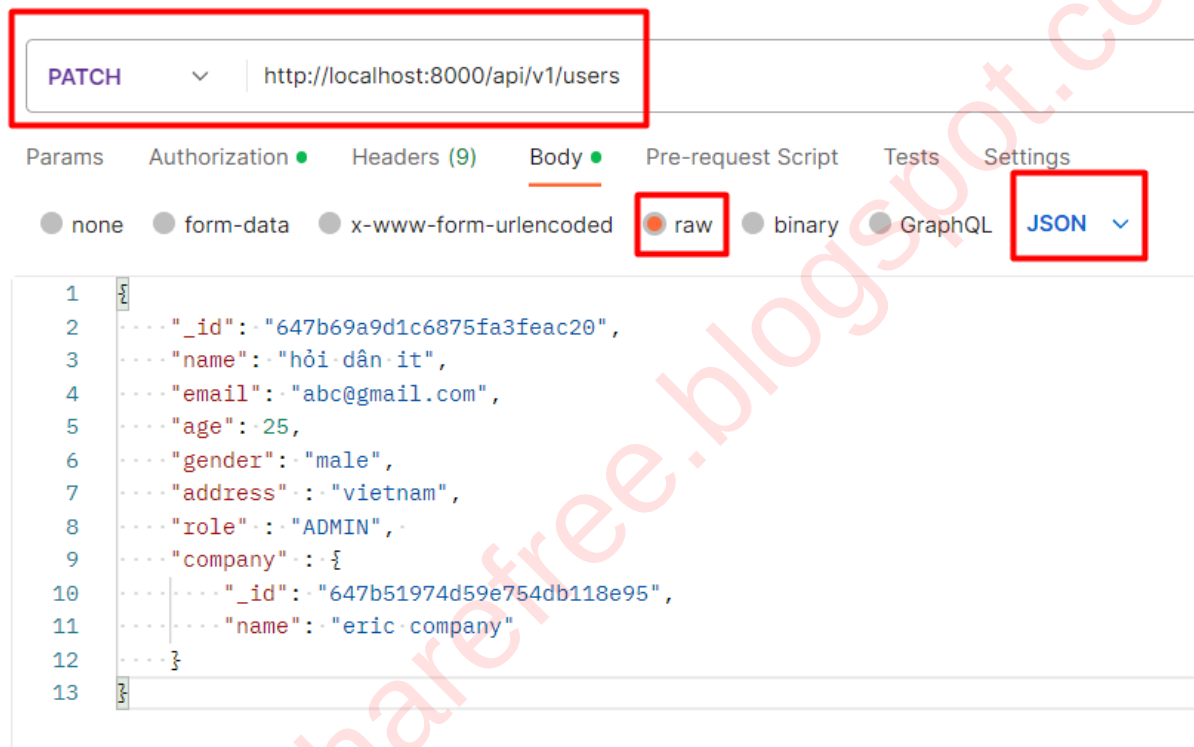
```
{
  "statusCode": 201,
  "message": "Create a new User",
  "data": {
    "_id": ".....",
    "createdAt": "....."
  }
}
```

3. PATCH /api/v1/users

- Cần truyền lên JWT
- Body:
_id , name , email, age, gender, address, role
company: object {_id, name}

Ở backend, tự động cập nhật **updatedBy**: object {id, email}

=> Data postman sử dụng dạng RAW (json)



Response:

```
{
  "statusCode": 200,
  "message": "Update a User",
  "data": {
    "acknowledged": ...,
    "modifiedCount": .....,
    "upsertedId": .....,
    "upsertedCount": .....,
    "matchedCount": .....
  }
}
```

4. DELETE /api/v1/users/:id

- Cần truyền lên JWT

The screenshot shows a REST client interface with a red box highlighting the method and URL. The method is 'DELETE' and the URL is 'http://localhost:8000/api/v1/users/64607ce86b2dad95a0f761e1'. Below the URL bar, there are tabs for 'Params', 'Authorization', 'Headers (7)', 'Body', 'Pre-request Script', 'Tests', and 'Settings'. The 'Authorization' tab is selected, showing a 'Bearer Token' dropdown and a text input field for the token. A warning message states: 'Heads up! These parameters hold sensitive data. To keep this data secure while working in a collaborator, we recommend using variables. Learn more about variables'. The token value is partially visible as 'eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ...'.

Ở backend, tự động cập nhật **deletedBy**: object {id, email} và sử dụng soft-delete

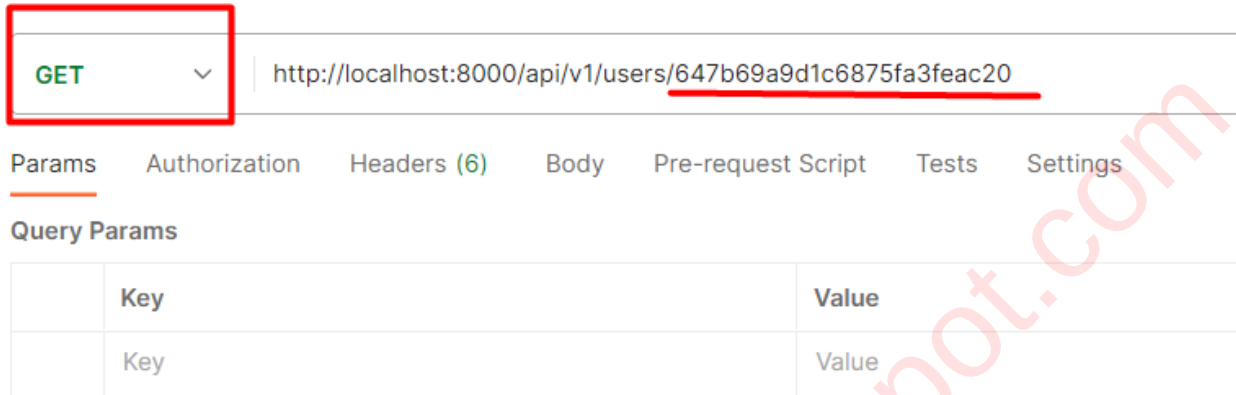
Response:

```
{
  "statusCode": 200,
  "message": "Delete a User",
  "data": {
    "deleted": .....
  }
}
```

5. GET /api/v1/users/:id

Fetch user by ID

- Không cần truyền lên JWT



GET ⌵ http://localhost:8000/api/v1/users/647b69a9d1c6875fa3feac20

Params Authorization Headers (6) Body Pre-request Script Tests Settings

Query Params

Key	Value
Key	Value

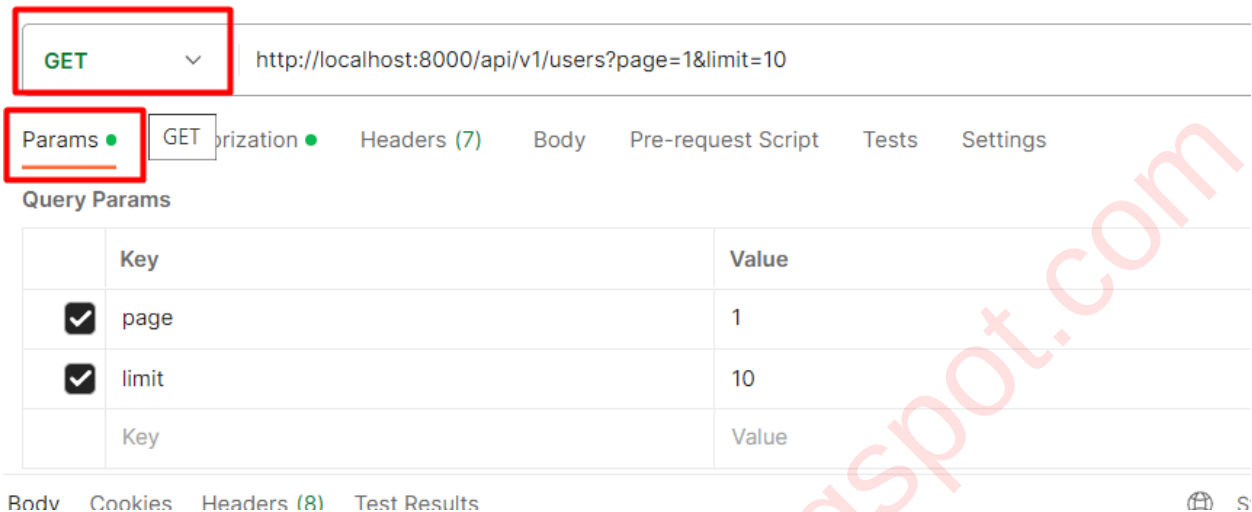
Response: (không trả về password) => trả về full record (without password)

```
{
  "statusCode": 200,
  "message": "Fetch user by id",
  "data": {
    "_id": ".....",
    "name": "hỏi dân it",
    "email": "abc@gmail.com",
    "age": 25,
    "gender": "male",
    "address": "vietnam",
    "company": ".....",
    "role": "ADMIN",
    "createdBy": "....",
    "isDeleted": ".....",
    "deletedAt": ".....",
    "createdAt": ".....",
    "updatedAt": ".....",
    "updatedBy": "...."
  }
}
```

6. GET /api/v1/users

Fetch users with paginate

- Cần truyền lên JWT



Response: (không trả về password) => trả về full record (without password)

```
{
  "statusCode": 200,
  "message": "Fetch user with paginate",
  "data": {
    "meta": {
      "current": 1,
      "pageSize": 10,
      "pages": .....,
      "total": ....
    },
    "result": [ ..... //users data ]
  }
}
```


#64.2 Hướng Dẫn Bài Tập CRUD Users

//todo

Về source code video này:

Các file thay đổi tại video, xem [tại đây](#)

Source code ứng với video này, xem [tại đây](#)

Video hướng dẫn cách download source sau khi kết thúc video này, xem [tại đây](#)

#65. Demo Chức năng login

- yêu cầu:

+ đăng nhập thành công, trả ra access_token và thông tin user

+ lưu refresh_token vào cookies

#66. API login

Yêu cầu:

1. Update API login

POST <http://localhost:8000/api/v1/auth/login>

Response:

```
{
  "statusCode": 201,
  "message": "User Login",
  "data": {
    "access_token": ".....",
    "user": {
      "_id": ".....",
      "name": ".....",
      "email": "....."
    }
  }
}
```

2. Refresh Token

Khi login, cần tạo ra Refresh token

- User login thành công, trước khi return access_token, **cần update thông tin user với refresh token được tạo**
- Refresh token có thể lưu dưới 2 định dạng: jwt hoặc là 1 chuỗi string ngẫu nhiên.
=> sử dụng jwt
- Cần lưu cookies (refresh_token) cho người dùng
<https://docs.nestjs.com/techniques/cookies>

Về source code video này:

Các file thay đổi tại video, xem [tại đây](#)

Source code ứng với video này, xem [tại đây](#)

Video hướng dẫn cách download source sau khi kết thúc video này, xem [tại đây](#)

#67. Set Cookies

Tài liệu: <https://docs.nestjs.com/techniques/cookies>

Cài đặt thư viện:

npm i --save-exact cookie-parser@1.4.6

npm i -D @types/cookie-parser@1.4.3

Về source code video này:

Các file thay đổi tại video, xem [tại đây](#)

Source code ứng với video này, xem [tại đây](#)

Video hướng dẫn cách download source sau khi kết thúc video này, xem [tại đây](#)

#68. Bài Tập API Get Account (F5 Refresh)

Bài toán:

- Với mô hình stateful, khi user F5 (refresh website), client sẽ gửi lên session_id (lưu ở cookies), server sẽ check session để biết user nào đang đăng nhập.
- **Với mô hình stateless**, không có sử dụng session, thay vào đấy là access_token và refresh_token.

=> khi user F5, cần gọi API của backend, vì client không có khả năng decode (giải mã) access_token để biết được ai là người đang đăng nhập.

Yêu cầu:

Tạo endpoint: **GET /api/v1/auth/account**

- **Chỉ cần truyền lên JWT ở header**

Response:

```
{
  "statusCode": 200,
  "message": "Get user information",
  "data": {
    "user": {
      "_id": "...",
      "name": "...",
      "email": "...",
      "role": "..."
    }
  }
}
```

Về source code video này:

Các file thay đổi tại video, xem [tại đây](#)

Source code ứng với video này, xem [tại đây](#)

Video hướng dẫn cách download source sau khi kết thúc video này, xem [tại đây](#)

#69.1 API Refresh Token (Part 1)

1. Bài toán:

Client sử dụng API, và khi access_token hết hạn => **server sẽ trả ra mã lỗi 401**

<https://developer.mozilla.org/en-US/docs/Web/HTTP/Status/401>

Code 401 (unauthorized), có nghĩa là không xác thực được người dùng => người dùng không truyền lên access_token, hoặc có truyền lên access_token, nhưng hết hạn.

=> khi nhận code 401, client (frontend) sẽ tự động gọi API refresh_token, sử dụng token này để đổi lấy {access_token, refresh_token} mới.

2. Yêu cầu: tạo endpoint

GET api/v1/auth/refresh

<https://docs.nestjs.com/techniques/cookies>

Response: (trả ra giống hệt như khi login)

```
{
  "statusCode": 200,
  "message": "Get User by refresh token",
  "data": {
    "access_token": "...",
    "user": {
      "_id": "...",
      "name": "...",
      "email": "...",
      "role": "...."
    }
  }
}
```

3. Các bước xử lý

- **Server lấy ra refresh_token từ cookies**
- **Server check (verify) để biết refresh_token có hợp lệ hay không ?**
- Server query database theo refresh_token
 - => lấy thông tin user
 - => issue access_token mới
- Server trả ra phản hồi (set cookies ứng với refresh_token mới)

Về source code video này:

Các file thay đổi tại video, xem [tại đây](#)

Source code ứng với video này, xem [tại đây](#)

Video hướng dẫn cách download source sau khi kết thúc video này, xem [tại đây](#)

#69.2. API Refresh Token (Part 2)

//Todo

Về source code video này:

Các file thay đổi tại video, xem [tại đây](#)

Source code ứng với video này, xem [tại đây](#)

Video hướng dẫn cách download source sau khi kết thúc video này, xem [tại đây](#)

#70. Bài Tập API Logout

Yêu cầu: Tạo endpoint

POST api/v1/auth/logout

Truyền lên JWT ở header

Response :

```
{
  "statusCode": 201,,
  "message": "Logout User",
  "data": "ok"
}
```

Xử lý ở backend:

- Update refresh_token === null (empty)
- Remove refresh_token ở cookies (remove cookies)
- Trả về phản hồi cho client

Về source code video này:

Các file thay đổi tại video, xem [tại đây](#)

Source code ứng với video này, xem [tại đây](#)

Video hướng dẫn cách download source sau khi kết thúc video này, xem [tại đây](#)

#71.1 Test giao diện frontend (Part 1)

//todo: fix bugs

Về source code video này:

Các file thay đổi tại video, xem [tại đây](#)

Source code ứng với video này, xem [tại đây](#)

Video hướng dẫn cách download source sau khi kết thúc video này, xem [tại đây](#)

#71.2 Test giao diện frontend (Part 2)

Link github: <https://github.com/harypham/react-for-nest>

Checkout sang nhánh code mới để test source code video này, sử dụng câu lệnh:
git checkout test-1

Với backend, khi frontend truyền lên "cookies", thì cần phải cấu hình CORS:

```
//config cors
app.enableCors(
  {
    "origin": true,
    "methods": "GET,HEAD,PUT,PATCH,POST,DELETE",
    "preflightContinue": false,
    "credentials": true
  }
);
```

Sự khác nhau giữa **"origin": true** và **"origin": *** là

***** : cho phép kết nối tới từ bất cứ nơi đâu

true: chỉ cho phép kết nối từ cùng origin với server,

ví dụ: localhost => localhost , không thể từ hoidanit.com => localhost.com

Về source code video này:

Các file thay đổi tại video, xem [tại đây](#)

Source code ứng với video này, xem [tại đây](#)

Video hướng dẫn cách download source sau khi kết thúc video này, xem [tại đây](#)

Chapter 13: Modules Job/Resume

Bài tập thực hành tạo module Job/Resume: CRUD job/resume, kết hợp việc sử dụng Multer để upload file cho Nest.JS, đồng thời sử dụng "ref" để "join data cho MongoDB"

#72. Bài Tập Tạo Module Jobs

`nest g resource jobs --no-spec`

1. Tạo Schema

jobs

- name: string
- skills: string []
- company: object { _id, name }
- location: string
- salary: number
- quantity: number
- level: string
- description : html <string>
- startDate: date
- endDate: date
- isActive: boolean

2. update dto (validate dữ liệu)

//todo

Về source code video này:

Các file thay đổi tại video, xem [tại đây](#)

Source code ứng với video này, xem [tại đây](#)

Video hướng dẫn cách download source sau khi kết thúc video này, xem [tại đây](#)

#73. Bài tập CRUD Jobs

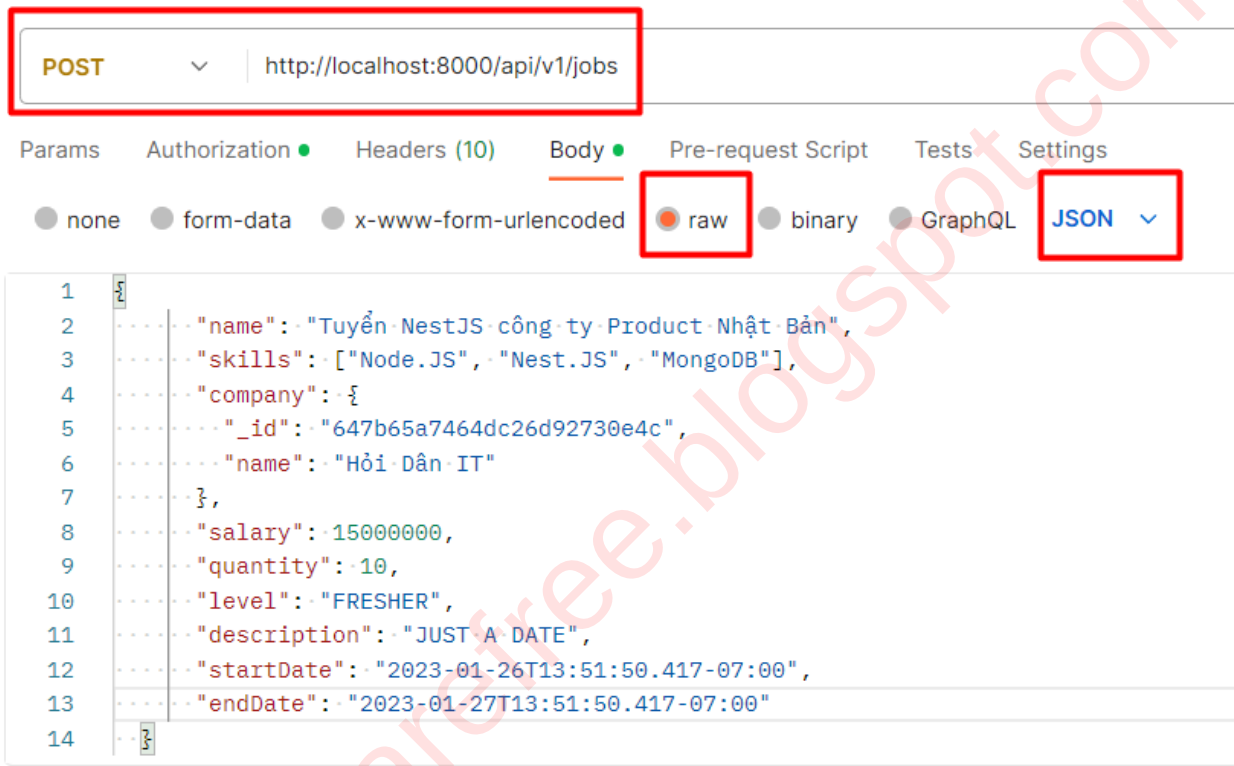
Hoàn thiện các endpoint sau:

1. Create a job

POST /api/v1/jobs

Yêu cầu:

- truyền JWT ở header
- body truyền dạng raw (JSON). Update thông tin createdBy khi tạo mới job



Response:

```
{
  "statusCode": 201,
  "message": "Create a new job",
  "data": {
    "_id": ".....",
    "createdAt": "....."
  }
}
```

Data mẫu khi tạo mới Job:

```
{
  "name": "Tuyển NestJS công ty Product Nhật Bản",
  "skills": ["Node.JS", "Nest.JS", "MongoDB"],
  "company": {
    "_id": "647b65a7464dc26d92730e4c",
    "name": "Hỏi Dân IT"
  },
  "salary": 15000000,
  "quantity": 10,
  "level": "FRESHER",
  "description": "JUST A DATE",
  "startDate": "2023-01-26T13:51:50.417-07:00",
  "endDate": "2023-01-27T13:51:50.417-07:00",
  "isActive": true
}
```

2. Update a job

PATCH /api/v1/jobs/id

Yêu cầu:

- truyền JWT ở header. Truyền động ID ở url
- body dạng raw (JSON). Update thông tin updatedBy khi tạo update job

REST client interface showing a PATCH request to update a job.

Method: PATCH | URL: http://localhost:8000/api/v1/jobs/6481d6210b16557ead4f271d

Params: Authorization (selected) | Headers (10) | Body (selected) | Pre-request Script | Tests | Settings

Body type: none | form-data | x-www-form-urlencoded | raw (selected) | binary | GraphQL | JSON (selected)

JSON Body:

```
1 {
2   "name": "Tuyển NestJS công ty Product Nhật Bản",
3   "skills": ["Node.JS", "Nest.JS"],
4   "company": {
5     "_id": "647b65a7464dc26d92730e4c",
6     "name": "Hỏi Dân IT"
7   },
8   "salary": 15000000,
9   "quantity": 10,
10  "level": "FRESHER",
11  "description": "JUST A DATE",
12  "startDate": "2023-01-26T13:51:50.417-07:00",
13  "endDate": "2023-01-27T13:51:50.417-07:00",
14  "isActive": false
15 }
```

Response:

```
{
  "statusCode": 200,
  "message": "Update a job",
  "data": {
    "acknowledged": .....,
    "modifiedCount": .....,
    "upsertedId": .....,
    "upsertedCount": .....,
    "matchedCount": .....
  }
}
```

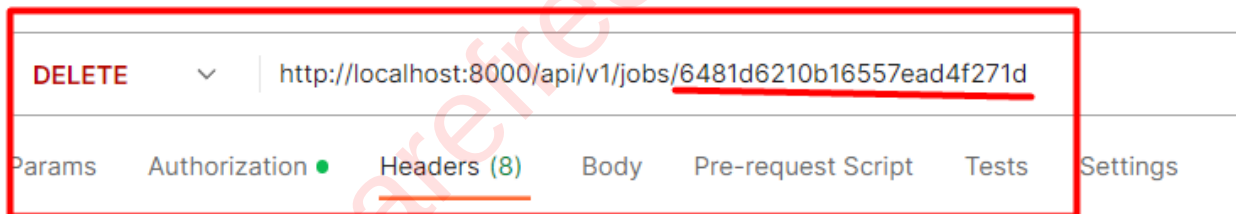
3. Delete a job

DELETE /api/v1/jobs/id

Yêu cầu:

- truyền JWT ở header. Truyền động ID ở url và sử dụng soft-delete

Update thông tin deletedBy khi tạo delete job



Response:

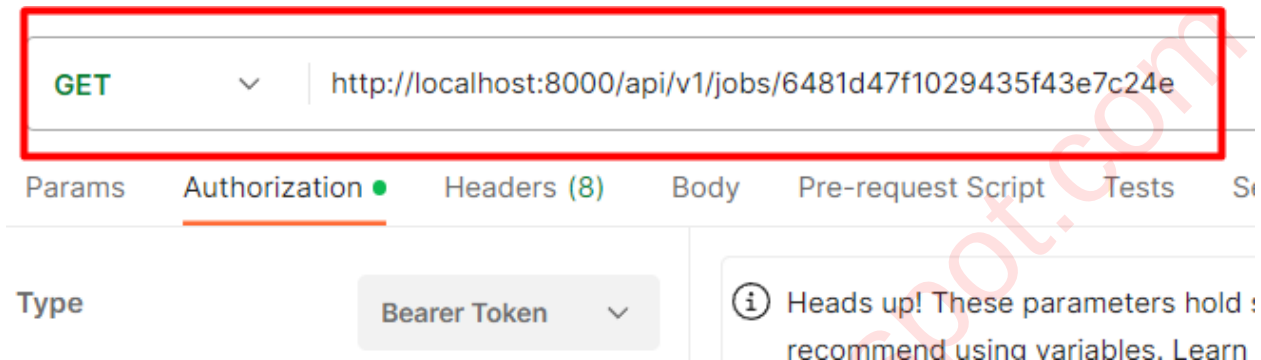
```
{
  "statusCode": 200,
  "message": "Delete a job",
  "data": {
    "deleted": .....
  }
}
```

4. Fetch job by id

GET /api/v1/jobs/id

Yêu cầu:

- truyền JWT ở header. Truyền động ID ở url



Response:

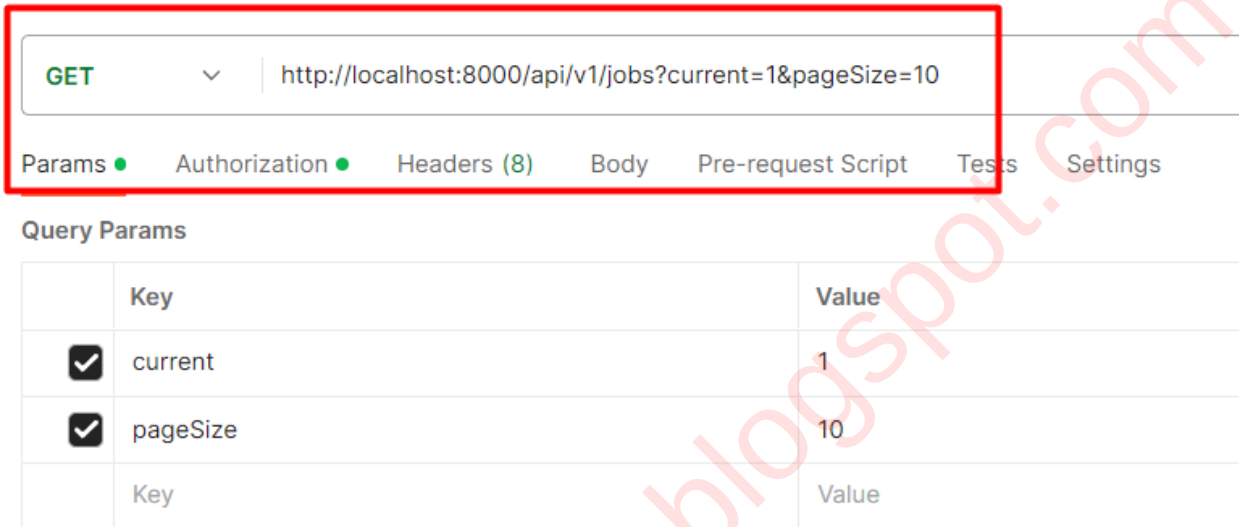
```
{
  "statusCode": 200,
  "message": "Fetch a job by id",
  "data": {
    "_id": ".....",
    "name": ".....",
    "skills": [ ..... ],
    "company": { "_id": "...." "name": "...."},
    "salary": .....,
    "quantity": .....,
    "level": ".....",
    "description": ".....",
    "startDate": "...",
    "endDate": "...",
    "isActive": ...,
    "createdBy": {
      "_id": ".....",
      "email": "....."
    },
    "isDeleted": ...,
    "deletedAt": ...,
    "createdAt": "....",
    "updatedAt": "...",
  }
}
```

5. Fetch job with paginate

GET /api/v1/jobs

Yêu cầu:

- truyền JWT ở header. Phân trang kết quả



GET

Params ● Authorization ● Headers (8) Body Pre-request Script Tests Settings

Query Params

	Key	Value
☒	current	1
☒	pageSize	10
	Key	Value

Response:

```
{
  "statusCode": 200,
  "message": "Fetch jobs with pagination",
  "data": {
    "meta": {
      "current": 1,
      "pageSize": 10,
      "pages": .....,
      "total": .....
    },
    "result": [..... ]
  }
}
```

#74. Hướng Dẫn Bài tập CRUD Jobs

//todo

Về source code video này:

Các file thay đổi tại video, xem [tại đây](#)

Source code ứng với video này, xem [tại đây](#)

Video hướng dẫn cách download source sau khi kết thúc video này, xem [tại đây](#)

#75. Giới Thiệu Upload files

Tài liệu: <https://docs.nestjs.com/techniques/file-upload>

1. Nguyên tắc khi upload file

- Sử dụng POST (create new data)
- Sử dụng form-data (xử lý file), data truyền dưới dạng **multipart/form-data**
- Sử dụng thư viện multer (hiệu năng cao) :
<https://www.npmjs.com/package/multer>

Trong khóa restful api, để đơn giản hóa, mình sử dụng thư viện :

<https://www.npmjs.com/package/express-fileupload>

2.Setup

Tạo service upload file: (vì sẽ sử dụng service này ở nhiều nơi)

nest g resource files --no-spec

Cài đặt thư viện: npm i -D @types/multer@1.4.7

(không cần cài đặt thư viện multer, vì Nestjs đã hỗ trợ sẵn multer, chỉ cần cài đặt package types/multer để hỗ trợ cú pháp khi code typescript)

3.Xử lý file

Tài liệu: <https://docs.nestjs.com/techniques/file-upload#basic-example>

File upload thực chất là 1 middleware

Req => middleware => Res

```
@Post('upload')
```

```
@UseInterceptors(FileInterceptor('file')) //tên field sử dụng trong form-data
```

```
uploadFile(@UploadedFile( ) file: Express.Multer.File) {
```

```
  console.log(file);
```

```
}
```

FileInterceptor('file' , options) => có 2 tham số đầu vào

<https://github.com/expressjs/multer#multeropts>

Về source code video này:

Các file thay đổi tại video, xem [tại đây](#)

Source code ứng với video này, xem [tại đây](#)

Video hướng dẫn cách download source sau khi kết thúc video này, xem [tại đây](#)

#76. Validate Upload Files

Tài liệu: <https://docs.nestjs.com/techniques/file-upload#file-validation>

1. Validate File

Sử dụng cách truyền thống, tương như sử dụng với Express và Multer

<https://stackoverflow.com/questions/49096068/upload-file-using-nestjs-and-multer>

```
@UploadedFile(  
  new ParseFilePipeBuilder()  
    .addFileTypeValidator({  
      fileType: 'jpeg',  
    })  
    .addMaxSizeValidator({  
      maxSize: 1000  
    })  
    .build({  
      errorHttpStatusCode: HttpStatus.UNPROCESSABLE_ENTITY  
    })  
)  
file: Express.Multer.File
```

https://developer.mozilla.org/en-US/docs/Web/HTTP/Basics_of_HTTP/MIME_types/Common_types

2. Giới thiệu về Upload Multiples

Trường hợp 1: upload array files (cùng key)

Tài liệu: <https://docs.nestjs.com/techniques/file-upload#array-of-files>

Trường hợp 2: upload multiple files (khác key)

<https://docs.nestjs.com/techniques/file-upload#multiple-files>

Về source code video này:

Các file thay đổi tại video, xem [tại đây](#)

Source code ứng với video này, xem [tại đây](#)

Video hướng dẫn cách download source sau khi kết thúc video này, xem [tại đây](#)

#77. Destination Upload File

Tài liệu: <https://docs.nestjs.com/techniques/file-upload>
<https://github.com/nestjs/nest/issues/2136>

1. Yêu cầu

- Cấu hình thư mục upload file (**destination**)
- **Thay đổi tên file** khi upload thành công (tránh trường hợp bị ghi đè/mất file do tên file upload bị trùng nhau)

2. Các bước thực hiện

- Config default options: **tại files modules**
<https://docs.nestjs.com/techniques/file-upload#default-options>
- Tạo class config, tham khảo:
`@Injectable()`
`export class MulterConfigService implements MulterOptionsFactory {`
 getRootPath = () => {
 return process.cwd();
 };

 ensureExists(targetDirectory: string) {
 fs.mkdir(targetDirectory, { recursive: true }, (error) => {
 if (!error) {
 console.log('Directory successfully created, or it already exists.'); return;
 }
 switch (error.code) {
 case 'EEXIST':
 // Error:
 // Requested location already exists, but it's not a directory.
 break;
 case 'ENOTDIR':
 // Error:
 // The parent hierarchy contains a file with the same name as the dir
 // you're trying to create.
 break;
 default:
 // Some other error like permission denied.
 console.error(error);
 break;
 }
 });
 }
}

```
createMulterOptions(): MulterModuleOptions {
  return {
    storage: diskStorage({
      destination: (req, file, cb) => {
        const folder = req?.headers?.folder_type ?? "default";
        this.ensureExists(`public/images/${folder}`);
        cb(null, join(this.getRootPath(), `public/images/${folder}`))
      },
      filename: (req, file, cb) => {
        //get image extension
        let extName = path.extname(file.originalname);

        //get image's name (without extension)
        let baseName = path.basename(file.originalname, extName);

        let finalName = `${baseName}-${Date.now()}${extName}`
        cb(null, finalName)
      }
    })
  };
}
```

Về source code video này:

Các file thay đổi tại video, xem [tại đây](#)

Source code ứng với video này, xem [tại đây](#)

Video hướng dẫn cách download source sau khi kết thúc video này, xem [tại đây](#)

#78. Bài Tập Update Company with Image

Yêu cầu:

- Tạo thêm field : **logo: string** đối với model Company
- Sửa 2 api là create và update company, cập nhật thêm field **logo: string**

Logic xử lý:

1. Khi **tạo mới/update** công ty, frontend upload ảnh

=> gọi api upload

=> backend sẽ upload (lưu trữ) file ảnh trên server, frontend nhận lại tên ảnh đã upload thành công

2. Frontend **submit tạo mới/update**, chỉ cần truyền **tên ảnh** đã upload thành công (**định dạng string**)

3. Backend **khi tạo mới/update công ty, chỉ lưu vào database tên ảnh** (field logo: string), còn **không cần xử lý logic upload file**, vì logic này đã được xử lý trước đó thông qua api upload file

Về source code video này:

Các file thay đổi tại video, xem [tại đây](#)

Source code ứng với video này, xem [tại đây](#)

Video hướng dẫn cách download source sau khi kết thúc video này, xem [tại đây](#)

#79. Test Giao Diện Frontend Upload File

//Part 1: fix bugs

```
@Post('upload')
@ResponseMessage("Upload Single File")
@UseInterceptors( FileInterceptor('fileUpload'))
```

=> bỏ decorator @Public

rename @UseInterceptors(FileInterceptor('fileUpload'))

- folder_type === "company"

- **@Public API fetch all company** => homepage

- **@Public API fetch all jobs** => homepage

- **@Public API fetch job by id** => view detail job

- **@Public API fetch company by id** => view detail company

- **khi tạo mới/update 1 jobs**, thông tin company sẽ bao gồm:

```
"company": {
  "_id": "647b65a7464dc26d92730e4c",
  "name": "Hỏi Dân IT"
  "logo: string
},
```

- **update thêm fields location** khi update/create jobs

- **Tạo fake data jobs/company (with logo)**

Link data fake:

<https://drive.google.com/drive/folders/1xqQETcMaksHvA9JSXXZNv0Bye76ZudO8?usp=sharing>

Về source code video này:

Các file thay đổi tại video, xem [tại đây](#)

Source code ứng với video này, xem [tại đây](#)

Video hướng dẫn cách download source sau khi kết thúc video này, xem [tại đây](#)

Part 2: run Frontend

Checkout sang branch **test-2**:

git checkout test-2

Cài đặt thư viện cần thiết:

npm i

tailieusharefree.blogspot.com

#80. Bài tập Tạo Modules Resumes

nest g resource resumes --no-spec

Model resume:

- email: string
- userId: objectId
- url: string
- status: string// PENDING-REVIEWING-APPROVED-REJECTED
- companyId: objectId
- jobId: objectId
- history: [
 - { status: string, updatedAt: Date, updatedBy: { _id, email} }]
- createdAt: Date
- updatedAt: Date
- deletedAt: Date
- isDeleted: boolean
- createdBy: { _id, email },
- updatedBy: { _id, email },
- deletedBy: { _id, email }

//todo: update Create/update DTO

//Bổ sung api get company by ID => phục vụ frontend

Về source code video này:

Các file thay đổi tại video, xem [tại đây](#)

Source code ứng với video này, xem [tại đây](#)

Video hướng dẫn cách download source sau khi kết thúc video này, xem [tại đây](#)

#81. Bài tập CRUD Resumes

Cần tạo 6 endpoints sau:

1. Create a Resume

POST /api/v1/resumes

The screenshot shows a REST client interface with a POST request to `http://localhost:8000/api/v1/resumes`. The request body is set to `x-www-form-urlencoded` and contains the following data:

	Key	Value
☒	url	abc
☒	companyId	647b51974d59e754db118e95
☒	jobId	647b51974d59e754db118e95
	Key	Value

Yêu cầu:

- Cần jwt ở header (người dùng đã đăng nhập).
- Body truyền lên:
url: string (url của cv)
companyId: string
jobId: string

Backend xử lý:

lưu các thông tin:

- email (lấy từ req.user/jwt)

- userId (lấy từ req.user)

- status: "PENDING"

- history: [

{

status: "PENDING",

updatedAt: new Date,

updatedBy: {

_id: user._id,

email: user.email

}

}

]

]

- createdBy: { _id, email} //lấy từ req.user

Response:

{

"statusCode": 201,

"message": "Create a new resume",

"data": {

_id: ".....",

"createdAt": "....."

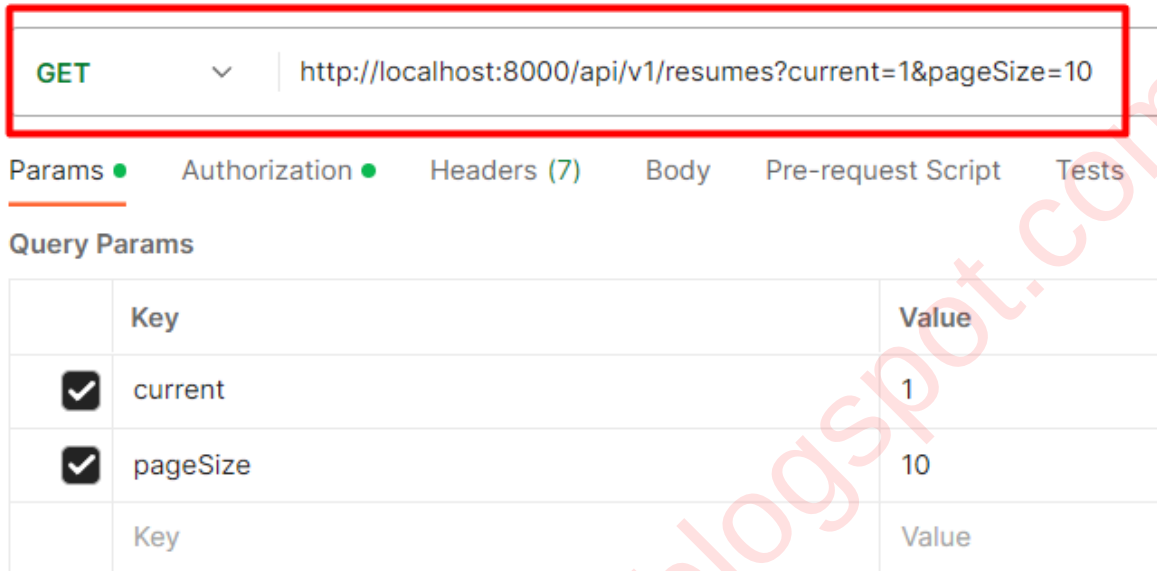
}

}

2. Fetch all resume (with paginate)

GET /api/v1/resumes

Yêu cầu: truyền lên JWT ở header, đồng thời query string ở url



GET

Params ● Authorization ● Headers (7) Body Pre-request Script Tests

Query Params

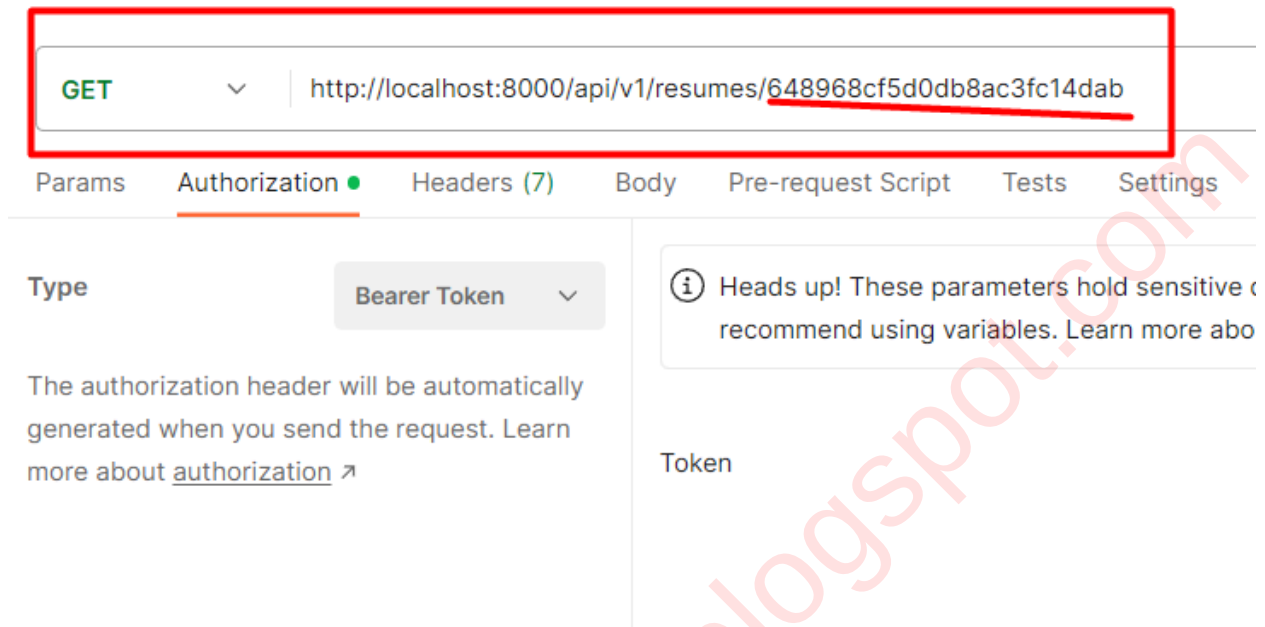
	Key	Value
☒	current	1
☒	pageSize	10
	Key	Value

Response:

```
{
  "statusCode": 200,
  "message": "Fetch all resumes with paginate",
  "data": {
    "meta": {
      "current": 1,
      "pageSize": 10,
      "pages": ...,
      "total": ....
    },
    "result": [
      {...}
    ]
  }
}
```

3. Fetch resume by id

GET /api/v1/resumes/id



Yêu cầu:

- Truyền động id ở url
- Truyền JWT ở header

Response:

```
{
  "statusCode": 200,
  "message": "Fetch a resume by id",
  "data": {
    //data resume here
  }
}
```

4. Change status a Resume (update a resume)

PATH /api/v1/resumes/id

The screenshot shows a REST client interface with a red box highlighting the method and URL. The method is **PATCH** and the URL is `http://localhost:8000/api/v1/resumes/648968cf5d0db8ac3fc14dab`. Below the URL bar, there are tabs for Params, Authorization, Headers (9), Body, Pre-request Script, Tests, and Settings. The **Body** tab is selected, and the content type is set to **x-www-form-urlencoded**. A table below shows the body data:

	Key	Value
☒	status	bbbbbb
	Key	Value

Yêu cầu:

- Header truyền JWT
- body truyền thêm **status**

Backend xử lý:

Update các thuộc tính: **status, updatedBy, history**

history là array => push thêm object mới vào array đã có

Response:

```
{
  "statusCode": 200,
  "message": "Update status resume",
  "data": {
    "Acknowledged": ...,
    "modifiedCount": ...,
    "upsertedId": ...,
    "upsertedCount": ...,
    "matchedCount": ...
  }
}
```

5. Delete a resume

DELETE /api/v1/resumes/id

DELETE

▼

http://localhost:8000/api/v1/resumes/648968cf5d0db8ac3fc14dab

ParamsAuthorization ●Headers (7)BodyPre-request ScriptTestsSettings

Query Params

	Key	Value
	Key	Value

Yêu cầu:

- Truyền JWT ở header
- Truyền động id trên url
- Sử dụng soft-delete

Response:

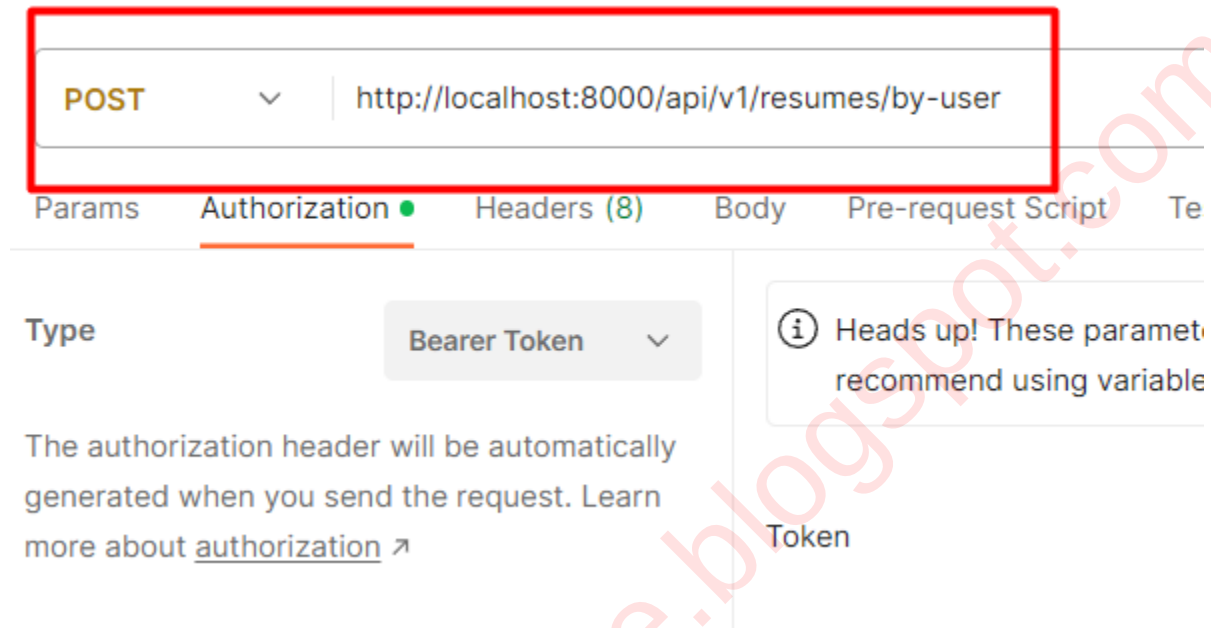
```
{
  "statusCode": 200,
  "message": "Delete a resume by id",
  "data": {
    "deleted": ....
  }
}
```

6. get CV by user

=> **frontend**, sau khi apply a job, có thể xem lại lịch sử

POST /api/v1/resumes/by-user

không dùng get => trùng với id



Yêu cầu:

- Truyền lên JWT ở header

Response:

```
{
  "statusCode": 201,
  "message": "Get Resumes by User",
  "data": [
    //data resumes
  ]
}
```

#82. Hướng Dẫn Bài tập CRUD Resumes

//todo

Về source code video này:

Các file thay đổi tại video, xem [tại đây](#)

Source code ứng với video này, xem [tại đây](#)

Video hướng dẫn cách download source sau khi kết thúc video này, xem [tại đây](#)

#83.1 Fetch Data with Ref

- fetch resume by Id => with Ref

<https://www.npmjs.com/package/api-query-params>

<https://docs.nestjs.com/techniques/mongodb>

- fetch resume with paginate

http://localhost:8000/api/v1/resumes?current=1&pageSize=10&populate=companyId,jobId&fields=companyId._id,companyId.name,companyId.logo,jobId._id,jobId.name

Về source code video này:

Các file thay đổi tại video, xem [tại đây](#)

Source code ứng với video này, xem [tại đây](#)

Video hướng dẫn cách download source sau khi kết thúc video này, xem [tại đây](#)

#83.2 Test Giao Diện Frontend

Để test giao diện, checkout sang branch **test-3**

git checkout test-3

npm i

Chapter 14: Modules Permission & Roles

Bài tập thực hành tạo module Permission/roles : CRUD Permissions/role, kết hợp áp dụng NestJS Life Cycle để tạo fake data cho dự án Backend.

#84. Bài tập Tạo Modules Permission & Roles

Sử dụng command:

nest g resource permissions --no-spec

nest g resource roles --no-spec

Mô hình phân quyền:

1 user => có 1 role . 1 role => có n permissions

=> 1 user có n permissions khi sử dụng hệ thống

Sau này muốn thay đổi permission của user, chỉ cần thay đổi role là xong
(update role hiện tại, hoặc tạo role mới => gán user vào)

1.Model Permission (Quyền hạn sử dụng hệ thống)

Mỗi permission chính là 1 api ở backend

- name : string
- apiPath : string
- method: string
- module: string //thuộc modules nào ?
- createdAt
- updatedAt
- deletedAt
- isDeleted
- createdBy : {_id, email}
- updatedBy: {_id, email}
- deletedBy: {_id, email}

2. Model Role (Vai trò trong hệ thống)

- name: string
- description: string
- isActive: boolean
- **permissions**: [] // array of object id. Ví dụ: ["648968cf5d0db8ac3fc14dab", "648968cf5d0db8ac3fc14dab"...]
- createdAt
- updatedAt
- deletedAt
- isDeleted
- createdBy : { _id, email }
- updatedBy: { _id, email }
- deletedBy: { _id, email }

=> lưu ý cách khai báo schema

3. Create/Update DTO

//todo

Về source code video này:

Các file thay đổi tại video, xem [tại đây](#)

Source code ứng với video này, xem [tại đây](#)

Video hướng dẫn cách download source sau khi kết thúc video này, xem [tại đây](#)

#85. Bài tập CRUD Permissions

Tạo các endpoint sau:

1. Create a Permission

POST api/v1/permissions

The screenshot shows a REST client interface with a POST request to `http://localhost:8000/api/v1/permissions`. The request body is set to `x-www-form-urlencoded`. The body contains the following fields:

Key	Value
☒ name	fetch user
☒ apiPath	/users
☒ method	POST
☒ module	BLA BLA

Yêu cầu:

- Truyền JWT ở header
- Body truyền lên các tham số: **name, apiPath, method, module**

Xử lý ở backend:

- Trước khi lưu, cần check xem "apiPath" + "method" đã tồn tại chưa? Nếu đã tồn tại thì thông báo lỗi. Chỉ tạo mới permission có "apiPath" + "method" chưa tồn tại
- Lưu thêm thông tin **createdBy**

Response:

```
{
  "statusCode": 201,
  "message": "Create a new permission",
  "data": {
    "_id": ".....",
    "createdAt": "...."
  }
}
```

2. Update a Permission

PATCH /api/v1/permissions/:id

The screenshot shows a REST client interface with a red box highlighting the method and URL. The method is 'PATCH' and the URL is 'http://localhost:8000/api/v1/permissions/648a767ca84217a045981e73'. Below the URL bar, there are tabs for 'Params', 'Authorization', 'Headers (9)', 'Body', 'Pre-request Script', 'Tests', and 'Settings'. The 'Body' tab is selected, and the content type is set to 'x-www-form-urlencoded'. The body is a JSON object with four fields: 'name', 'apiPath', 'method', and 'module', each with a checked checkbox and a value.

	Key	Value
☒	name	àd
☒	apiPath	àdasf
☒	method	ádf
☒	module	ádf
	Key	Value

Yêu cầu:

- Truyền lên JWT ở header
- Truyền động ID trên url

Response:

```
{
  "statusCode": 200,
  "message": "Update a permission",
  "data": {
    "acknowledged": .....,
    "modifiedCount": ...,
    "upsertedId": ...,
    "upsertedCount": .....,
    "matchedCount": ....
  }
}
```

3. Fetch Permission with paginate

GET /api/v1/permissions

GET

⌵

http://localhost:8000/api/v1/permissions?current=1&pageSize=10

Params ● Authorization ● Headers (7) Body Pre-request Script Tests Settings

Query Params

	Key	Value
☒	current	1
☒	pageSize	10
	Key	Value

Yêu cầu:

- Truyền JWT ở header
- Truyền động params để phân trang

Response:

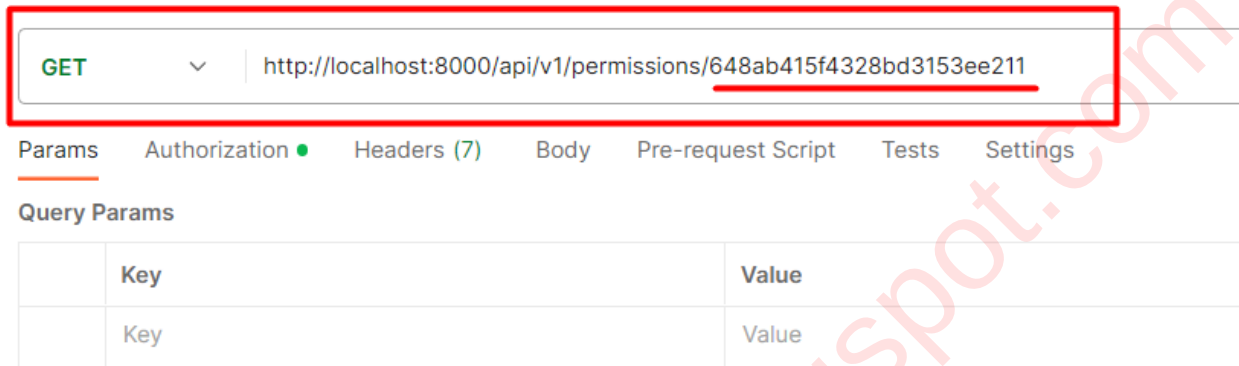
```
{
  "statusCode": 200,
  "message": "Fetch permissions with paginate",
  "data": {
    "meta": {
      "current": 1,
      "pageSize": 10,
      "pages": .....,
      "total": .....
    },
    "result": [...]
  }
}
```

4. Fetch Permission by ID

GET /api/v1/permissions/:id

Yêu cầu:

- Truyền JWT ở header
- Truyền động id ở header



GET http://localhost:8000/api/v1/permissions/648ab415f4328bd3153ee211

Params Authorization Headers (7) Body Pre-request Script Tests Settings

Query Params

Key	Value
Key	Value

Response:

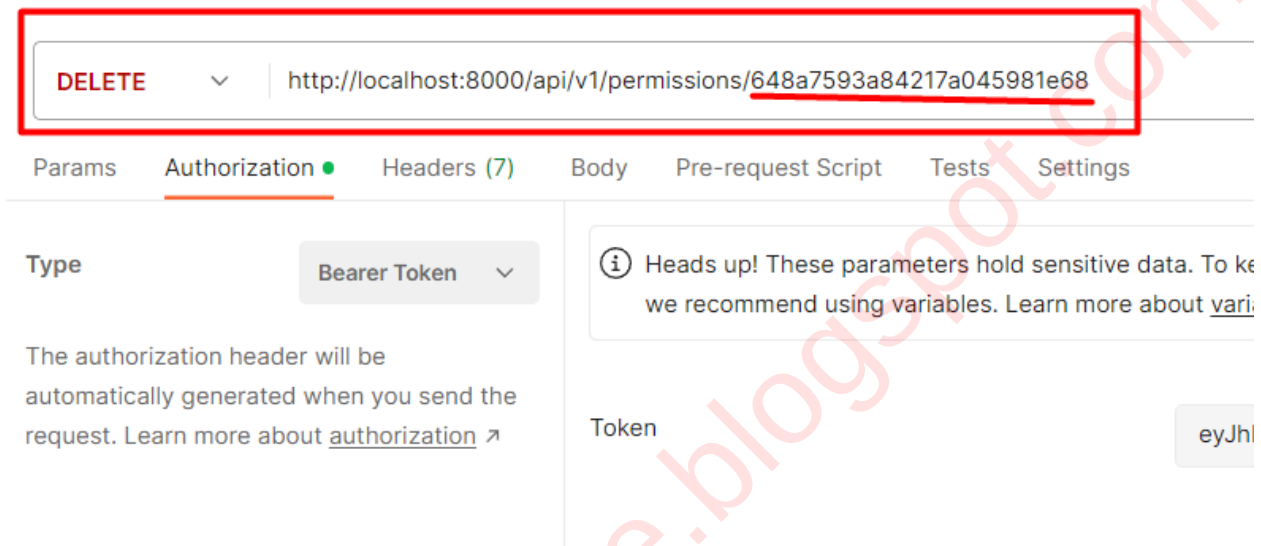
```
{
  "statusCode": 200,
  "message": "Fetch a permission by id",
  "data": {
    ...
  }
}
```

5. Delete a permission

DELETE /api/v1/permissions/:id

Yêu cầu:

- Truyền JWT ở header
- Truyền động id trên url
- **Sử dụng Soft delete**



Response:

```
{
  "statusCode": 200,
  "message": "Delete a permission",
  "data": {
    "deleted": ...
  }
}
```

#86. Hướng Dẫn Bài tập CRUD Permissions

//todo

Về source code video này:

Các file thay đổi tại video, xem [tại đây](#)

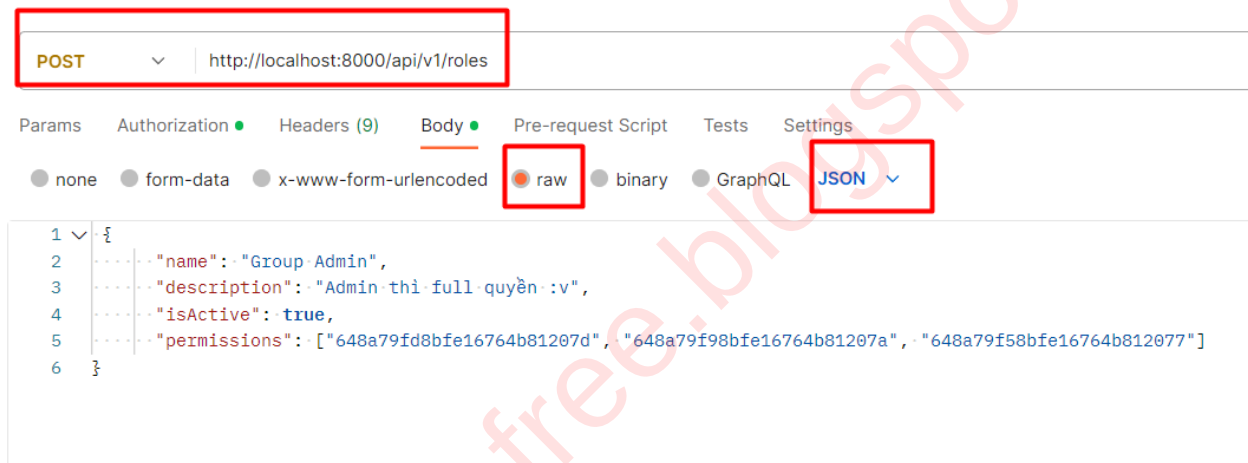
Source code ứng với video này, xem [tại đây](#)

Video hướng dẫn cách download source sau khi kết thúc video này, xem [tại đây](#)

#87. Bài tập CRUD Roles

1. Create Role

POST /api/v1/roles



Yêu cầu:

- Truyền JWT ở header
- Truyền body dưới dạng raw/JSON, **bao gồm các trường name, description, isActive, permissions**

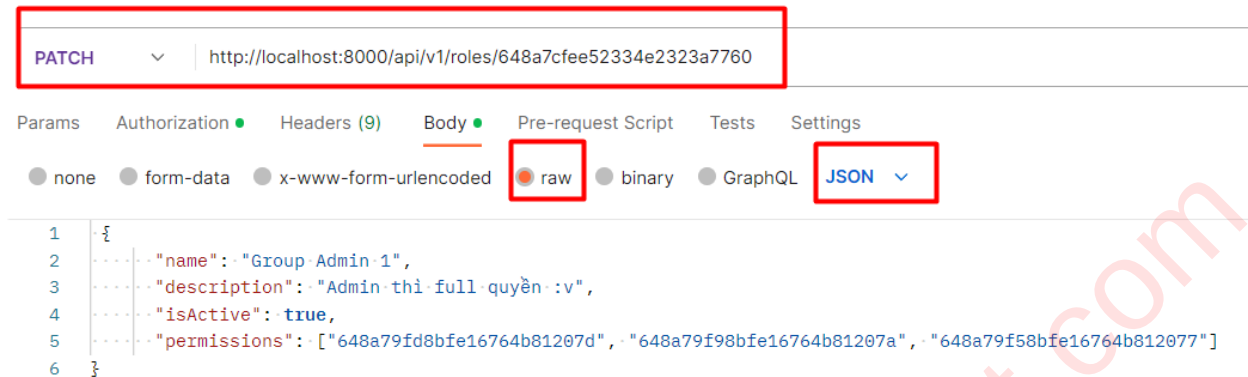
Backend xử lý: không lưu trùng tên của group. **Check "name" trước khi lưu**

Response:

```
{
  "statusCode": 201,
  "message": "Create a new role",
  "data": {
    "_id": ".....",
    "createdAt": "....."
  }
}
```

2. Update a Role

PATCH /api/v1/roles/:id



Yêu cầu:

- Truyền JWT ở header
- Truyền data ở body, dưới dạng raw/json

Backend cần check "name" trước khi cho update, đảm bảo "name" là khác nhau

Response:

```
{
  "statusCode": 200,
  "message": "Update a role",
  "data": {
    "acknowledged": .....,
    "modifiedCount": .....,
    "upsertedId": .....,
    "upsertedCount": ...
    "matchedCount": ...
  }
}
```


3. Fetch Role with paginate

GET /api/v1/role

Yêu cầu:

- Truyền jwt ở header
- Truyền động params trên url

GET ▼ http://localhost:8000/api/v1/roles?current=1&pageSize=10 ...

Params ● Authorization ● Headers (7) Body Pre-request Script Tests Settings

Query Params

	Key	Value
☒	current	1
☒	pageSize	10

4. Fetch Role by ID

GET /api/v1/role/:id

Yêu cầu:

- Truyền jwt ở header
- Truyền động id trên url

GET ▼ http://localhost:8000/api/v1/roles/648a7cf5e52334e2323a7760

Params Authorization ● Headers (7) Body Pre-request Script Tests Settings

Query Params

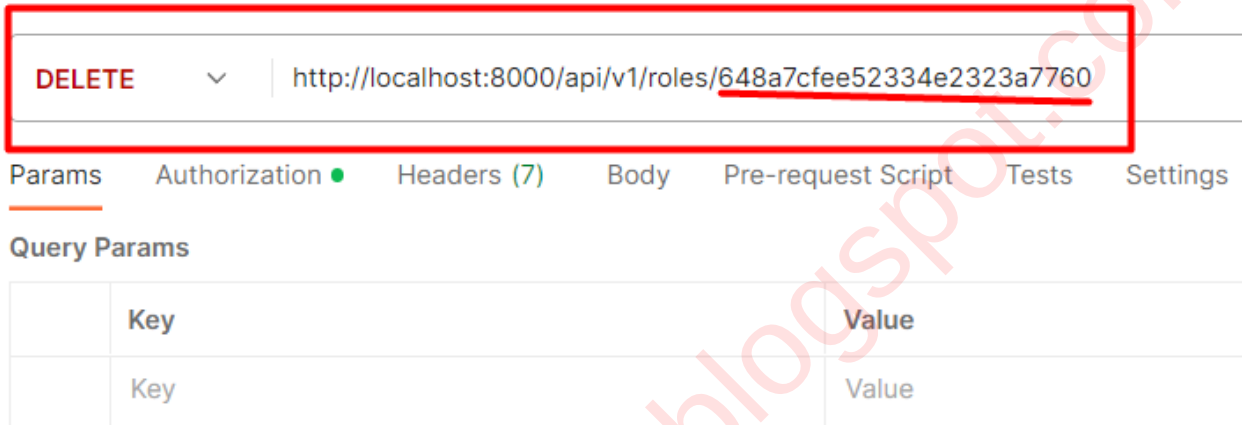
	Key	Value
	Key	Value

5. Delete a Role

DELETE /api/v1/role/:id

Yêu cầu:

- Truyền jwt ở header
- Truyền động id trên url
- **Sử dụng soft delete**



DELETE ⌵ http://localhost:8000/api/v1/roles/648a7cfee52334e2323a7760

Params Authorization ● Headers (7) Body Pre-request Script Tests Settings

Query Params

	Key	Value
	Key	Value

#88. Hướng Dẫn Bài tập CRUD Roles

//todo

Về source code video này:

Các file thay đổi tại video, xem [tại đây](#)

Source code ứng với video này, xem [tại đây](#)

Video hướng dẫn cách download source sau khi kết thúc video này, xem [tại đây](#)

#89. Fix bugs

1. lưu ý khi viết fetch Role by Id

=> trả thêm field module

```
async findOne(id: string) {  
  if (!mongoose.Types.ObjectId.isValid(id)) {  
    throw new BadRequestException("not found role")  
  }  
  
  return (await this.roleModel.findById(id)).populate({  
    path: "permissions",  
    select: { _id: 1, apiPath: 1, name: 1, method: 1, module: 1 }  
  });  
}
```

2. khi update role => ko check name

3. api fetch resume - by - user => populate companyId, jobId

```
async findByUser(user: IUser) {  
  return await this.resumeModel.find({  
    userId: user._id,  
  })  
  .sort("-createdAt")  
  .populate([  
    {  
      path: "companyId",  
      select: { name: 1 }  
    },  
    {  
      path: "jobId",  
      select: { name: 1 }  
    }  
  ])  
}
```

```
}  
  })  
}
```

4. fix api delete user/role

= không delete admin

= không delete role admin

5. Fix user role

=> chuyển qua dùng ref

6. api fetch user by id/ fetch with paginate

=> role : {_id, name}

7. api login

- trả thêm permission của người dùng

Về source code video này:

Các file thay đổi tại video, xem [tại đây](#)

Source code ứng với video này, xem [tại đây](#)

Video hướng dẫn cách download source sau khi kết thúc video này, xem [tại đây](#)

#90. Nest Lifecycle Event

Tài liệu: <https://docs.nestjs.com/fundamentals/lifecycle-events>

1. Lifecycle là gì ?

Vòng đời (lifecycle) là quá trình ứng dụng Nest.js chạy.

Ví dụ với con người, lifecycle gồm có: sinh ra => lớn lên => die

Tương tự, với **Nest.js**, gồm **3 phases chính**:

Phase 1: **initializing** (khởi tạo). ví dụ như khởi tạo connections, DI (dependencies injection), modules...

Phase 2: **running** (chính là lúc ứng dụng đã chạy thành công) => enjoy

Phase 3: **terminating**: trong quá trình chạy, nếu có "bugs" xảy ra, khiến server bị lỗi => die

Để cho nó không die nữa thì easy => restart lại server. quá trình trên lại lặp lại từ phase 1 tới phase 3

2. Tại sao cần lifecycle

- Sử dụng lifecycle để can thiệp vào quá trình chạy ứng dụng, ví dụ:
 - + khi ứng dụng bị die => do something :v
 - + khi ứng dụng chưa chạy lên => check logic, init fake data ...

3. Sử dụng với Nest.JS

Mục đích: tạo init data (dữ liệu thật, không phải dữ liệu fake) cho database

Trong thực tế, có 3 cách tạo data cho database:

- Cách 1: sử dụng script của database, tức là mở database => tự thêm dữ liệu vào
Với công ty, nếu database do team khác quản lý => dev không tự làm được điều này :v
- Cách 2: cài đặt/sử dụng thư viện hỗ trợ **migration/seeder** (rất tiếc là có rất ít thư viện làm điều này với Mongoose/mongodb)
- Cách 3: tạo data init từ code :v => dev có thể làm :v

Ý tưởng:

- Mỗi lần (everytime) ứng dụng Nest.js khởi tạo, sẽ gọi function để check:
- + Để biết có dữ liệu hay chưa => dùng hàm find, hoặc hàm count
- + Nếu có dữ liệu init rồi => không làm gì cả
- + Nếu chưa có dữ liệu => tạo init data

Cách làm:

Tài liệu: <https://docs.nestjs.com/fundamentals/lifecycle-events#usage>

nest g resource databases --no-spec

=> không cần generate crud

Viết hàm onModuleInit bên trong service

Về source code video này:

Các file thay đổi tại video, xem [tại đây](#)

Source code ứng với video này, xem [tại đây](#)

Video hướng dẫn cách download source sau khi kết thúc video này, xem [tại đây](#)

#91. Bài tập Tạo Sample Data

Xóa 3 collections users, roles và permissions trong database

- Mặc định 2 role: admin và user
- mặc định 2 users: admin và user
- mặc định permissions

Thứ tự tạo:

- Tạo permissions
 - Tạo role (với permissions) được tạo ở trên
 - Tạo user với role được tạo ở trên
- Cần setup tham số trong .env

#92. Hướng Dẫn Bài Tập Tạo Fake Data

Sử dụng logger:

<https://docs.nestjs.com/techniques/logger#using-the-logger-for-application-logging>

Download source code module databases:

https://drive.google.com/file/d/1B9zqP-qHt1dqcc0ZvBrAOfG_bY7m-oZO/view?usp=sharing

Về source code video này:

Các file thay đổi tại video, xem [tại đây](#)

Source code ứng với video này, xem [tại đây](#)

Video hướng dẫn cách download source sau khi kết thúc video này, xem [tại đây](#)

#93. Test Giao Diện Frontend

PART 1: FIX BUGS

<https://stackoverflow.com/questions/14504385/why-cant-you-modify-the-data-returned-by-a-mongoose-query-ex-findbyid>

Mục tiêu cần đạt được sau khi kết thúc video này:

Các apis sau:

- api/v1/auth/login
- api/v1/auth/account
- api/v1/auth/refresh

=> Cần trả thêm thêm thông tin về role và permissions

Khi register => tạo fake role user

Api login/refresh_token/fetch account, trả thêm permission []

Các phần kiểm tra (dựa vào phần fix tại #89):

1. lưu ý khi viết fetch Role by Id

=> trả thêm field module

```
async findOne(id: string) {
  if (!mongoose.Types.ObjectId.isValid(id)) {
    throw new BadRequestException("not found role")
  }

  return (await this.roleModel.findById(id)).populate({
    path: "permissions",
    select: { _id: 1, apiPath: 1, name: 1, method: 1, module: 1 }
  });
}
```

2. khi update role => ko check name

3. api fetch resume - by - user => populate companyId, jobId

```
async findByUsers(user: IUser) {
  return await this.resumeModel.find({
    userId: user._id,
  })
}
```



```
.sort("-createdAt")
  .populate([
    {
      path: "companyId",
      select: { name: 1 }
    },
    {
      path: "jobId",
      select: { name: 1 }
    }
  ])
}
```

4. fix api delete user/role

= không delete admin

= không delete role admin

5. Fix user role

=> chuyển qua dùng ref

6. api fetch user by id/ fetch with paginate

=> role : { _id, name }

7. api login

- trả thêm permission của người dùng

Về source code video này:

Các file thay đổi tại video, xem [tại đây](#)

Source code ứng với video này, xem [tại đây](#)

Video hướng dẫn cách download source sau khi kết thúc video này, xem [tại đây](#)

PART 2: TEST GIAO DIỆN

Checkout code sang nhánh mới **test-4**
git checkout test-4

Cài đặt thư viện cần thiết: **npm i**

tailieusharefree.blogspot.com

#94. Roles Guards

1. Lũồng JWT hiện tại

Khi User gửi 1 request sử dụng jwt ở header, tới 'protected endpoint' (tức là những api không sử dụng @public):

Bước 1: JWT auth guard sẽ được chạy (config trong file main.ts)

app.useGlobalGuards(new JwtAuthGuard(reflector));

Bước 2: Passport JWT strategy chạy (file jwt.strategy.ts)

Nếu token hết hạn/không hợp lệ => thông báo lỗi.

Trường hợp token hợp lệ, decode (giải mã) token => gán vào req.user

2. Các bước cần sửa

- Sau khi giải mã token jwt, cần gán thêm permissions vào req.user (tương tự như api login và api refresh token)

- Khi JWT auth guard được chạy, cần check permission tại đây.

Cần so sánh **req.route** và **permission**

Tài liệu: <https://expressjs.com/en/5x/api.html#req.route>

Về source code video này:

Các file thay đổi tại video, xem [tại đây](#)

Source code ứng với video này, xem [tại đây](#)

Video hướng dẫn cách download source sau khi kết thúc video này, xem [tại đây](#)

Chapter 15: Module Subscribers

Bài tập thực hành, tạo module Subscribers: CRUD subscribers, đồng thời tìm hiểu cách gửi email với Nest.js sử dụng "template xây dựng sẵn", kết hợp với "cron jobs" để gửi email tự động.

#95. Bài tập CRUD Subscribers

Yêu cầu:

1. Tạo module Subscribers:

nest g resource subscribers --no-spec

2. Fix lỗi permission ?

Sử dụng source code frontend tại branch test-4

- Fix lỗi permissions
- Tạo permissions cho module subscribers

3. Tạo DTO

//todo

4. Tạo CRUD endpoint

//todo

Về source code video này:

Các file thay đổi tại video, xem [tại đây](#)

Source code ứng với video này, xem [tại đây](#)

Video hướng dẫn cách download source sau khi kết thúc video này, xem [tại đây](#)

#96. Giới thiệu về gửi email với Node.js

Để gửi email, cần 2 thứ:

- Một là server để gửi email.

ví dụ: bạn ngồi tại cty A, sử dụng email của cty A, gửi email tới địa chỉ hoidanit@gmail.com

Email sẽ được vận chuyển từ Server Email của cty A tới Google (gmail).

Hiểu đơn giản, **server để gửi Email chính là "bưu điện"** (post office)

- Hai là nội dung email, chính là "lá thư" gửi tới người nhận (letter)

Ở đây, bưu điện (server) sẽ là nơi gửi lá thư (email) tới người nhận thông qua đường truyền internet.

Người nhận muốn đọc email => login vào server (chứa email) => thấy email được gửi tới

Cách Gửi email với Nodejs

Sử dụng thư viện Node Mailer : <https://nodemailer.com/about/>

Quá trình gửi email gồm 3 bước:

1. cài đặt thư viện: npm install nodemailer

2. config transporter

```
let transporter = nodemailer.createTransport(transport[, defaults])
```

ở đây, chúng ta cần setup server để gửi email (email này sẽ được xuất phát từ đâu)

3. sending email

```
transporter.sendMail(data[, callback])
```

data gửi đi sẽ bao gồm:

- nội dung email : text, images...
- **người nhận** => đây chính là đích đến của email

#97. Template Email

1. Tại sao cần template email ?

- Nếu bạn chỉ gửi email thông thường, không quan tâm tới format (cấu trúc, định dạng...) thì không cần "template"

- Template chính là cấu trúc định dạng cho 1 email, giúp email được gửi đi theo định dạng nhất định

- **Việc sử dụng "template" giúp email gửi đi được chuyên nghiệp hơn**

Ví dụ, nếu bạn cần gửi đi 1 loạt email, chỉ "khác nhau" tên người nhận
=> nên tạo ra template để gửi

Hiểu đơn giản, template, giống hệt như việc bạn đi làm giấy tờ "tại cơ quan hành chính"

Cơ quan đưa bạn "form mẫu" (template). bạn điền thông tin
=> end game (email đc gửi đi)

2. Viết template bằng công cụ gì ?

Template là "form mẫu", và việc của bạn là "điền thêm data" vào chỗ trống.

Cách 1: sử dụng function của javascript.

- Cách làm này cũng tạo ra được template, tuy nhiên, không thể tạo ra được email "đẹp".
ví dụ như:

+ format table

+ highlight text...

Cách 2 (được sử dụng phổ biến): sử dụng template với view engine, tức là viết dưới định dạng HTML

3. Ví dụ về template engine html

Thực chất nội dung body là 1 trang html

Việc sử dụng view engine giúp bạn có thể viết được "logic code" bên trong html.

ví dụ với handlebars:

<https://www.npmjs.com/package/handlebars>

```
var data = {  
  "name": "Alan", "hometown": "Somewhere, TX",  
  "kids": [  
    {"name": "Jimmy", "age": "12"},  
    {"name": "Sally", "age": "4"}  
  ]  
};
```

```
var source = "<p>Hello, my name is {{name}}. I am from {{hometown}}. I have " +  
  "{{kids.length}} kids:</p>" +  
  "<ul>{{#kids}}<li>{{name}} is {{age}}</li>{{/kids}}</ul>";
```

```
var template = Handlebars.compile(source);
```

```
var result = template(data);
```

```
// Would render:
```

```
// <p>Hello, my name is Alan. I am from Somewhere, TX. I have 2 kids:</p>
```

```
// <ul>
```

```
// <li>Jimmy is 12</li>
```

```
// <li>Sally is 4</li>
```

```
// </ul>
```

#98. Setup Tài khoản/Server gửi Email

- Trong thực tế, đa phần bạn sẽ sử dụng server email do bạn xây dựng
- Để cho đơn giản và tiết kiệm chi phí, chúng ta sử dụng Gmail để gửi email
- Sở đồ gửi email với NestJS và Gmail

1. Server backend (NestJS) => **cung cấp user/pass + nội dung email** cho Gmail
2. Gmail tiếp nhận thông tin, đọc thông tin email để biết người gửi/người nhận
3. Gmail gửi email tới đích đến

Như vậy, chúng ta cần cung cấp user/password của tài khoản gmail cho Google Email Server

1. Về app password

Tài liệu: <https://support.google.com/accounts/answer/185833>

App password là 1 chuỗi string gồm 16 ký tự, dùng để login (sử dụng) tài khoản Google của bạn.

App password khác với password bạn hay dùng để đăng nhập tài khoản google.

Why ?

Ngoài password, chúng ta sử dụng app password để có thể truy cập Google Account.

Cách làm này dùng để giao tiếp giữa server => server

(vì server không có giao diện để login). và ngoài ra, không cần để lộ password.

vì khi mất password => game over

Ví dụ: server facebook gọi api (cung cấp email/app password) => gọi tới server google để truy cập tài khoản.

Như vậy, app password là cách server/app có quyền để truy cập vào tài khoản google của bạn (mà không cần mật khẩu)

2. Yêu cầu của app password

- Tương tự như password, nên lưu trữ vào file .env và không nên public để hạn chế rủi ro. Nếu bạn muốn public, hãy sử dụng tài khoản email test của bạn, không nên dùng email chính.

- Tài khoản của bạn phải bật 2-step Verification

Tài liệu: <https://support.google.com/accounts/answer/185839>

3. Tạo app password

Tài liệu: <https://support.google.com/accounts/answer/185833>

Yêu cầu : cần tạo được app password. nếu không tạo được thì không thể làm các video tiếp theo -.-

#99. Nestjs - Mailer

Tài liệu: <https://nest-modules.github.io/mailer/docs/mailer.html>

1. Cài đặt thư viện

```
npm install --save-exact handlebars@4.7.7 @nestjs-modules/mailer@1.8.1  
nodemailer@6.9.3
```

```
npm install --save-dev @types/nodemailer@6.4.8
```

Theo thời gian, thư viện update. Vì vậy, có thể các bạn sẽ gặp lỗi như sau:

```
npm WARN EBADENGINE Unsupported engine {  
npm WARN EBADENGINE   package: 'cheerio@1.0.0',  
npm WARN EBADENGINE   required: { node: '>=18.17' },  
npm WARN EBADENGINE   current: { node: 'v16.20.0', npm: '8.19.4' }  
npm WARN EBADENGINE }  
npm WARN EBADENGINE Unsupported engine {  
npm WARN EBADENGINE   package: 'undici@6.19.8',  
npm WARN EBADENGINE   required: { node: '>=18.17' },  
npm WARN EBADENGINE   current: { node: 'v16.20.0', npm: '8.19.4' }  
npm WARN EBADENGINE }  
npm WARN EBADENGINE Unsupported engine {  
npm WARN EBADENGINE   package: 'whatwg-mimetype@4.0.0',  
npm WARN EBADENGINE   required: { node: '>=18' },  
npm WARN EBADENGINE   current: { node: 'v16.20.0', npm: '8.19.4' }  
npm WARN EBADENGINE }  
npm WARN EBADENGINE Unsupported engine {  
npm WARN EBADENGINE   package: 'whatwg-encoding@3.1.1',  
npm WARN EBADENGINE   required: { node: '>=18' },  
npm WARN EBADENGINE   current: { node: 'v16.20.0', npm: '8.19.4' }  
npm WARN EBADENGINE }
```

Để fix lỗi trên, các bạn làm như sau:

Bước 1: xóa file **package_lock.json** và thư mục **node_modules**

Bước 2: trong file **package.json**, thêm nội dung sau (phần khoanh đỏ)

```
"overrides": {  
  "@nestjs-modules/mailer": {  
    "inline-css": {  
      "cheerio": "1.0.0-rc.10"  
    }  
  }  
}
```



Bước 3: Cài lại thư viện với câu lệnh: **npm i**

Giải thích nguyên nhân lỗi:

Thư viện **@nestjs-modules/mailer**, cài đặt thư viện "inline-css", thư viện này lại cài đặt tiếp "cheerio", và thư viện cheerio ra version 1.0.0 có thay đổi (khác với version 1.0.0-rc.10)

2. Tạo Module

nest g resource mail --no-spec

=> không cần generate CRUD

- Lấy config từ file .env

//mail.controller.ts

```
@Get()
@Public()
@ResponseMessage("Test email")
async handleTestEmail() {
  await this.mailerService.sendMail({
    to: "haryphamdev@gmail.com",
    from: "Support Team" <support@example.com>, // override default from
    subject: 'Welcome to Nice App! Confirm your Email',
    html: '<b>welcome bla bla</b>', // HTML body content
  });
}
```

//mail.module.ts

```
@Module({
  imports: [
    MailerModule.forRootAsync({
      useFactory: async (configService: ConfigService) => ({
        transport: {
          host: 'smtp.gmail.com',
          secure: false,
          auth: {
            user: 'user@example.com',
            pass: 'topsecret',
          },
        },
      }),
      inject: [ConfigService],
    }),
  ],
  controllers: [MailController],
  providers: [MailService]
})
export class MailModule { }
```

=> chưa dùng template

Về source code video này:

Các file thay đổi tại video, xem [tại đây](#)

Source code ứng với video này, xem [tại đây](#)

Video hướng dẫn cách download source sau khi kết thúc video này, xem [tại đây](#)

#100. Gửi Email với template

```
import { HandlebarsAdapter } from '@nestjs-modules/mailer/dist/adapters/handlebars.adapter';
```

1. Cấu hình CLI dịch file template

//nest-cli.json

```
{
  "collection": "@nestjs/schematics",
  "sourceRoot": "src",
  "compilerOptions": {
    "assets": ["mail/templates/**/*.hbs"], // or "**/*.hbs" all files ending with .hbs
    "watchAssets": true // copy assets in watch mode
  }
}
```

2. Sử dụng template

Download file template trong video:

<https://drive.google.com/file/d/1hVR3w3VTnz3Map66QYC1qSH41RgUax30/view?usp=sharing>

Preview Template before send email

//set preview = true

=> lưu ý: config env để on/off tính năng preview

Cách tạo ra template theo ý muốn:

- Code HTML và style CSS trong cùng 1 file (không code riêng lẻ HTML và css) => sử dụng css với tag <style> </style>
- Convert HTML/CSS => inline CSS
- Mặc định Gmail sẽ bỏ qua CSS ở phần header => để đảm bảo an toàn hơn cho người dùng (tránh nhúng link javascript ở header)
- Nên sử dụng layout table (không dùng css flex)

3. Test gửi email với template

- Fill mặc định thông tin vào template
- Fill động thông tin vào template
- Sử dụng vòng lặp for để render list job

Tổng kết:

Tính năng tạo template và gửi mail, có thể áp dụng cho :

- + gửi mã code khi register/login
- + gửi link confirm khi quên mật khẩu

Về source code video #100.1:

Các file thay đổi tại video, xem [tại đây](#)

Source code ứng với video này, xem [tại đây](#)

Video hướng dẫn cách download source sau khi kết thúc video này, xem [tại đây](#)

Về source code video #100.2:

Các file thay đổi tại video, xem [tại đây](#)

Source code ứng với video này, xem [tại đây](#)

Video hướng dẫn cách download source sau khi kết thúc video này, xem [tại đây](#)

#101. Fetch Subscribers by skills

1. Truyền động tham số vào template

//todo

2. Fetch Subscribers by skills

```
const subscribers = await this.subscriberModel.find({ });
for (const subs of subscribers) {
  const subsSkills = subs.skills;
  const jobWithMatchingSkills = await this.jobModel.find({ skills: { $in: subsSkills } });
  //todo
  //build template
}
```

Hàm format VND:

```
salary: `${item.salary}`.replace(/\B(?=(\d{3})+(?!\d))/g, ',') + " đ",
```

Về source code video này:

Các file thay đổi tại video, xem [tại đây](#)

Source code ứng với video này, xem [tại đây](#)

Video hướng dẫn cách download source sau khi kết thúc video này, xem [tại đây](#)

#102. Test giao diện frontend

git checkout master

tạo thêm api:

+ fetch subscriber by email => lấy thông tin subscribe skills của user đăng nhập
//add decorator skip permission

fix bugs update ko truyền full data => bị mất dữ liệu
//bugs update dto

```
app.useGlobalPipes(new ValidationPipe({  
  whitelist: true,  
  // forbidNonWhitelisted: true  
}));
```

<https://docs.nestjs.com/techniques/validation#using-the-built-in-validationpipe>

+ upsert subscriber => update function update => set upsert = true
<https://stackoverflow.com/questions/9661081/mongoose-update-upsert>

Về source code video này:

Các file thay đổi tại video, xem [tại đây](#)

Source code ứng với video này, xem [tại đây](#)

Video hướng dẫn cách download source sau khi kết thúc video này, xem [tại đây](#)

#103. Cron job

Tài liệu:

<https://en.wikipedia.org/wiki/Cron>

<https://docs.nestjs.com/techniques/task-scheduling>

1. Cài đặt thư viện

npm i --save-exact @nestjs/schedule@3.0.1

npm install --save-exact cron@2.4.3

2. Sử dụng cron job

```
// * * * * *  
// | | | | |  
// | | | | | day of week  
// | | | | months  
// | | | day of month  
// | | hours  
// | minutes  
// seconds (optional)
```

0 10 0 5 * *

run at 12h10 a.m on the fifth day of the month.

Về source code video này:

Các file thay đổi tại video, xem [tại đây](#)

Source code ứng với video này, xem [tại đây](#)

Video hướng dẫn cách download source sau khi kết thúc video này, xem [tại đây](#)

#104. Tự động gửi email với cron job

//todo

Về source code video này:

Các file thay đổi tại video, xem [tại đây](#)

Source code ứng với video này, xem [tại đây](#)

Video hướng dẫn cách download source sau khi kết thúc video này, xem [tại đây](#)

#105. Nhận Xét về Dự Án Thực Hành Frontend

= Phần chưa làm

HR chỉ xem đc job của cty ấy. admin full quyền.

trước khi fetch list job => cần thêm điều kiện tìm theo company id

tương tự, trước khi fetch list resume

Chapter 16. Giới Thiệu Kiến Thức Nâng Cao

Giới thiệu các kiến thức nâng cao với NestJS, giúp tối ưu server backend. Ngoài ra, chúng ta sẽ tìm hiểu về cách upgrade version NestJS và cách build dự án backend với Docker.

#106. Authorization

Authentication: xác định người dùng là ai

=> chức năng login để biết là guest/users hệ thống

=> ở đây chúng ta sử dụng cơ chế JWT

Authorization: sau khi đã đăng nhập thành công (authenticated), chúng ta cần xác định, người dùng có thể làm gì ?

=> role và permission

Tài liệu: <https://docs.nestjs.com/security/authorization>

Về Cách làm hiện tại:

- Chia role và permission "tại database"

=> việc khai báo "role" và permission, nếu làm ko chuẩn (không chính xác), sẽ không chạy :v

Cách làm tối ưu hơn, sử dụng :

<https://docs.nestjs.com/security/authorization#integrating-casl>

Trang chủ: <https://casl.js.org/v6/en/>

Với CASL, hỗ trợ việc phân quyền sử dụng:

- Prisma (SQL)
- Mongoose (NoSQL)
- React/Angular/Vue (phân quyền ở frontend)

- CASL hỗ trợ việc phân quyền ở backend, và cũng hỗ trợ phân quyền ở databas (tương tự như cách chúng ta đã làm)

Tại sao mình không hướng dẫn CASL:

- Cú pháp viết không phù hợp với beginners
- CASL chỉ phù hợp khi bạn dùng backend để phân quyền người dùng.

Trong thực tế, nếu bạn dùng SSO, thì việc phân quyền sẽ làm luôn tại SSO.

tailieusharefree.blogspot.com

#107. Helmet

<https://docs.nestjs.com/security/helmet>

npm i --save-exact helmet@7.0.0

```
import helmet from 'helmet';  
// somewhere in your initialization file  
app.use(helmet());
```

=> Test lại upload file (rải CV)

Về source code video này:

Các file thay đổi tại video, xem [tại đây](#)

Source code ứng với video này, xem [tại đây](#)

Video hướng dẫn cách download source sau khi kết thúc video này, xem [tại đây](#)

#108. Rate Limiting

Tài liệu: <https://docs.nestjs.com/security/rate-limiting>

npm i --save-exact @nestjs/throttler@4.1.0

Về source code video này:

Các file thay đổi tại video, xem [tại đây](#)

Source code ứng với video này, xem [tại đây](#)

Video hướng dẫn cách download source sau khi kết thúc video này, xem [tại đây](#)

#109. Swagger

Tài liệu:

<https://docs.nestjs.com/openapi/introduction>

npm i --save-exact @nestjs/swagger@7.0.4

1. Todo

- add CLI => bind CLI (với DTO)

<https://docs.nestjs.com/openapi/cli-plugin#using-the-cli-plugin>

- add tags (cho controller)

- add bearer token

<https://stackoverflow.com/a/76223916>

```
.addBearerAuth(  
  {  
    type: 'http',  
    scheme: 'Bearer',  
    bearerFormat: 'JWT',  
    in: 'header',  
  },  
  'token',  
)  
.addSecurityRequirements('token')
```

- update login:

<https://stackoverflow.com/questions/57457231/how-nestjs-uses-nestjs-swagger-to-generate-documentation-on-passport-strategies>

tạo thêm dto:

```
//create-user.dto
```

```
export class UserLoginDto {
```

```
  @IsString()
```

```
  @IsNotEmpty()
```

```
  @ApiProperty({ example: 'ajanuw', description: '账号' })
```

```
  readonly username: string;
```

```
  @IsString()
```

```
  @IsNotEmpty()
```

```
  @ApiProperty({
```

```
    example: '123456',
```

```
    description: '密码',
```

```
  })
```

```
  readonly password: string;
```

```
}
```

```
//auth.controller
```

```
update controller:
```

```
@ApiBody({ type: UserLoginDto, })
```

2. Limitation

- Chưa add theo version

<https://github.com/nestjs/swagger/issues/1810>

- Chưa auto add to controller

- Chưa handle type MongoDB

Code tham khảo phần cấu hình swagger:

```
//config swagger
const config = new DocumentBuilder()
  .setTitle('NestJS Series APIs Document')
  .setDescription('All Modules APIs')
  .setVersion('1.0')
  .addBearerAuth(
    {
      type: 'http',
      scheme: 'Bearer',
      bearerFormat: 'JWT',
      in: 'header',
    },
    'token',
  )
  .addSecurityRequirements('token')
  .build();
const document = SwaggerModule.createDocument(app, config);
SwaggerModule.setup('swagger', app, document, {
  swaggerOptions: {
    persistAuthorization: true,
  }
});
```

Về source code video #109.1:

Các file thay đổi tại video, xem [tại đây](#)

Source code ứng với video này, xem [tại đây](#)

Video hướng dẫn cách download source sau khi kết thúc video này, xem [tại đây](#)

Về source code video #109.2:

Các file thay đổi tại video, xem [tại đây](#)

Source code ứng với video này, xem [tại đây](#)

Video hướng dẫn cách download source sau khi kết thúc video này, xem [tại đây](#)

//fix bug swagger và upload file

```
@Public()
@Post('upload')
@ResponseMessage("Upload Single File")
@UseInterceptors(FileInterceptor('fileUpload'))
@UseFilters(new HttpExceptionFilter())
@ApiConsumes('multipart/form-data')
@ApiBody({
  schema: {
    type: 'object',
    properties: {
      fileUpload: {
        type: 'string',
        format: 'binary',
      },
    },
  },
})
@ApiHeader({
  name: 'folder_type',
  required: true,
  description: 'Custom header',
})
uploadFile(@UploadedFile() file: Express.Multer.File) {
  return {
    fileName: file.filename
  }
}
```

Tham khảo thêm tại đây:

<https://docs.nestjs.com/openapi/operations#file-upload>

<https://github.com/nestjs/swagger/issues/417#issuecomment-562869578>

<https://docs.nestjs.com/openapi/operations#headers>

#110. Healthchecks

Tài liệu: <https://docs.nestjs.com/recipes/terminus>

npm install --save-exact @nestjs/terminus@10.0.1

1. Tạo health module

nest g module health

nest g controller health

2. Check database

//controller

@Controller('health')

export class HealthController {

 constructor(

 private health: HealthCheckService,

 private db: MongooseHealthIndicator,

) {}

 @Get()

 @Public()

 @HealthCheck()

 check() {

 return this.health.check([
 () => this.db.pingCheck('database'),
]);
 }

}

}

3. Devops

prometheus vs grafana

Về source code video này:

Các file thay đổi tại video, xem [tại đây](#)

Source code ứng với video này, xem [tại đây](#)

Video hướng dẫn cách download source sau khi kết thúc video này, xem [tại đây](#)

#111. Docs Whole App Backend

Tài liệu: <https://compodoc.app/>

<https://docs.nestjs.com/recipes/documentation>

npm i -D @compodoc/compodoc

- add to git ignore

Về source code video này:

Source code ứng với video này, xem [tại đây](#)

Video hướng dẫn cách download source sau khi kết thúc video này, xem [tại đây](#)

#112. Fix Bugs Còn Tồn Động

1. Upload DTO => whitelist: true

2. upload file invalid => still uploaded

```
createMulterOptions(): MulterModuleOptions {
  return {
    storage: diskStorage({
      destination: (req, file, cb) => {
        const folder = req?.headers?.folder_type ?? "default";
        this.ensureExists(`public/images/${folder}`);
        cb(null, join(this.getRootPath(), `public/images/${folder}`))
      },
      filename: (req, file, cb) => {
        //get image extension
        let extName = path.extname(file.originalname);

        //get image's name (without extension)
        let baseName = path.basename(file.originalname, extName);

        let finalName = `${baseName}-${Date.now()}${extName}`
        cb(null, finalName)
      },
    }),
    fileFilter: (req, file, cb) => {
      const allowedFileTypes = ['jpg', 'jpeg', 'png', 'gif', 'pdf', 'doc', 'docx'];
      const fileExtension = file.originalname.split('.').pop().toLowerCase();
      const isValidFileType = allowedFileTypes.includes(fileExtension);

      if (!isValidFileType) {
        cb(new HttpException('Invalid file type', HttpStatus.UNPROCESSABLE_ENTITY), null);
      } else {
        cb(null, true);
      },
      limits: {
        fileSize: 1024 * 1024 * 1 // 1MB
      }
    },
  };
}
```

Về source code video này:

Các file thay đổi tại video, xem [tại đây](#)

Source code ứng với video này, xem [tại đây](#)

Video hướng dẫn cách download source sau khi kết thúc video này, xem [tại đây](#)

#113. Upgrade NestJS Version

Tài liệu:

<https://docs.nestjs.com/migration-guide>

1. Update version thư viện

Với npm, chúng ta có options để update với npm:

<https://docs.npmjs.com/cli/v9/commands/npm-update>

Cài đặt thư viện check version mới của package:

<https://www.npmjs.com/package/npm-check-updates>

Cài đặt global: **npm install -g npm-check-updates**

or dùng với npx: **npx npm-check-updates**

Sử dụng câu lệnh sau để check version update: **ncu**

Lưu ý: sử dụng version control để kiểm soát file package.json

Vì nếu update failed => cài đặt lại version cũ

Để check update "tất cả" packages, sử dụng: **ncu -u**

Trường hợp update "tất cả" gây ra lỗi => chỉ update các package cần thiết

Câu lệnh: **ncu -u sẽ làm thay đổi package.json**

=> cần chạy câu lệnh npm i để cài đặt

2. Một vài lỗi thường gặp

```
npm ERR! code ERESOLVE
npm ERR! ERESOLVE could not resolve
npm ERR!
npm ERR! While resolving: nest-basic-hoidanit@0.0.1
npm ERR! Found: @nestjs/core@9.4.0
npm ERR! node_modules/@nestjs/core
npm ERR!   @nestjs/core@"10.0.3" from the root project
npm ERR!   node_modules/@nestjs-modules/mailer
npm ERR!     @nestjs-modules/mailer@"1.8.1" from the root project
npm ERR!   7 more (@nestjs/mongoose, @nestjs/platform-express, ...)
npm ERR!
npm ERR! Could not resolve dependency:
npm ERR! @nestjs/core@"10.0.3" from the root project
npm ERR!
npm ERR! Conflicting peer dependency: @nestjs/platform-express@10.0.3
npm ERR! node_modules/@nestjs/platform-express
npm ERR!   peerOptional @nestjs/platform-express@"^10.0.0" from
  @nestjs/core@10.0.3
npm ERR!   node_modules/@nestjs/core
npm ERR!     @nestjs/core@"10.0.3" from the root project
npm ERR!
npm ERR! Fix the upstream dependency conflict, or retry
npm ERR! this command with --force, or --legacy-peer-deps
npm ERR! to accept an incorrect (and potentially broken) dependency resolution.
npm ERR! See C:\Users\tuan.pv\AppData\Local\npm-cache\eresolve-report.txt for a full
report.
```

=> sử dụng **--legacy-peer-deps**

npm i --legacy-peer-deps

=====

> nest start --watch

```
node:internal/modules/cjs/loader:936
  throw err;
  ^
```

Error: Cannot find module 'webpack'

Require stack:

Nguyên nhân lỗi có thể là do conflict version của @nestjs/cli bên trong devDependencies và CLI cài đặt global.

Để check, kiểm tra version cài đặt global: `ncu -g`

Gỡ version global: `npm uninstall -g @nestjs/cli`

Cài đặt (upgrade) version global:

`npm i -g @nestjs/cli@10.0.3` => trùng version với devDependencies

Kiểm tra version cài đặt global: `npm list -g`

Trường hợp bị lỗi => gỡ version global

===

Lưu ý về mail template => lưu ý về đặt tên thư mục không chứa ký tự có dấu, ký tự đặc biệt => webpack sẽ không dịch code assets

Về source code video này:

Các file thay đổi tại video, xem [tại đây](#)

Source code ứng với video này, xem [tại đây](#)

Video hướng dẫn cách download source sau khi kết thúc video này, xem [tại đây](#)

#114. Build Docker

Yêu cầu: máy tính đã cài đặt docker và đã có kiến thức về docker

=> đây là kiến thức đi làm sẽ dùng.

render.com

- Không deploy lên host miễn phí, vì tốn time và thực tế không làm vậy

- Build docker image test local

Download file docker sử dụng trong video:

https://drive.google.com/file/d/1fwc_a4HhDgtq-HfSUBI6821bCoKArlIm/view?usp=sharing

Về source code video này:

Các file thay đổi tại video, xem [tại đây](#)

Source code ứng với video này, xem [tại đây](#)

Video hướng dẫn cách download source sau khi kết thúc video này, xem [tại đây](#)

#115. Nhận Xét Về Khóa Học

Học gì tiếp đây :v

Đừng quên review khóa học để nhận được ưu đãi cho khóa học kế tiếp các bạn nhé :v

Về các khóa học của Hỏi Dân IT, tham khảo [tại đây](#)

#116. Tự Học Fullstack Nest.js/Next.js (Extra)

Link series thực hành, xem miễn phí [tại đây](#)

Đánh Giá (Review/Rating) Khóa Học

1. Lời cảm ơn:

Mình rất hy vọng bạn thấy khóa học này có giá trị, nhưng dù thế nào đi nữa, hãy để lại đánh giá và chia sẻ trải nghiệm của bạn.

Hãy cho mình biết bạn nghĩ gì và viết đánh giá khi bạn có thể.

Mình cũng luôn sẵn sàng trả lời câu hỏi của bạn - hãy gửi tin nhắn trực tiếp cho mình bất cứ lúc nào.

2. Cách đánh giá (review) khóa học, tham khảo chi tiết [tại đây](#)

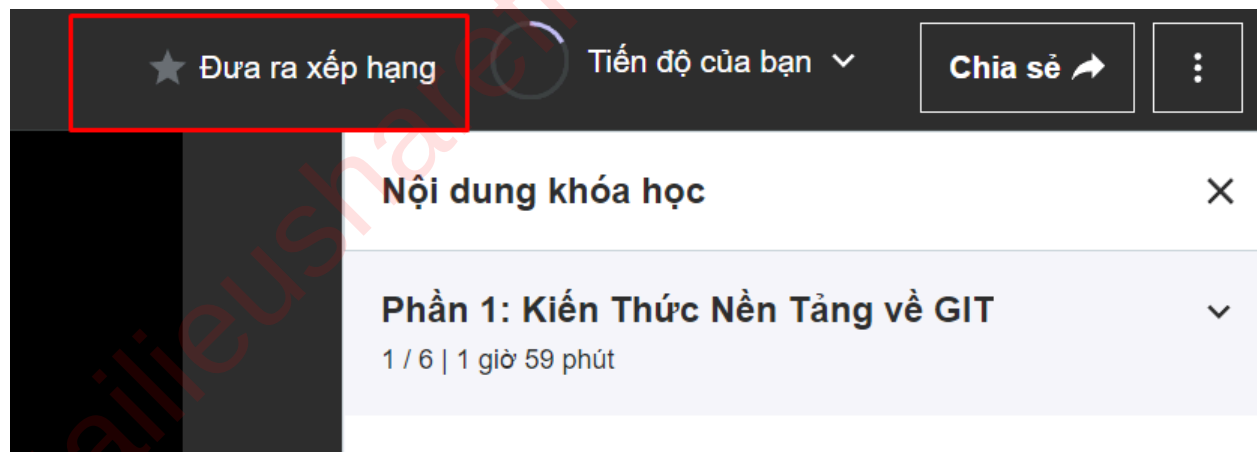
Tạo review đánh giá:

Nhấn vào "đưa ra xếp hạng". Sau đấy rating (star) và viết đánh giá chia sẻ trải nghiệm.

Nếu màn hình của bạn không hiển thị, có 2 nguyên nhân:

1 là bạn đã đánh giá rồi, xem mục 3 bên dưới để biết cách sửa/xóa đánh giá

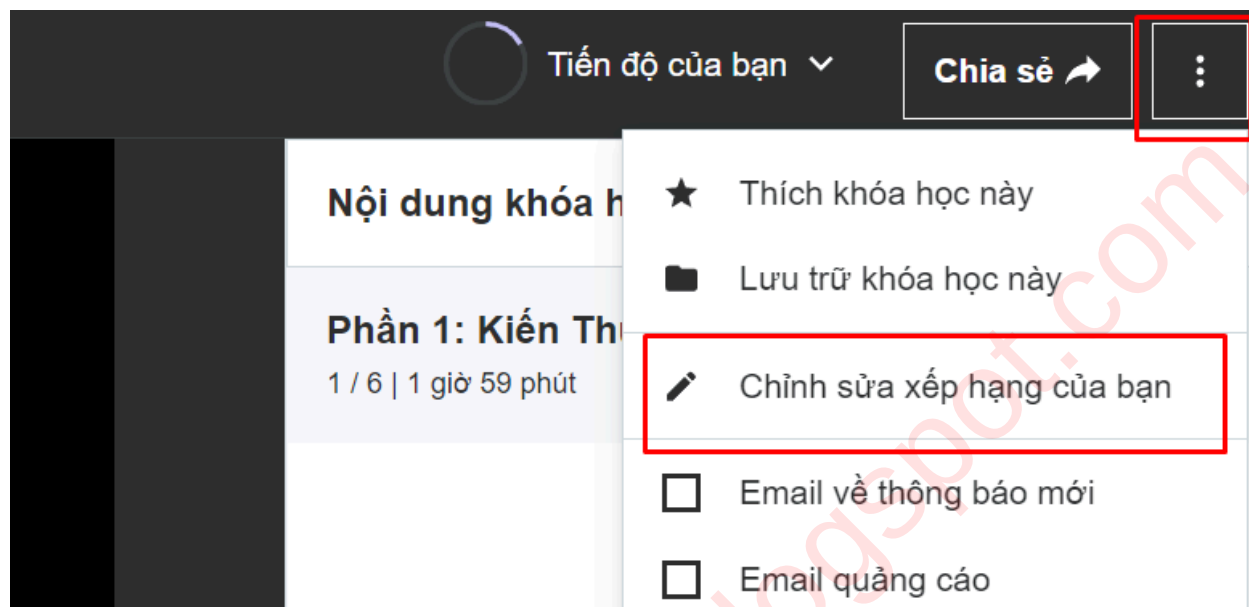
2 là thời gian bạn xem khóa học chưa đủ nên không thể đánh giá



3. Chỉnh sửa/xóa review đã đánh giá:

Bước 1: chọn vào hình 3 dấu chấm tại góc màn hình

Bước 2: chọn chỉnh sửa review



4. Mẹo để Viết Đánh Giá Tốt

Dưới đây là một số điều bạn nên lưu ý khi viết đánh giá cho một khóa học:

Hãy cho chúng tôi biết lý do: Ngoài việc để lại đánh giá sao, vui lòng chia sẻ suy nghĩ của bạn.

Ý kiến của bạn về khóa học rất có giá trị đối với những người học khác, nhưng khi chỉ có đánh giá sao, thật khó để người dùng khác hiểu vì sao bạn lại đưa ra đánh giá đó.

Cụ thể là tốt: Tính cụ thể giúp người học khác quyết định xem khóa học có phù hợp với họ không. Có điểm nào cần cải thiện không?

Khóa học có đáp ứng được kỳ vọng của bạn không?

Phần nào của khóa học khiến bạn thích nhất?

Trung thực: Đánh giá là một trong những yếu tố quan trọng nhất mà mọi người xem xét trước khi đăng ký một khóa học.

Tuy nhiên, đánh giá chỉ có giá trị khi bạn trung thực về cảm nhận của mình về khóa học.

Miễn là bạn chia sẻ cảm nhận một cách tôn trọng, ý kiến phản hồi của bạn rất có giá trị và hữu ích cho cộng đồng học tập của chúng tôi.

Lời Kết

Như vậy là chúng ta đã cùng nhau trải qua hơn 130+ videos về sử dụng framework NestJS dành cho backend NodeJS (typescript)

Tất cả các kiến thức mình chia sẻ, đều được lấy từ kinh nghiệm đi làm của mình và...

trang tài liệu NestJS: <https://docs.nestjs.com/>

Dĩ nhiên rằng, trong quá trình quá trình thực hiện khóa học này, mình sẽ không thể tránh khỏi những thiếu sót.

Vì vậy, nếu thấy sai sót, các bạn cứ thoải mái đóng góp qua Fanpage Hỏi Dân IT nhé.

<https://www.facebook.com/askITwithERIC>

Nếu bạn thấy khóa học này hữu ích, đừng quên Review đánh giá trên Udemy nhé ^^

Hẹn gặp lại các bạn ở các khóa học tiếp theo

Hỏi Dân IT (Eric)